

Università
Ca'Foscari
Venezia

Corso di Laurea magistrale
in Economia e Finanza

Tesi di Laurea

Volatility Forecasting: confronto tra modelli GARCH e architetture neurali

Relatore
Ch. Prof. Claudio Pizzi

Laureando
Luca Celio
Matricola 885954

Anno Accademico
2024 / 2025

Indice

Introduzione **6**

1 La volatilità nei mercati finanziari **7**
1.1 Volatilità come misura di incertezza e rischio 7
1.2 Volatilità realizzata 8
1.3 Volatilità implicita 9
1.4 Proprietà empiriche della volatilità 11

2 Modelli econometrici per la volatilità condizionata **17**
2.1 Il modello ARCH(q) di Engle 17
2.2 Il modello GARCH(p,q) di Bollerslev 18
2.3 Estensioni asimmetriche dei modelli GARCH 19
2.4 Distribuzioni per le innovazioni nei modelli GARCH 20

3 Fondamenti delle reti neurali artificiali **22**
3.1 Struttura e funzionamento di un neurone artificiale 22
3.2 Funzioni di attivazione 24
3.3 Costruzione di una rete neurale 26
3.4 Apprendimento supervisionato dei parametri 28
3.4.1 Funzioni di perdita per la regressione 29
3.4.2 Backpropagation e calcolo del gradiente 30
3.4.3 Algoritmi di ottimizzazione 31
3.5 Overfitting e tecniche di regolarizzazione 34
3.6 I tensori e la rappresentazione dei dati 36

4 Architetture neurali per la previsione della volatilità **38**
4.1 Limiti delle reti feedforward e modellazione di dati sequenziali 39
4.2 Recurrent Neural Networks: struttura e funzionamento 42
4.3 Architetture delle RNN: strutture input-output 44
4.4 Apprendimento nelle RNN: Backpropagation Through Time 45
4.5 Oltre le RNN: le motivazioni 46
4.6 LSTM: Long Short-Term Memory 48
4.7 GRU: Gated Recurrent Units 50
4.8 CNN-1D: reti neurali convoluzionali monodimensionali 51
4.9 N-BEATS: un approccio avanzato per il forecasting 53

5 Confronto empirico dei modelli **56**
5.1 Dataset e analisi preliminare 56
5.2 Stima e risultati dei modelli GARCH 61
5.2.1 Modello per la media 61
5.2.2 Modello per la volatilità 61
5.2.3 Stima del modello GARCH(1,1) 63
5.2.4 Distribuzioni alternative 65
5.2.5 Modelli GARCH asimmetrici 66
5.2.6 Analisi predittiva 68
5.3 Risultati dei modelli neurali 69
5.4 Confronto complessivo e interpretazione dei risultati 81
5.5 Generalizzazione dei risultati 83

Conclusione **88**

Bibliografia **90**

A. Grafici **94**

B. Codici R e Python **103**

Elenco delle figure **111**

Elenco delle tabelle **113**

Elenco dei simboli **114**

Introduzione

La volatilità rappresenta una delle grandezze centrali nella teoria e nella pratica dei mercati finanziari. Essa misura il grado di incertezza e di rischio associato ai rendimenti di un'attività finanziaria, influenzando la valutazione degli strumenti derivati, la gestione del portafoglio e le politiche di controllo del rischio. Comprendere e prevedere l'andamento della volatilità è quindi fondamentale per una corretta interpretazione delle dinamiche di mercato e per l'elaborazione di strategie di investimento e copertura efficienti.

La letteratura econometrica ha sviluppato nel tempo numerosi modelli volti a descrivere l'evoluzione della volatilità condizionata. Tra questi, i modelli della famiglia ARCH-GARCH, introdotti da Engle e Bollerslev, rappresentano uno dei contributi più significativi, poiché consentono di catturare fenomeni empirici quali la persistenza e il *clustering* della volatilità. Tuttavia, tali modelli si basano su specifiche funzionali relativamente rigide e possono risultare limitati nel rappresentare relazioni non lineari e strutture dinamiche complesse nei dati finanziari.

Negli ultimi anni, l'ampia disponibilità di dati e i progressi nelle tecniche di *machine learning*, in particolare nel *deep learning*, hanno favorito l'impiego di modelli di apprendimento automatico per la previsione della volatilità. Le reti neurali, in particolare, offrono un approccio flessibile e non parametrico, capace di apprendere relazioni non lineari e dipendenze temporali di diversa natura. Ciò ha suscitato un crescente interesse nel confronto tra modelli econometrici tradizionali e architetture neurali, con l'obiettivo di valutarne l'efficacia e la robustezza in contesti di previsione finanziaria.

Il presente lavoro si inserisce in questo filone di ricerca e si propone di confrontare la capacità previsiva di modelli appartenenti a due approcci distinti: da un lato, i modelli econometrici della famiglia GARCH, e dall'altro, diverse architetture di reti neurali, quali *Feedforward*, *Recurrent* e *Convolutional Neural Networks*.

La tesi si articola in due parti principali. La prima parte, di natura teorica, introduce il concetto di volatilità, ne descrive le principali misure e proprietà empiriche, e presenta i modelli di riferimento utilizzati per la sua modellazione. La seconda parte è dedicata all'analisi empirica, nella quale vengono stimati e confrontati i diversi modelli in termini di accuratezza previsiva, stabilità e capacità di adattamento a contesti di mercato eterogenei. I codici utilizzati per l'implementazione dei modelli neurali sono sviluppati in `Python`, mentre la stima dei modelli GARCH è condotta in `R`; tutti i dati impiegati nell'analisi empirica sono estratti da Bloomberg.

Capitolo 1

La volatilità nei mercati finanziari

1.1 Volatilità come misura di incertezza e rischio

In finanza, la volatilità rappresenta una misura dell'incertezza associata al tasso di rendimento di un'attività finanziaria [21]. Essa quantifica la dispersione dei rendimenti intorno al loro valore medio, fornendo un'indicazione del rischio connesso alla variabilità del processo dei rendimenti nel tempo. La volatilità può quindi essere interpretata come l'intensità delle fluttuazioni dei rendimenti e, di conseguenza, come un indice della stabilità o instabilità dei prezzi di mercato.

Indicando con R_t il rendimento di un'attività finanziaria e con $\mu = \mathbb{E}[R_t]$ il suo valore atteso, la varianza del rendimento è definita come:

$$\mathrm{Var}(R_t) = \mathbb{E}\left[(R_t - \mu)^2\right] = \sigma^2. \tag{1.1}$$

La volatilità è comunemente misurata tramite la deviazione standard del rendimento,

$$\sigma = \sqrt{\sigma^2}.$$

Poiché la volatilità non è direttamente osservabile, ma rappresenta una caratteristica latente del processo dei rendimenti, l'analisi finanziaria fa ampio ricorso a misure di approssimazione, o *proxy*, della volatilità attesa su orizzonti futuri.

Nella pratica empirica, una prima misura di volatilità è rappresentata dalla volatilità storica (*historical volatility*, HV), calcolata come:

$$\sigma_{\mathrm{HV}} = \sqrt{\frac{1}{T-1}\sum_{t=1}^{T}(R_t - \bar{R})^2}, \tag{1.2}$$

dove $\bar{R}$ indica il rendimento medio campionario.

Tuttavia, il calcolo della volatilità storica richiede la stima del rendimento medio, la quale risulta spesso affetta da un'elevata variabilità, soprattutto in campioni di dimensioni limitate. Inoltre, anche all'aumentare della dimensione campionaria, periodi caratterizzati da andamenti ribassisti dei prezzi possono generare rendimenti medi negativi, in contrasto con il vincolo di non negatività comunemente imposto al rendimento atteso *ex ante* [32]. Alla luce di tali criticità, è prassi diffusa porre il rendimento medio pari a zero, ottenendo una misura semplificata della variabilità basata esclusivamente sulla somma dei rendimenti al quadrato. Sebbene tale quantità sia stata talvolta indicata in letteratura come volatilità realizzata, nel presente lavoro con tale termine si farà riferimento esclusivamente alla definizione moderna introdotta da

Andersen e Bollerslev [1], fondata sulla variazione quadratica del processo dei prezzi (cfr. Sezione 1.2). In questo contesto, la volatilità realizzata e la volatilità implicita rappresentano due approcci complementari: la prima fornisce una misura *ex post* della variabilità effettivamente osservata, mentre la seconda sintetizza le aspettative del mercato circa la variabilità futura dei rendimenti, incorporando le condizioni correnti di mercato riflesse nei prezzi degli strumenti derivati.

1.2 Volatilità realizzata

La volatilità realizzata, o *realized volatility* (RV), rappresenta una misura *backward-looking* della variabilità dei rendimenti costruita a partire da osservazioni storiche dei prezzi e quantifica l'incertezza effettivamente osservata su un dato orizzonte temporale. A differenza della volatilità implicita, che riflette le aspettative del mercato sulla variabilità futura, la volatilità realizzata si basa esclusivamente sull'informazione storica disponibile.

Dal punto di vista teorico, l'intuizione di Andersen e Bollerslev, resa possibile dalla disponibilità di dati ad alta frequenza, trova fondamento nel contesto dei modelli a tempo continuo, nei quali la volatilità è strettamente connessa alla variazione quadratica (*quadratic variation*, QV) del processo dei prezzi. Si assume che il prezzo di un'attività finanziaria evolva secondo un processo di diffusione continuo, noto come moto browniano geometrico (GBM), descritto da:

$$dS = \mu S\,dt + \sigma S\,dW, \tag{1.3}$$

dove μ rappresenta il termine di drift, σ la volatilità istantanea e W un moto browniano standard [21, cap. 14]. Definendo il log-prezzo come $G = \ln S$, l'applicazione del Lemma di Itô conduce al processo:

$$dG = \left(\mu - \tfrac{1}{2}\sigma^2\right) dt + \sigma\,dW. \tag{1.4}$$

Il rendimento continuamente composto osservato su un orizzonte temporale di ampiezza $h > 0$ può quindi essere espresso come:

$$G(t) - G(t-h) = \int_{t-h}^{t} \left(\mu(\tau) - \tfrac{1}{2}\sigma^2(\tau)\right) d\tau + \int_{t-h}^{t} \sigma(\tau)\,dW(\tau). \tag{1.5}$$

Per il processo del log-prezzo G, la variazione quadratica sull'intervallo $[t-h,t]$ è definita come:

$$QV(t,h) = \int_{t-h}^{t} \sigma^2(\tau)\,d\tau. \tag{1.6}$$

Tale quantità misura esclusivamente la variabilità generata dalla componente stocastica del processo, poiché il termine di drift non contribuisce alla QV: essa rappresenta pertanto una misura teorica della volatilità integrata del processo dei rendimenti.

Poiché la QV non è direttamente osservabile, in ambito empirico essa viene approssimata mediante osservazioni discrete dei rendimenti. Considerando una partizione regolare dell'intervallo temporale $[t-h,t]$ in sottointervalli di ampiezza $1/n$, una misura discreta di volatilità realizzata è data da:

$$RV(t,h;n) = \sqrt{\sum_{j=1}^{nh} r^2\left(t-h+\frac{j}{n},\frac{1}{n}\right)}. \tag{1.7}$$

Sotto l'ipotesi di rendimenti serialmente incorrelati e di continuità del processo di volatilità, la teoria [23] dimostra che:

$$RV^2(t,h;n) \xrightarrow[n\to\infty]{p} QV(t,h). \tag{1.8}$$

Questo risultato fornisce il fondamento teorico dell'utilizzo della somma dei quadrati dei rendimenti come proxy empirica della volatilità integrata.

In linea teorica, l'accuratezza della volatilità realizzata aumenta al crescere della frequenza di campionamento, rendendo l'utilizzo di dati intraday particolarmente attraente. Tuttavia, l'impiego di dati ad alta frequenza comporta anche criticità operative, tra cui la presenza di rumore di microstruttura. Per tali ragioni, e in coerenza con numerosi studi applicativi, nel presente lavoro la volatilità realizzata è costruita a partire da rendimenti giornalieri e successivamente aggregata a frequenza mensile.

Indicando con $r_{d,m}$ il rendimento logaritmico osservato nel giorno d appartenente al mese m, la volatilità realizzata mensile è definita come:

$$\mathrm{RV}_m = \sqrt{\frac{252}{n_m} \sum_{d=1}^{n_m} r_{d,m}^2}, \tag{1.9}$$

dove n_m rappresenta il numero di giorni di negoziazione nel mese m e il fattore 252 consente l'annualizzazione della misura secondo la convenzione di mercato.

Nel contesto della previsione della volatilità, la RV rappresenta una proxy ampiamente utilizzata per valutare l'accuratezza dei modelli previsivi. Come evidenziato da Poon e Granger [36], essa costituisce un benchmark naturale per il confronto tra modelli econometrici e approcci alternativi, in quanto fornisce una misura non parametrica della variabilità effettivamente osservata.

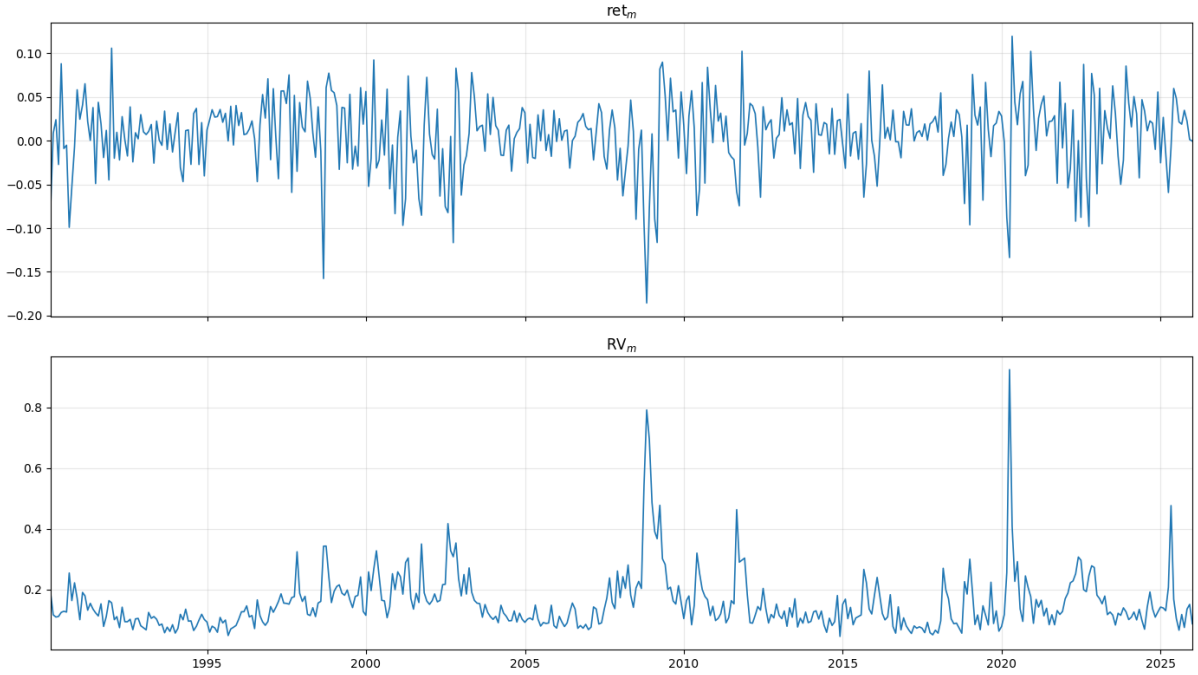


Figura 1.1: Rendimenti logaritmici mensili e volatilità realizzata mensile dell'S&P 500.

1.3 Volatilità implicita

La volatilità implicita o *implied volatility* (IV) è una misura prospettica, quindi "rivolta al futuro" della volatilità, ricavata dai prezzi delle opzioni osservati sul mercato. Essa riflette le aspettative degli operatori riguardo alla variabilità futura del prezzo dell'attività sottostante e costituisce una stima di consenso del mercato sul livello di rischio atteso.

Nel modello di *Black–Scholes–Merton* (BSM), la volatilità è un parametro esogeno che influisce direttamente sul valore teorico di un'opzione. Poiché il prezzo di mercato incorpora le forze di domanda e offerta, è possibile determinare, per inversione, il valore di σ che rende coerente il prezzo teorico con quello osservato sul mercato. Tale valore prende il nome di volatilità implicita e viene indicato con IV. Il modello BSM per il prezzo di un'opzione call europea [21, cap. 15] è descritto dall'equazione alle derivate parziali:

$$\frac{\partial C}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 C}{\partial S^2} + rS\frac{\partial C}{\partial S} - rC = 0, \tag{1.10}$$

che, sotto le ipotesi di assenza di arbitraggio, mercati completi e rendimenti log-normali, ammette la soluzione analitica:

$$C_{BS}(S_0, K, r, T, \sigma) = S_0 N(d_1) - Ke^{-rT} N(d_2), \tag{1.11}$$

$$d_1 = \frac{\ln(S_0/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}. \tag{1.12}$$

La volatilità implicita IV è definita come il valore del parametro di volatilità che, inserito nel modello di BSM, rende il prezzo teorico dell'opzione uguale al prezzo osservato sul mercato, ovvero come la soluzione dell'equazione

$$C_{BS}(S_0, K, T, r, \sigma_{IV}) = C_{\text{market}}, \tag{1.13}$$

dove C_{market} indica il prezzo dell'opzione osservato sul mercato, mentre C_{BS} rappresenta il valore teorico determinato dal modello di Black–Scholes–Merton. Poiché tale modello non ammette una soluzione analitica inversa per la volatilità, il valore di IV viene ottenuto numericamente mediante algoritmi di ricerca delle radici, quali il metodo di *Newton–Raphson*.

Per azioni e indici azionari, la volatilità implicita può essere calcolata a partire dai prezzi delle opzioni negoziate sul mercato. Nel caso degli indici azionari, sono inoltre disponibili misure sintetiche già costruite che riassumono l'informazione contenuta in un ampio insieme di contratti derivati.

Per l'S&P 500, l'indice *VIX*, pubblicato dalla CBOE (*Chicago Board Options Exchange*), rappresenta una misura sintetica della volatilità implicita a 30 giorni, ottenuta da un paniere di opzioni sull'indice (Figura 1.2).

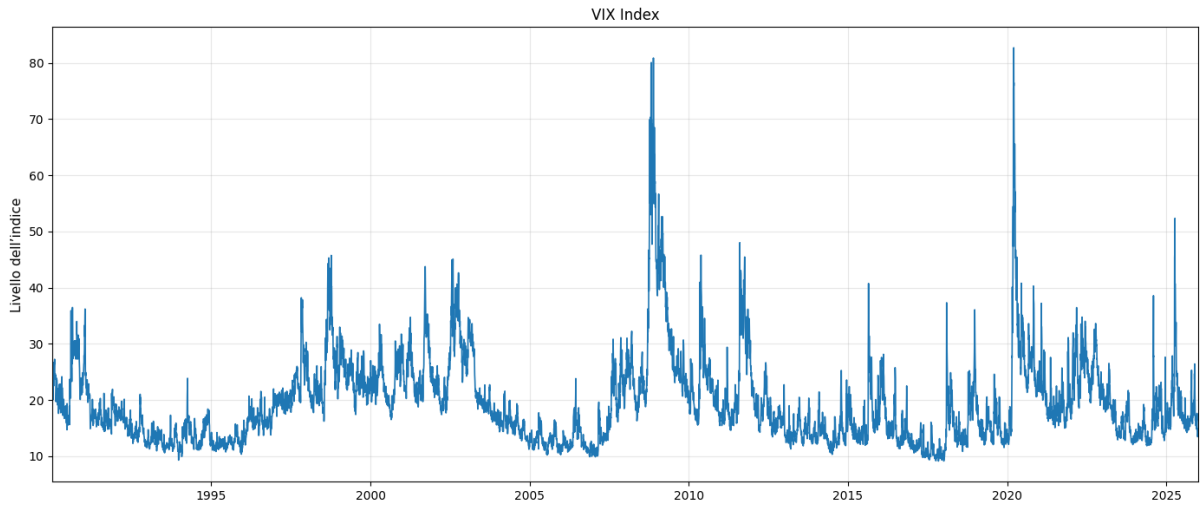


Figura 1.2: Andamento giornaliero dell'indice VIX nel periodo 1990–2025.

Oltre al VIX, esistono altri indici di volatilità implicita sviluppati per diverse asset class. Ad esempio, il *VXN* misura la volatilità attesa sulle opzioni del NASDAQ 100, mentre il *VDAX–NEW* rappresenta l'analogo indicatore calcolato dalla Deutsche Börse per il mercato azionario tedesco (indice DAX 40). Per l'Eurostoxx 50, la volatilità implicita è sintetizzata dallo *VSTOXX*, pubblicato da Eurex, che riflette le aspettative di rischio sul principale indice europeo. Sono inoltre disponibili misure analoghe per il mercato valutario (*CBOE EuroCurrency Volatility Index*) e per quello obbligazionario (*MOVE Index*), che forniscono un'indicazione prospettica dell'incertezza attesa rispettivamente sui cambi e sui tassi d'interesse. Questi indici, costruiti con metodologie simili a quella del VIX, rappresentano strumenti standardizzati e comparabili per la misurazione della volatilità implicita nei diversi mercati finanziari.

La volatilità implicita fornisce quindi una stima delle aspettative del mercato sulla variabilità futura dei rendimenti del sottostante, poiché riflette le valutazioni reali degli investitori nel momento in cui acquistano o vendono opzioni. Essa è pertanto una misura *forward-looking*, complementare alla volatilità realizzata, che invece si basa su dati storici osservati.

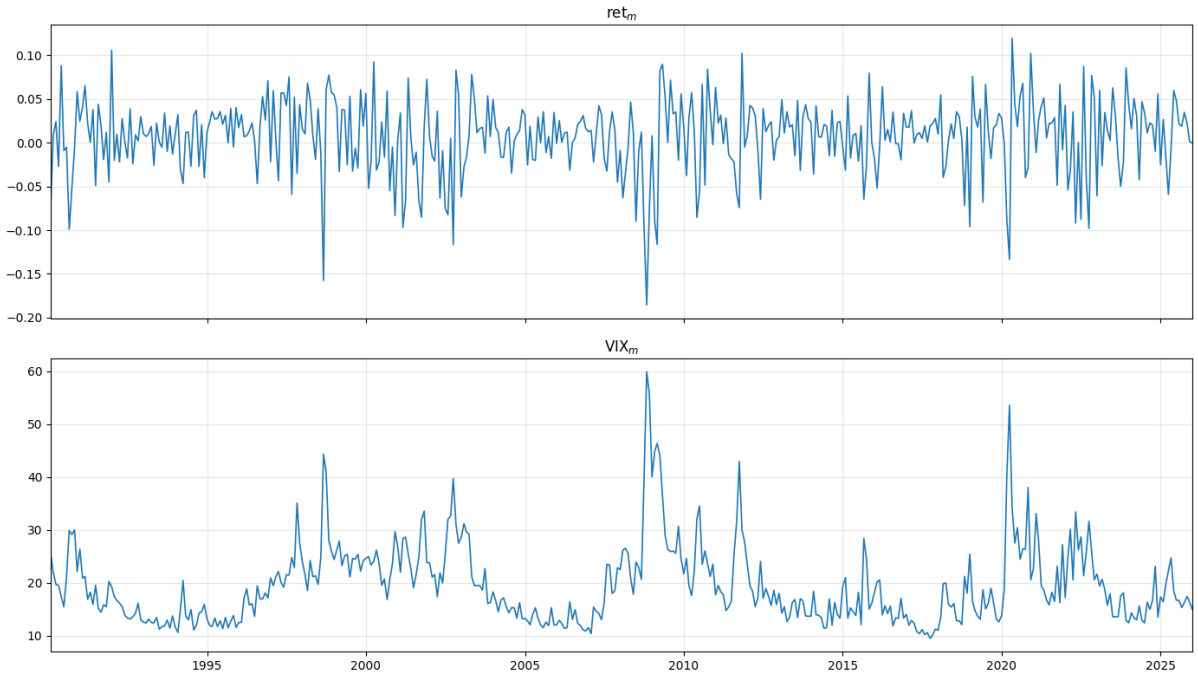


Figura 1.3: Rendimenti mensili dell'S&P 500 e indice VIX.

1.4 Proprietà empiriche della volatilità

L'analisi empirica dei rendimenti finanziari ha evidenziato nel tempo una serie di regolarità statistiche comuni a diversi mercati e orizzonti temporali, note in letteratura come *stylized facts*. Tali proprietà descrivono il comportamento caratteristico della volatilità e ne giustificano la modellizzazione tramite processi a varianza condizionata. Un buon modello di volatilità deve quindi essere in grado di catturare e riprodurre tali caratteristiche. Nella presente sezione, l'analisi è condotta su dati relativi al periodo 1990–2025.

Heavy Tails

La distribuzione dei rendimenti finanziari si discosta dalla normale teorica, mostrando code pesanti (*heavy tails*) che indicano una frequenza maggiore di variazioni estreme rispetto a quanto previsto da un modello gaussiano. Questo fenomeno, associato a un eccesso di curtosi, riflette la maggiore probabilità di eventi anomali e implica una possibile sottostima del rischio di coda se si assume una distribuzione normale dei rendimenti. La Figura 1.4 mostra l'istogramma dei rendimenti logaritmici giornalieri confrontato con la densità normale teorica. L'elevata concentrazione centrale e le code più spesse della distribuzione empirica sono confermate dal valore della curtosi empirica (kurt = 13.88), nettamente superiore a quella teorica della distribuzione normale (kurt = 3). Questa evidenza empirica dimostra la natura leptocurtica dei rendimenti, tipica dei mercati finanziari.

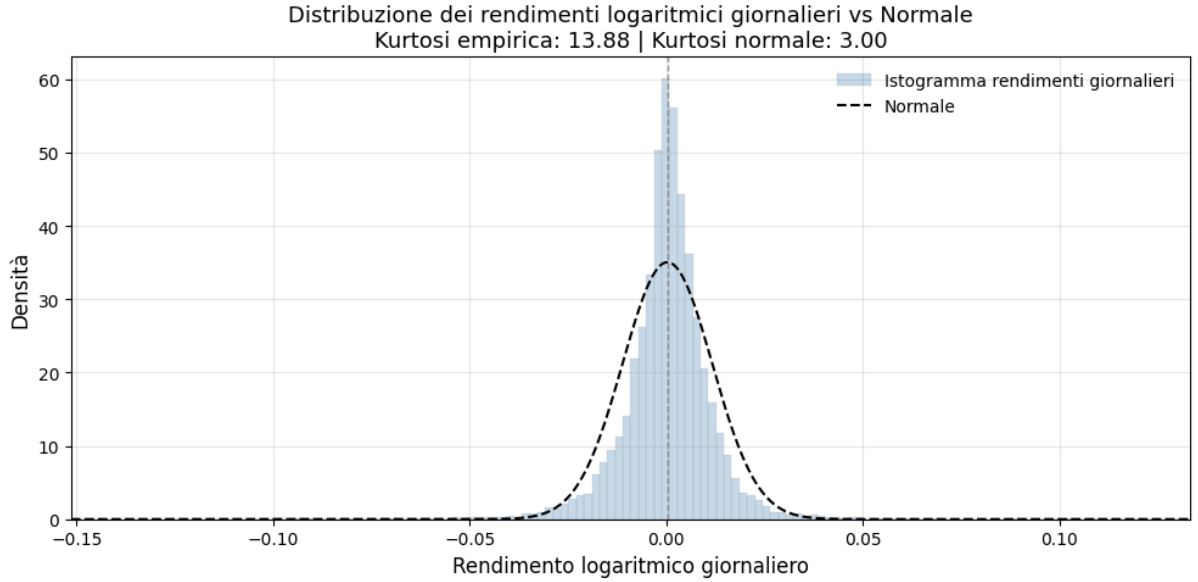


Figura 1.4: Distribuzione dei rendimenti logaritmici giornalieri.

Il comportamento non gaussiano dei rendimenti è confermato anche dal Q–Q plot (Figura 1.5).

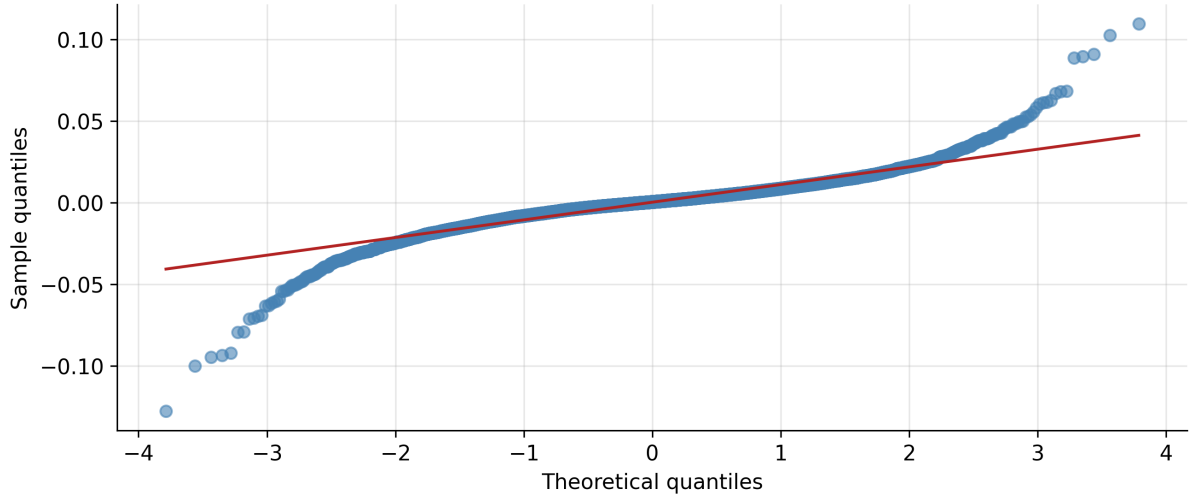


Figura 1.5: Q–Q plot dei rendimenti logaritmici giornalieri.

Volatility Clustering

La volatilità dei rendimenti non è costante nel tempo: periodi di forte turbolenza tendono a concentrarsi e a essere seguiti da altri periodi di alta volatilità, mentre fasi di calma risultano più persistenti. Questo fenomeno, noto come *volatility clustering*, riflette la presenza di persistenza e autocorrelazione nella varianza condizionata, in contrasto con l'ipotesi di indipendenza seriale.

La Figura 1.6 riporta lo *scatter plot* tra la volatilità realizzata nel mese t e quella del mese successivo $t+1$. L'andamento crescente della nuvola di punti, insieme a un coefficiente di determinazione ($R^2 = 0.44$) e a un coefficiente di autocorrelazione di primo ordine ($\rho_1 = 0.66$), evidenzia una forte persistenza: mesi con volatilità elevata tendono a essere seguiti da mesi ugualmente turbolenti, e viceversa.

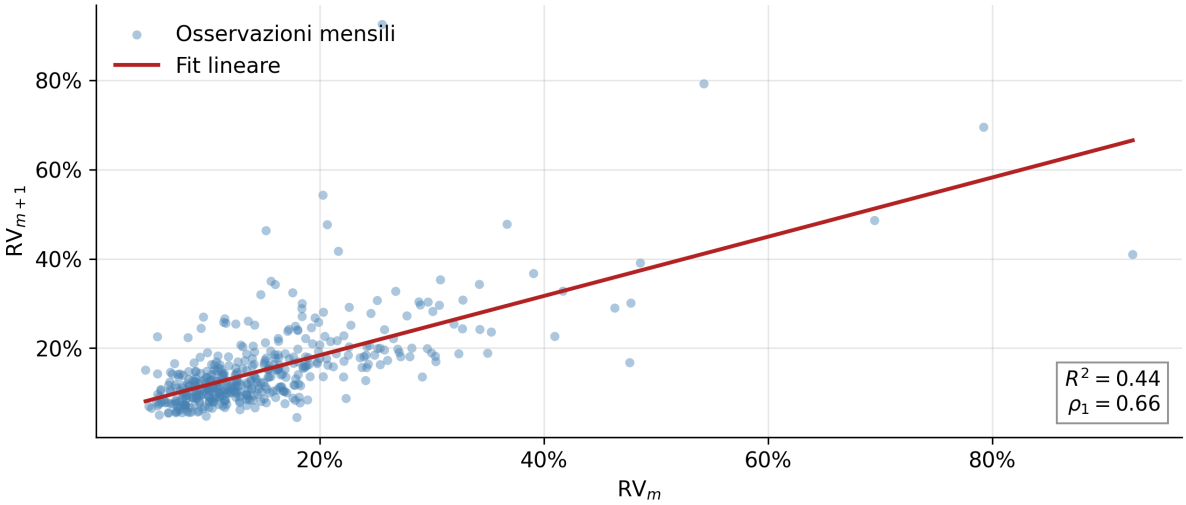


Figura 1.6: Relazione tra RV_m e RV_{m+1}.

Mean Reversion

Pur mostrando evidenti fenomeni di volatility clustering, la volatilità tende nel lungo periodo a convergere verso un livello medio di equilibrio. Gli episodi di volatilità estremamente elevata o particolarmente contenuta risultano pertanto transitori, dando luogo a un comportamento noto come *mean reversion*.

La Figura 1.7 fornisce un'evidenza empirica di tale proprietà analizzando l'evoluzione della volatilità realizzata mensile nei mesi successivi a condizioni iniziali estreme. In particolare, il gruppo *Top 10%* include i mesi in cui la volatilità osservata appartiene al 10% più alto della distribuzione storica, mentre il *Bottom 10%* comprende i mesi caratterizzati dai valori di volatilità più bassi (10% inferiore della distribuzione). Per ciascun gruppo viene tracciato il percorso medio della volatilità nei mesi successivi all'osservazione iniziale. Questo comportamento conferma la natura mean-reverting della volatilità e fornisce una motivazione empirica per l'impiego di modelli dinamici in grado di catturarne il processo di aggiustamento nel tempo.

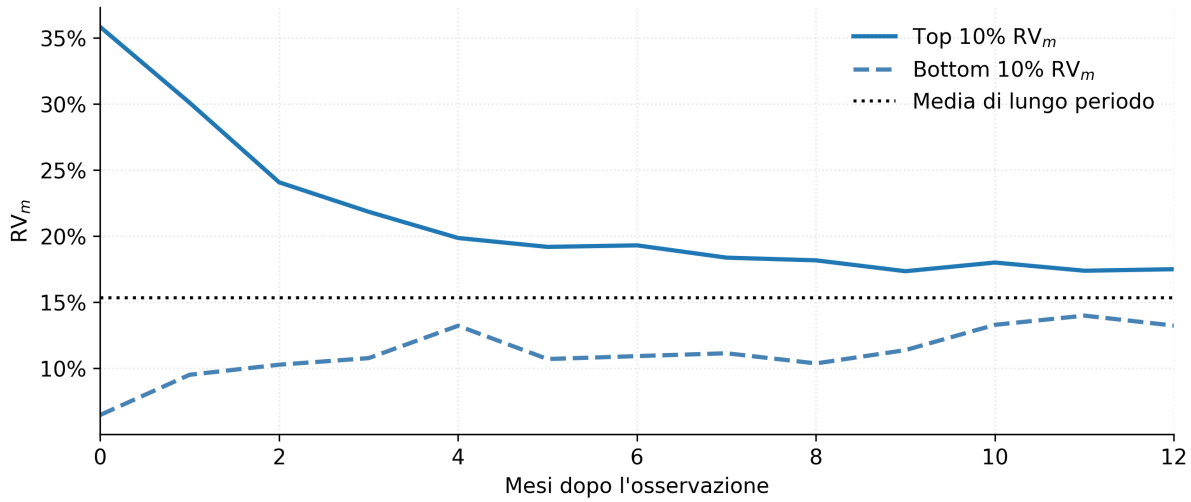


Figura 1.7: Evidenza del fenomeno di mean reversion nella RV$_m$.

Leverage Effect

Nei mercati finanziari si osserva il *leverage effect* [5]: i rendimenti negativi determinano un aumento della volatilità più pronunciato rispetto a variazioni positive di pari entità.

La Figura 1.8 mostra la relazione tra i rendimenti giornalieri dell'S&P 500 e la variazione percentuale del VIX. Si osserva una chiara relazione negativa e non lineare: i ribassi marcati si associano ad aumenti della volatilità implicita, mentre le fasi di rialzo coincidono con riduzioni del VIX. È opportuno ricordare che, pur essendo una proxy pratica del fenomeno, il VIX riflette anche effetti di *volatility feedback* e di variazione del premio per il rischio.

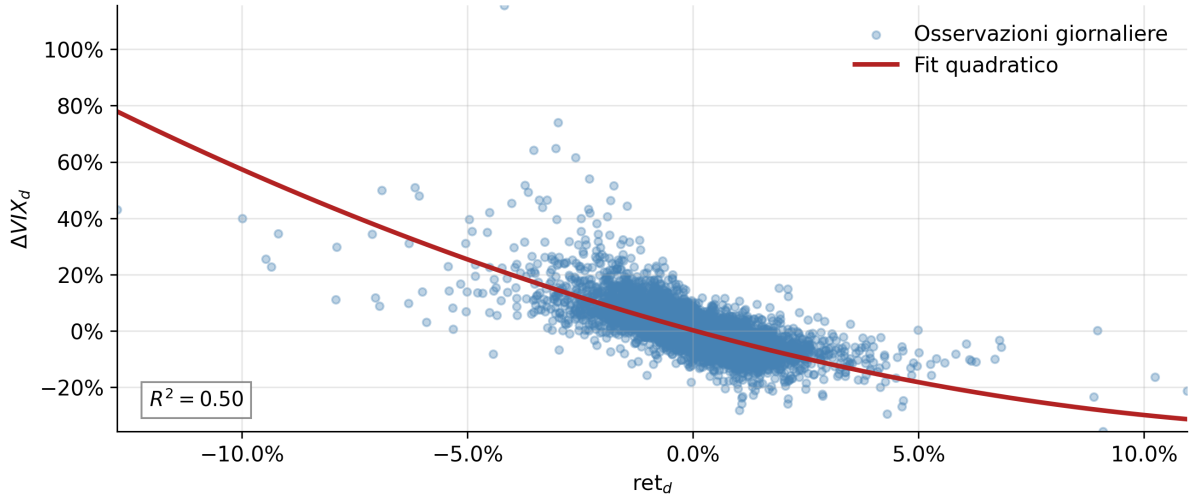


Figura 1.8: Relazione tra rendimenti logaritmici giornalieri e variazione del VIX.

Long Memory

Un'ulteriore caratteristica empirica della volatilità è la presenza di una *long memory*, ossia una dipendenza persistente nel tempo. In altri termini, gli shock alla volatilità non si esauriscono rapidamente, ma tendono a propagarsi su orizzonti temporali prolungati, influenzando anche le osservazioni future.

Questa proprietà emerge chiaramente analizzando la funzione di autocorrelazione della volatilità realizzata settimanale. Come mostrato in Figura 1.9, i coefficienti di autocorrelazione rimangono positivi e statisticamente significativi anche per lag elevati (diverse decine di settimane), decrescendo lentamente anziché annullarsi rapidamente. Tale andamento è indicativo di una persistenza di lungo periodo nel processo di volatilità, incompatibile con un modello a memoria corta.

La presenza di long memory giustifica l'impiego di modelli a varianza condizionata dinamica, come quelli della famiglia *ARCH–GARCH*. In letteratura sono state inoltre proposte estensioni specifiche per catturare questa persistenza di lungo periodo, tra cui i modelli *FIGARCH* e le specificazioni basate sulla volatilità realizzata, come il modello *HAR-RV*, che consentono di modellare efficacemente la dipendenza della volatilità su più orizzonti temporali.

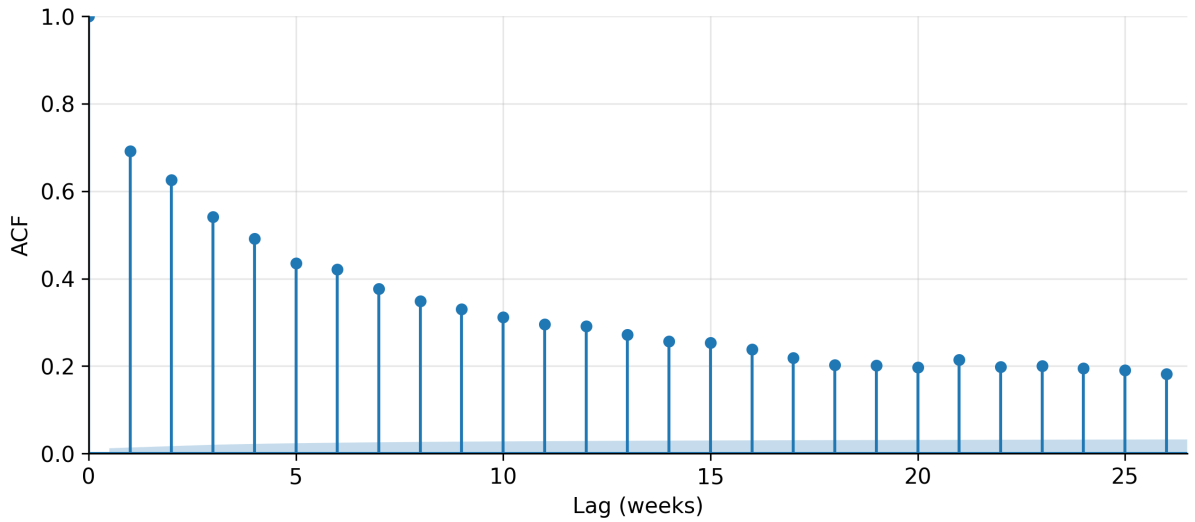


Figura 1.9: ACF della volatilità realizzata settimanale RV_w.

Volatility Risk Premium

Un aspetto centrale nell'analisi della volatilità riguarda la differenza sistematica tra la volatilità implicita e la volatilità realizzata, comunemente indicata come *Volatility Risk Premium* (VRP) [8]. Dal punto di vista empirico, la volatilità implicita tende a collocarsi, in media, su livelli superiori rispetto alla volatilità successivamente osservata, dando origine a un differenziale positivo persistente nel tempo.

Tale differenziale può essere interpretato come un premio per il rischio di volatilità, che riflette la compensazione richiesta per l'assunzione di esposizione a variazioni estreme dell'incertezza. Poiché le opzioni forniscono protezione contro perdite rilevanti in fasi di turbolenza dei mercati, i venditori di opzioni richiedono un prezzo più elevato per offrire tale copertura, al fine di essere remunerati per il rischio di shock improvvisi e non diversificabili. Il VRP può pertanto essere inteso come un premio assicurativo incorporato nei prezzi delle opzioni e sostenuto dagli investitori che domandano protezione contro scenari avversi.

La Figura 1.10 mostra l'andamento congiunto della volatilità implicita e della volatilità realizzata per l'indice S&P 500. Ai fini della rappresentazione grafica, entrambe le serie sono state smussate mediante una media mobile a 3 mesi. L'area ombreggiata rappresenta il Volatility Risk Premium, definito come la differenza tra la volatilità implicita e quella realizzata.

Dal grafico emerge come il VRP risulti mediamente positivo lungo l'intero campione e presenti una marcata variabilità nel tempo, tendendo ad ampliarsi in corrispondenza di fasi di

elevata incertezza o stress finanziario. Ciò suggerisce che il mercato incorpori nei prezzi delle opzioni una valutazione prospettica dell'incertezza futura che include, oltre alle aspettative sulla volatilità, anche una compensazione per il rischio associato a eventi estremi.

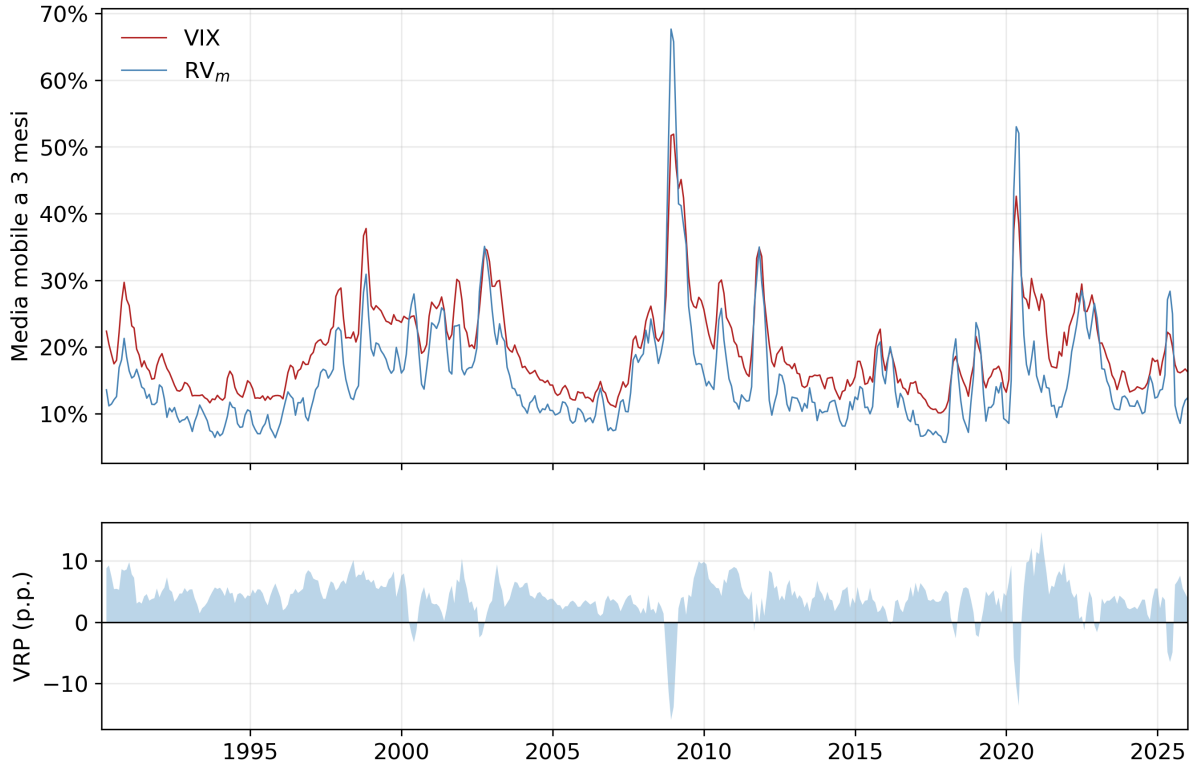


Figura 1.10: Volatility Risk Premium.

Il confronto mette infine in luce come la volatilità rappresenti una grandezza dinamica e non direttamente osservabile, la cui modellizzazione richiede strumenti in grado di catturarne la persistenza e le variazioni nel tempo. Nei capitoli successivi verranno pertanto presentati i modelli econometrici e le architetture neurali impiegate per la previsione della volatilità realizzata, al fine di valutarne la capacità predittiva e confrontarne l'efficacia.

Capitolo 2

Modelli econometrici per la volatilità condizionata

Dopo aver definito nel capitolo precedente le diverse misure di volatilità e le loro proprietà empiriche, si passa ora alla modellazione statistica della sua dinamica nel tempo. I modelli a varianza condizionata, introdotti da Engle (1982) e successivamente estesi da Bollerslev (1986), costituiscono il principale strumento econometrico per descrivere e prevedere la volatilità dei rendimenti finanziari.

La trattazione che segue fornisce una sintesi operativa dei principali modelli della famiglia GARCH, con l'obiettivo di inquadrare le specificazioni utilizzate nell'analisi empirica. Per una discussione esaustiva si rimanda, tra gli altri, a McNeil, Frey e Embrechts [31] e Hull [21].

2.1 Il modello ARCH(q) di Engle

Il modello *Autoregressive Conditional Heteroskedasticity* (ARCH), proposto da Engle [13], rappresenta il primo approccio econometrico in grado di catturare in modo esplicito la natura variabile della volatilità nel tempo. Prima dell'introduzione di questo modello, si assumeva comunemente che la varianza degli errori fosse costante (omoschedasticità), ipotesi che tuttavia si dimostrava inadeguata per le serie finanziarie, nelle quali la volatilità mostra tipicamente fasi di calma seguite da periodi di forte turbolenza (cfr. Sezione 1.4) [10].

Il termine Autoregressive Conditional Heteroskedasticity riflette le tre caratteristiche principali del modello: *autoregressive* poiché la varianza dipende dai propri valori passati; *conditional* perché è stimata condizionatamente all'informazione disponibile fino al tempo $t-1$; *heteroskedastic* in quanto la varianza non è costante nel tempo, ma varia in risposta agli shock di mercato. In particolare, la varianza condizionata può essere formalmente definita come $\sigma_t^2 = \mathrm{Var}(\varepsilon_t \mid \mathscr{F}_{t-1})$, dove $\mathscr{F}_{t-1}$ rappresenta l'insieme delle informazioni disponibili fino al tempo $t-1$.

Il modello ARCH parte dall'idea che la varianza condizionata dell'errore al tempo t dipenda linearmente dai quadrati dei residui osservati nei periodi precedenti. Formalmente, un modello ARCH(q) può essere scritto come:

$$\begin{cases} r_t = \mu + \varepsilon_t, \\ \varepsilon_t = \sigma_t z_t, \quad z_t \sim i.i.d., \ \mathbb{E}[z_t] = 0, \ \mathbb{V}[z_t] = 1, \\ \sigma_t^2 = \alpha_0 + \alpha_1 \varepsilon_{t-1}^2 + \alpha_2 \varepsilon_{t-2}^2 + \cdots + \alpha_q \varepsilon_{t-q}^2, \end{cases} \tag{2.1}$$

dove:

- r_t rappresenta il rendimento osservato o il residuo di un modello di media condizionata;
- ε_t è l'innovazione (errore) al tempo t;
- σ_t^2 è la varianza condizionata;
- z_t è un termine di rumore bianco standardizzato;
- i coefficienti α_i devono essere non negativi e $\alpha_0 > 0$ per garantire la positività della varianza condizionata, mentre la condizione $\sum_{i=1}^{q} \alpha_i < 1$ assicura la stazionarietà del processo.

In forma compatta:

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^{q} \alpha_i \varepsilon_{t-i}^2, \tag{2.2}$$

che mostra come la volatilità condizionata sia una combinazione lineare degli shock passati. In questa struttura, la volatilità dipende esclusivamente dagli shock recenti, catturando il fatto che periodi di elevata variabilità tendono a essere seguiti da periodi anch'essi più volatili. Il parametro q controlla la memoria del processo: valori più elevati permettono di considerare un maggior numero di shock passati.

Dal punto di vista economico, il modello ARCH è in grado di riprodurre alcune delle stylized facts tipiche dei mercati finanziari, come la presenza di code pesanti nella distribuzione dei rendimenti e la persistenza della volatilità [6]. Tuttavia, all'aumentare dell'ordine q, il numero di parametri da stimare cresce rapidamente, rendendo il modello meno parsimonioso e più instabile. Inoltre, il modello reagisce solo agli shock recenti e non incorpora una vera componente di persistenza di lungo periodo nella volatilità (cfr. Sezione 1.4), spesso riscontrata nei dati finanziari [31].

Per ovviare a questi limiti, Bollerslev [7] propose una generalizzazione nota come *Generalized ARCH* (GARCH), in cui la varianza condizionata dipende non solo dai quadrati degli errori passati ma anche dai propri valori ritardati. Prima ancora, nel contesto operativo della gestione del rischio, modelli empirici più semplici come l'*Exponentially Weighted Moving Average* (EWMA) venivano utilizzati per aggiornare la volatilità in modo reattivo, sebbene privi di una base probabilistica rigorosa [22].

Il modello ARCH rappresenta dunque il punto di partenza teorico da cui derivano tutte le successive specificazioni della famiglia GARCH, che estendono e raffinano la modellazione della volatilità condizionata mantenendo la stessa intuizione di fondo: la varianza non è costante ma dipende dall'informazione passata disponibile sul mercato.

2.2 Il modello GARCH(p,q) di Bollerslev

Il modello *Generalized Autoregressive Conditional Heteroskedasticity* (GARCH), proposto da Bollerslev [7], estende la formulazione di Engle introducendo una componente autoregressiva anche nella varianza condizionata. L'obiettivo è ottenere una rappresentazione più parsimoniosa della volatilità, in grado di catturare la persistenza empiricamente osservata nei mercati finanziari [14].

Nel modello GARCH, la varianza condizionata al tempo t dipende non solo dai quadrati dei residui passati, ma anche dai propri valori ritardati, consentendo così una memoria più lunga e

un'evoluzione più graduale della volatilità. Formalmente:

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^{q} \alpha_i \varepsilon_{t-i}^2 + \sum_{j=1}^{p} \beta_j \sigma_{t-j}^2. \tag{2.3}$$

Affinché il processo GARCH sia stazionario in varianza, è necessario che $\sum_{i=1}^{q} \alpha_i + \sum_{j=1}^{p} \beta_j < 1$, condizione che garantisce la convergenza della varianza condizionata verso un valore finito nel lungo periodo.

La versione più utilizzata è il GARCH(1,1), che bilancia semplicità e capacità descrittiva, catturando gran parte della persistenza tipica della volatilità:

$$\sigma_t^2 = \omega + \alpha_1 \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2, \tag{2.4}$$

dove α_0, spesso indicato come ω nella letteratura applicativa e nei software econometrici, rappresenta il termine costante della varianza condizionata.

Rispetto all'ARCH, il GARCH rappresenta un'evoluzione più efficiente e parsimoniosa. Un modello ARCH di ordine elevato può essere approssimato, in termini di dinamica della varianza, da un GARCH(1,1), che utilizza solo tre parametri mantenendo una buona capacità descrittiva [31]. Un caso particolare del GARCH(1,1) è il modello *Exponentially Weighted Moving Average* (EWMA), ottenuto ponendo $\alpha_0 = 0$ e $\alpha_1 + \beta_1 = 1$, utilizzato nel framework *RiskMetrics* sviluppato da J.P. Morgan [22].

Dal punto di vista economico, il modello GARCH incorpora la proprietà di mean reversion della varianza: in assenza di nuovi shock, la volatilità tende progressivamente a convergere verso il proprio valore di equilibrio di lungo periodo:

$$V_L = \frac{\alpha_0}{1 - \alpha_1 - \beta_1}, \tag{2.5}$$

purché $\alpha_1 + \beta_1 < 1$, condizione di stazionarietà del processo.

2.3 Estensioni asimmetriche dei modelli GARCH

Sebbene il GARCH(1,1) descriva efficacemente la persistenza della volatilità, esso assume che gli shock positivi e negativi abbiano lo stesso impatto, risultando quindi simmetrico. Tuttavia, come discusso in precedenza (cfr. Sezione 1.4), nei mercati azionari si osserva una risposta asimmetrica agli shock: rendimenti negativi tendono ad amplificare maggiormente la volatilità rispetto a quelli positivi. Questo comportamento riflette la maggiore avversione al rischio degli investitori nei periodi di ribasso.

Per modellare tale dinamica sono state proposte diverse estensioni asimmetriche dei modelli GARCH, tra cui l'*Exponential GARCH* (EGARCH) di Nelson [33] e il *GJR-GARCH* di Glosten, Jagannathan e Runkle [15], quest'ultimo riconducibile al più generale *Threshold GARCH* (TGARCH) di Zakoian [41].

Il modello EGARCH(1,1) è formulato come:

$$\ln(\sigma_t^2) = \omega + \beta_1 \ln(\sigma_{t-1}^2) + \alpha_1 \left| \frac{\varepsilon_{t-1}}{\sigma_{t-1}} \right| + \gamma_1 \frac{\varepsilon_{t-1}}{\sigma_{t-1}}. \tag{2.6}$$

La modellazione del logaritmo della varianza garantisce $\sigma_t^2 > 0$ senza imporre vincoli parametrici e consente di catturare asimmetrie nella risposta agli shock: se $\gamma_1 < 0$, gli shock negativi producono un aumento della volatilità maggiore rispetto a quelli positivi di pari ampiezza.

Nel modello GJR-GARCH(1,1):

$$\sigma_t^2 = \omega + \alpha_1 \varepsilon_{t-1}^2 + \gamma_1 I_{t-1} \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2, \tag{2.7}$$

dove $I_{t-1} = 1$ se $\varepsilon_{t-1} < 0$ e 0 altrimenti. In questo caso, valori positivi di γ_1 indicano che gli shock negativi amplificano la varianza più di quelli positivi di pari ampiezza, in linea con l'evidenza empirica.

Le specificazioni EGARCH e GJR-GARCH consentono di modellare in modo più realistico i meccanismi di formazione della volatilità, risultando particolarmente efficaci nell'analisi di serie azionarie caratterizzate da asimmetrie e leverage effect [31].

Un'ulteriore estensione rilevante della famiglia GARCH è rappresentata dal modello *Asymmetric Power ARCH* (APARCH), proposto da Ding, Granger e Engle [11], che generalizza le specificazioni precedenti introducendo una trasformazione flessibile della volatilità. La caratteristica distintiva dell'APARCH consiste nel modellare una potenza arbitraria della deviazione standard condizionata, anziché direttamente la varianza, consentendo al dato di determinare la trasformazione più appropriata della volatilità. Nel caso di un modello APARCH(1,1), la dinamica è descritta da:

$$\sigma_t^\delta = \omega + \alpha_1 \left(|\varepsilon_{t-1}| - \gamma_1 \varepsilon_{t-1} \right)^\delta + \beta_1 \sigma_{t-1}^\delta, \tag{2.8}$$

dove $\delta > 0$ è il parametro di potenza e γ_1 cattura l'asimmetria nella risposta della volatilità agli shock. Valori positivi di γ_1 implicano che gli shock negativi producano un incremento più marcato della volatilità rispetto a quelli positivi, coerentemente con il leverage effect. Il modello APARCH include come casi particolari il GARCH standard ($\delta = 2$, $\gamma = 0$) e il GJR-GARCH ($\delta = 2$, $\gamma \neq 0$), rappresentando una delle formulazioni più generali nell'ambito dei modelli a varianza condizionata [11, 31].

Nel presente lavoro, tali specificazioni sono state stimate e confrontate empiricamente con il modello GARCH standard, al fine di valutarne la capacità predittiva rispetto alla volatilità realizzata dell'indice S&P 500 e di altre serie finanziarie.

2.4 Distribuzioni per le innovazioni nei modelli GARCH

Nella formulazione classica dei modelli ARCH e GARCH, le innovazioni z_t vengono generalmente assunte come indipendenti e identicamente distribuite secondo una distribuzione normale standard, $z_t \sim \mathcal{N}(0,1)$. Tuttavia, numerose evidenze empiriche mostrano che i rendimenti finanziari presentano distribuzioni con code pesanti (cfr. Sezione 1.4) e, spesso, una certa asimmetria, in contrasto con l'assunzione di normalità. Per tener conto di tali caratteristiche, sono state introdotte distribuzioni alternative più flessibili, capaci di descrivere meglio la forma empirica dei rendimenti. Le più utilizzate nella modellizzazione GARCH sono:

- la *Student-t* distribuzione, che consente di catturare la presenza di code più spesse rispetto alla normale, grazie al parametro dei gradi di libertà ν, che regola il peso delle code;
- la *Generalized Error Distribution* (GED), che introduce un parametro di forma κ in grado di adattare la curtosi della distribuzione, rendendola più o meno leptocurtica rispetto alla normale;
- le versioni asimmetriche di tali distribuzioni, come la *skewed-t* e la *skewed-GED*, che aggiungono un parametro di asimmetria e permettono di modellare la maggiore probabilità di variazioni negative (*left skewness*) tipica dei rendimenti azionari.

La scelta della distribuzione per z_t ha implicazioni dirette sulla stima dei parametri e sull'accuratezza delle previsioni di volatilità. Distribuzioni con code più pesanti o asimmetriche consentono di rappresentare in modo più realistico il comportamento estremo dei rendimenti e di ottenere modelli di volatilità più coerenti con l'evidenza empirica. Per questo motivo, nella letteratura applicata e nel presente lavoro, i modelli GARCH, EGARCH, GJR-GARCH e APARCH vengono stimati assumendo diverse distribuzioni per le innovazioni, al fine di valutare quale specificazione risulti più adatta a descrivere la dinamica osservata dei rendimenti.

La trattazione teorica dei modelli a varianza condizionata e delle diverse specificazioni distribuzionali costituisce la base per l'analisi empirica sviluppata nel Capitolo 5.

Capitolo 3

Fondamenti delle reti neurali artificiali

3.1 Struttura e funzionamento di un neurone artificiale

Per comprendere il funzionamento delle reti neurali, è utile partire da un esempio semplice. Si consideri l'obiettivo di stimare la volatilità futura $\hat{\sigma}_t$ di un titolo azionario a partire dalle informazioni passate, come la volatilità del giorno precedente σ_{t-1}. In altri termini, si vuole approssimare una funzione ignota $f(\cdot)$ tale che:

$$\hat{\sigma}_t = f(\sigma_{t-1}), \tag{3.1}$$

dove f descrive la relazione tra la volatilità futura e quella passata.

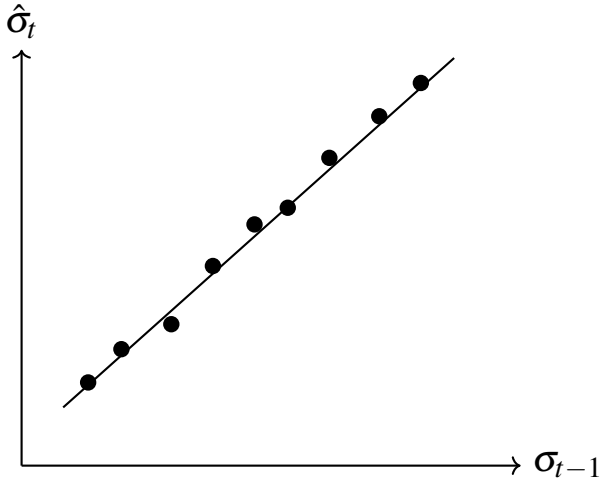


Figura 3.1: Relazione lineare tra volatilità futura stimata $\hat{\sigma}_t$ e volatilità passata σ_{t-1}.

Nel caso più semplice, si può rappresentare questa relazione mediante una regressione lineare:

$$\hat{\sigma}_t = w_0 + w_1 \sigma_{t-1} + \varepsilon_t, \tag{3.2}$$

dove w_0 è il termine di bias (intercetta), w_1 è il peso associato all'informazione passata e ε_t rappresenta il termine d'errore residuo. Una volta stimati questi coefficienti, il modello è in grado di fornire previsioni della volatilità futura sulla base dei valori passati. Questa relazione lineare può essere visualizzata come una rete neurale estremamente semplice, in cui le operazioni di regressione vengono interpretate come passaggi eseguiti da nodi interconnessi. In particolare:

- il primo input della rete è un valore costante $x_0 = 1$, associato al bias w_0;
- il secondo input è la variabile indipendente $x_1 = \sigma_{t-1}$, associata al peso w_1;
- i due contributi vengono combinati linearmente nel nodo centrale, producendo $w_0 + w_1\sigma_{t-1}$.

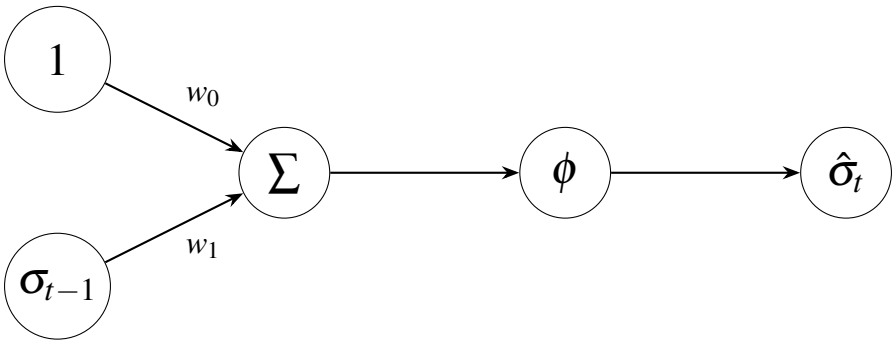


Figura 3.2: Schema semplificato di neurone artificiale.

L'elemento che distingue le reti neurali da una semplice regressione lineare è la funzione di attivazione, qui indicata con $\phi(\cdot)$. Essa viene applicata all'uscita della combinazione lineare per introdurre una componente non lineare nel modello. Ad esempio, se si desidera che il modello restituisca valori positivi, si può adottare come funzione di attivazione la *Rectified Linear Unit* (ReLU), che pone a zero gli input negativi. L'impiego delle funzioni di attivazione consente di catturare relazioni non lineari e pattern complessi nei dati, migliorando la capacità predittiva del modello [20]. In assenza di una funzione di attivazione, la rete si ridurrebbe a una pura combinazione lineare degli input, risultando equivalente a una regressione lineare.

Il componente fondamentale di una rete neurale è il neurone artificiale (o *perceptron*), ispirato al funzionamento dei neuroni biologici. Ogni neurone riceve in input una serie di valori $x_1, x_2, \ldots, x_n$, ai quali sono associati corrispondenti pesi $w_1, w_2, \ldots, w_n$, che ne determinano l'importanza relativa. Alla combinazione lineare degli input si aggiunge il termine di bias w_0.

Il risultato lineare del neurone, indicato con a (*activation input*), è dato da:

$$a = w_0 + \sum_{i=1}^{n} w_i x_i. \tag{3.3}$$

Successivamente, il valore a viene trasformato mediante una funzione di attivazione $\phi(\cdot)$, producendo l'output finale del neurone:

$$\hat{y} = \phi(a). \tag{3.4}$$

In sintesi, all'interno di un neurone vengono svolte due operazioni fondamentali: una trasformazione lineare (combinazione pesata degli input) seguita da una trasformazione non lineare (applicazione della funzione di attivazione).

La Figura 3.3 mostra come le volatilità passate $\sigma_{t-1}, \ldots, \sigma_{t-n}$ vengono combinate linearmente tramite i pesi w_i e trasformate dalla funzione di attivazione ϕ per generare la previsione della volatilità futura $\hat{\sigma}_t$. L'output $\hat{y}$ non rappresenta il risultato finale dell'intera rete, ma solo l'uscita di un singolo neurone. In una rete multilivello, tale uscita diventa l'input per altri neuroni negli strati successivi, consentendo di costruire modelli sempre più complessi e capaci di apprendere relazioni non lineari nei dati.

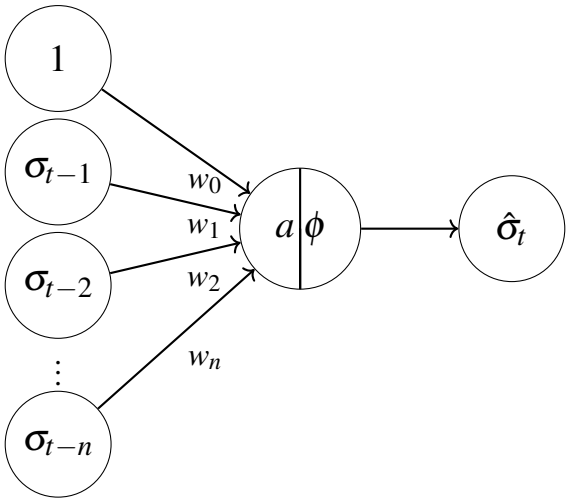


Figura 3.3: Struttura generale di un neurone artificiale con n input.

3.2 Funzioni di attivazione

L'applicazione di una funzione di attivazione è essenziale per introdurre non linearità nel modello e permettere alla rete neurale di apprendere relazioni complesse tra le variabili: il suo scopo è quindi quello di consentire alla rete di modellare pattern non lineari nei dati, migliorandone la capacità predittiva.

Esistono numerose tipologie di funzioni di attivazione, ma alcune si sono affermate come standard per le loro proprietà matematiche e computazionali:

- *Sigmoid*: funzione logistica che restituisce valori compresi tra 0 e 1, motivo per cui il suo output può essere interpretato come una probabilità.

$$\phi(a) = \frac{1}{1+e^{-a}} \tag{3.5}$$

  Tuttavia, la Sigmoid è soggetta al problema della saturazione: per input estremi ($a \to \pm\infty$) il gradiente tende a zero, rallentando l'apprendimento della rete.

$$\phi'(a) = \phi(a)\big(1-\phi(a)\big) \tag{3.6}$$

  In particolare, per $a \to -\infty$, $\phi(a) \to 0$ e quindi $\phi'(a) \to 0$; per $a \to +\infty$, $\phi(a) \to 1$ e ancora $\phi'(a) \to 0$.

- *Tanh*: funzione iperbolica che restituisce valori compresi tra -1 e 1. È simmetrica rispetto all'origine e spesso preferita alla Sigmoid negli strati intermedi della rete.

$$\phi(a) = \frac{e^{a}-e^{-a}}{e^{a}+e^{-a}} \tag{3.7}$$

- *Rectified Linear Unit*: è la funzione di attivazione più utilizzata. Essa restituisce 0 per input negativi e a per input positivi.

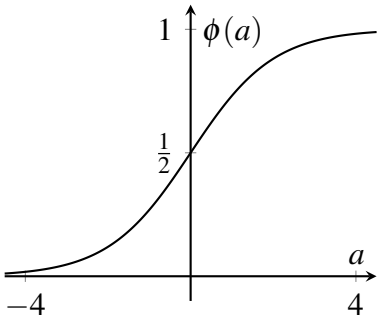
$$\phi(a) = \max(0,a) \tag{3.8}$$

  La ReLU è computazionalmente efficiente e accelera l'addestramento, ma può generare il problema del *dead neuron*: alcuni neuroni, ricevendo solo input negativi, restituiscono sempre zero, smettendo di aggiornare i propri pesi durante il training.

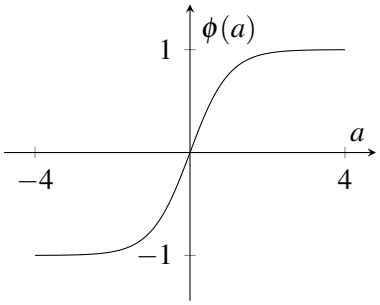
- *Leaky ReLU*: è una variante della ReLU che assegna un piccolo valore negativo agli input inferiori a zero, evitando che il gradiente si annulli completamente. In questo modo, anche i neuroni che ricevono input negativi continuano a contribuire all'apprendimento.

$$\phi(a) = \max(\alpha a, a) \quad \text{con } \alpha \in (0,1) \tag{3.9}$$

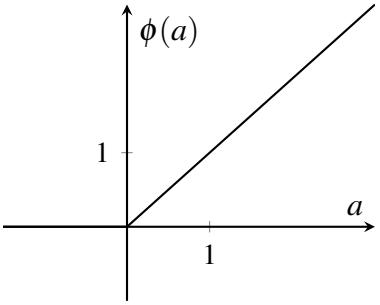
Le principali funzioni di attivazione descritte sono rappresentate graficamente in Figura 3.4.



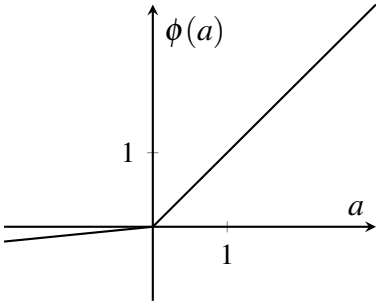
(a) Funzione di attivazione Sigmoid.



(b) Funzione di attivazione Tanh.



(c) Funzione di attivazione ReLU.



(d) Funzione di attivazione Leaky ReLU.

Figura 3.4: Confronto tra le principali funzioni di attivazione.

Non esiste una funzione di attivazione universalmente migliore: la scelta dipende dal contesto, dall'architettura della rete e dalle caratteristiche dei dati. In generale, è opportuno testare più funzioni e confrontarne le prestazioni.

La derivata della funzione rispetto all'input rappresenta il gradiente, un elemento cruciale per determinare come aggiornare i pesi durante l'addestramento, con l'obiettivo di ridurre l'errore complessivo del modello. La Figura 3.5 mostra le principali funzioni di attivazione insieme alle loro derivate. Per le funzioni Sigmoid e Tanh, il gradiente tende a zero per valori estremi di a, rendendo inefficace l'aggiornamento dei pesi: questo fenomeno è noto come *vanishing gradient problem* [3]. Tale criticità è analoga a quella dei dead neurons della ReLU.

Nel contesto della regressione su serie storiche, come la previsione della volatilità realizzata, la scelta della funzione di attivazione è particolarmente importante. Negli strati nascosti (o *hidden layers*) si utilizzano comunemente funzioni come la ReLU, la Tanh o la Leaky ReLU, che offrono un buon compromesso tra capacità di apprendimento e stabilità numerica. La Sigmoid, invece, è raramente impiegata negli strati nascosti poiché tende a saturarsi e rallenta l'addestramento.

Per lo strato di uscita (o *output layer*), trattandosi di un problema di regressione, la rete deve essere in grado di produrre valori reali non vincolati in un intervallo specifico; pertanto, si utilizza una funzione lineare $\phi(a) = a$. In questo modo, il valore di output non viene trasformato e può assumere qualsiasi valore reale.

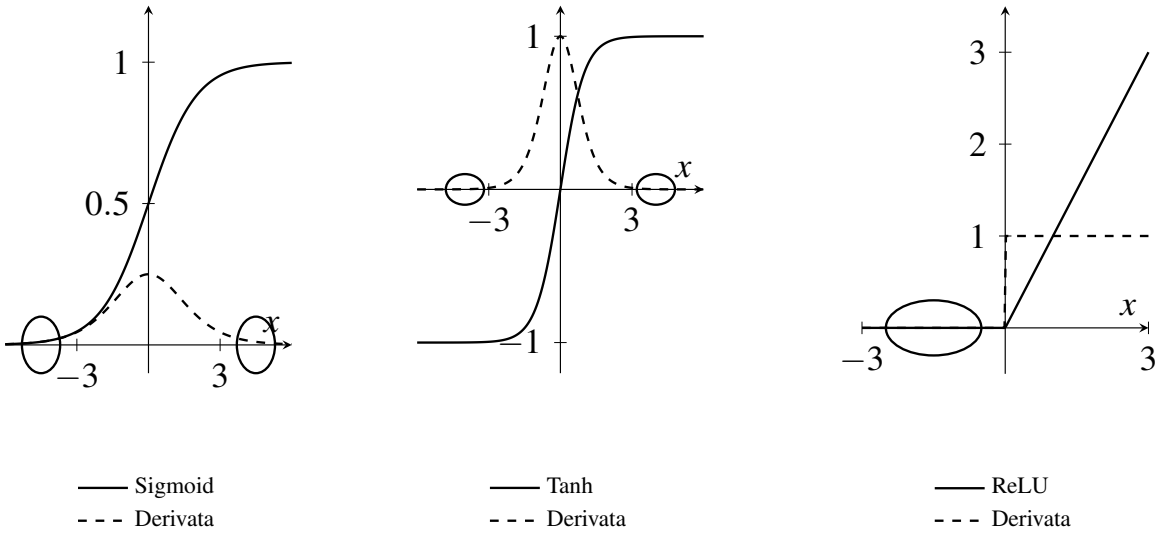


Figura 3.5: Funzioni di attivazione e relative derivate.

3.3 Costruzione di una rete neurale

Una rete neurale è un modello di machine learning composto da strati di neuroni interconnessi, ciascuno dei quali elabora e trasmette informazioni.

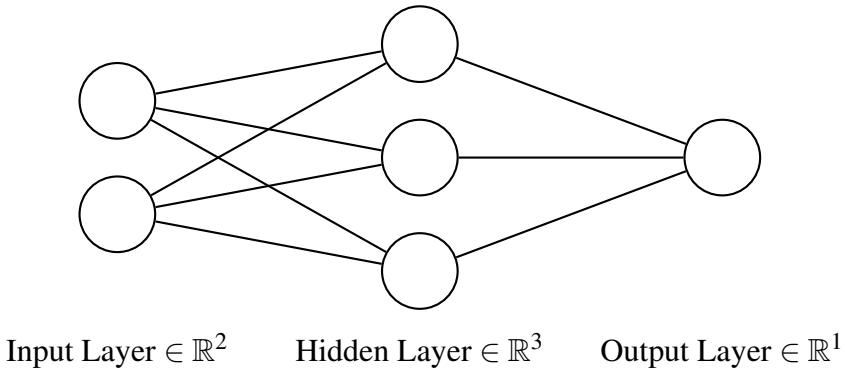


Figura 3.6: Rete neurale semplice con struttura $\mathbb{R}^2 \to \mathbb{R}^3 \to \mathbb{R}^1$.

Nell'esempio riportato in Figura 3.6 è illustrata una rete neurale composta da uno strato di input con due neuroni, uno strato nascosto con tre neuroni e uno strato di output con un singolo neurone. L'output finale della rete corrisponde al valore calcolato dal neurone nell'ultimo strato, ovvero la previsione del modello.

I neuroni si distinguono in tre categorie, in base allo strato in cui si trovano:

- neuroni di input: non eseguono alcun calcolo ma fungono da canali di ingresso del modello; ciascun neurone di input corrisponde a una variabile (*feature*) del dataset. Ad esempio, se l'input è un vettore composto da x_1, x_2, x_3, saranno presenti tre neuroni nello strato di input;

- neuroni nascosti: sono presenti negli hidden layers e costituiscono i veri neuroni computazionali della rete. Eseguono la somma pesata degli input ricevuti e applicano una funzione di attivazione. Sono responsabili del processo di apprendimento e rappresentano le trasformazioni interne dei dati lungo la rete.

- neuroni di output: svolgono le stesse operazioni dei neuroni nascosti (somma pesata e attivazione), ma la funzione $\phi(\cdot)$ utilizzata nello strato di uscita dipende dalla natura del problema. Nei problemi di regressione, come la previsione della volatilità, si utilizza tipicamente una funzione lineare.

Un concetto fondamentale è quello di *forward propagation*: si tratta del meccanismo attraverso il quale, dato un input, una rete neurale calcola il corrispondente output [16, 20]. Questo flusso di calcolo attraversa i vari strati della rete, dai neuroni di input a quelli di output, attraverso operazioni di somma pesata e funzioni di attivazione [16].

In generale, è utile distinguere tra due principali architetture di rete:

- reti neurali con flusso in avanti (o *Feedforward Neural Network* - FNN): costituiscono la struttura più semplice, in cui l'informazione fluisce in un'unica direzione, senza cicli, dagli input verso l'output. Ogni neurone riceve input dallo strato precedente e trasmette il risultato allo strato successivo, senza retroconnessioni;

- reti neurali ricorrenti (o *Recurrent Neural Network* - RNN): sono progettate per elaborare dati sequenziali o temporali. Al loro interno esistono connessioni che formano cicli, permettendo alla rete di mantenere memoria degli stati passati. Ciò le rende adatte a compiti come la modellazione di serie storiche, il riconoscimento vocale e l'elaborazione del linguaggio naturale.

La struttura dello strato di output può variare a seconda del tipo di previsione desiderata:

- *Single output*: lo strato di uscita contiene un solo neurone, utilizzato per:
  - *one-step forecast*, ossia la previsione del valore della serie a un singolo istante futuro;
  - *multi-step forecast*, in cui lo stesso neurone restituisce un vettore di valori futuri $(y_{t+1}, y_{t+2}, \ldots, y_{t+H})$. Anche se il numero di passi previsti è maggiore di uno, formalmente si tratta ancora di un singolo neurone con output multidimensionale.

- *Multiple output*: lo strato di uscita contiene più neuroni, ciascuno dei quali produce una previsione distinta. Questa configurazione è utile in contesti multivariati, oppure quando si desidera ottenere previsioni su orizzonti multipli o su variabili diverse.

Le applicazioni più avanzate del deep learning si basano su architetture più profonde, note come *Deep Neural Network* (DNN) (Figura 3.7). Rispetto alle MLP tradizionali, le DNN presentano numerosi strati nascosti, e questa maggiore profondità consente di approssimare funzioni altamente complesse [27]. In sintesi, una rete neurale può essere vista come una sequenza di trasformazioni successive tra input e output, dove ciascun livello apprende una rappresentazione più astratta dei dati. Nel paragrafo seguente verrà analizzato come tali reti apprendano i propri parametri attraverso il processo di *backpropagation*.

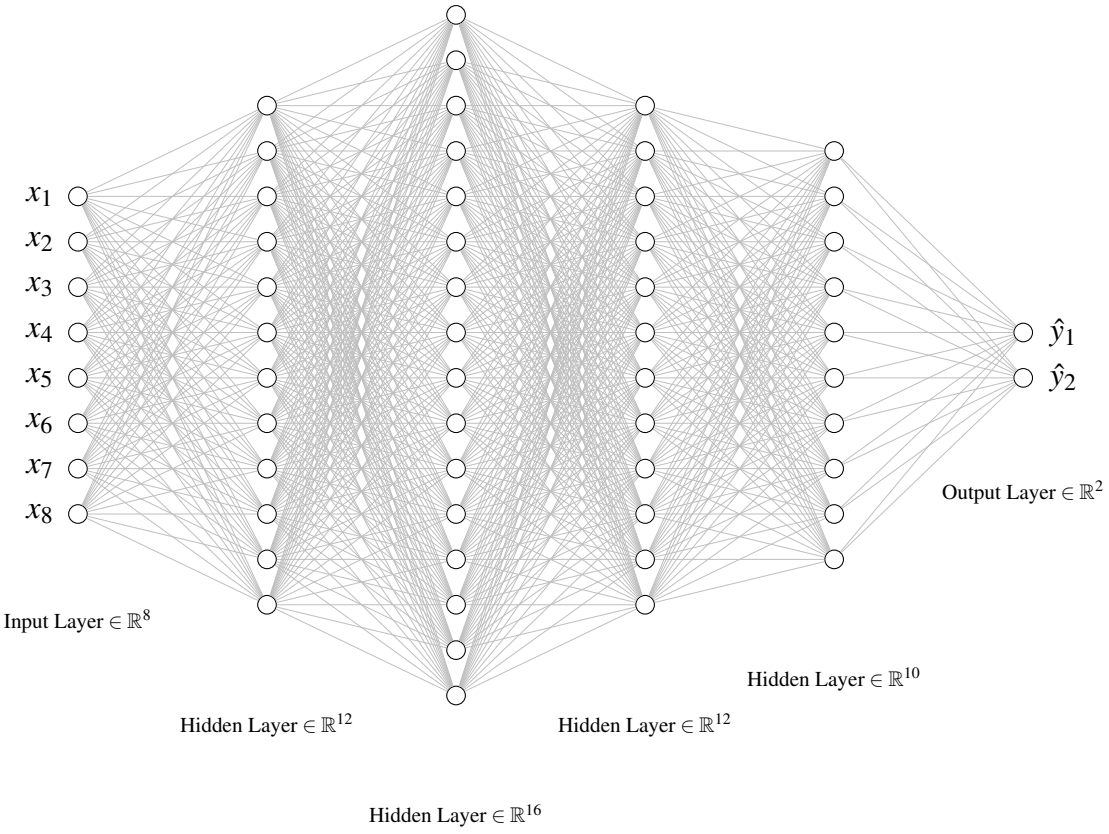


Figura 3.7: Esempio di DNN con 4 strati nascosti.

3.4 Apprendimento supervisionato dei parametri

Come visto in precedenza, una rete neurale, dato un input x, produce una previsione $\hat{y}$ attraverso il processo di forward propagation. Per addestrare la rete e migliorarne le prestazioni, è necessario misurare quanto la previsione $\hat{y}$ si discosti dal valore reale osservato y. A tale scopo si utilizza una funzione di costo (o *loss function*) $\mathscr{L}(\hat{y}, y)$ che quantifica l'errore commesso dal modello.

L'obiettivo dell'apprendimento supervisionato è stimare i pesi W e i bias della rete in modo da minimizzare la funzione di costo, ovvero ridurre l'errore medio sulle osservazioni. Per farlo, la rete impiega l'algoritmo di backpropagation, che calcola il gradiente della funzione di costo rispetto ai parametri del modello e li aggiorna nella direzione in cui l'errore decresce più rapidamente. Attraverso iterazioni successive, dette epoche (o *epochs*), la rete neurale apprende dai dati, migliorando progressivamente la propria capacità predittiva.

Per garantire un addestramento corretto e una valutazione imparziale, il dataset viene suddiviso in sottoinsiemi distinti:

- *Training set*: utilizzato per stimare i parametri del modello (pesi e bias) durante la fase di apprendimento;
- *Validation set*: impiegato per ottimizzare gli iperparametri e prevenire l'overfitting monitorando le prestazioni del modello su dati non visti durante il training;
- *Test set*: utilizzato per valutare la capacità di generalizzazione del modello su dati completamente nuovi.

Formalmente, l'apprendimento supervisionato si traduce in un problema di ottimizzazione:

$$W^* = \arg\min_W \frac{1}{n} \sum_{i=1}^{n} \mathscr{L}\big(f(x^{(i)}; W), y^{(i)}\big), \tag{3.10}$$

dove:

- n è il numero di osservazioni nel training set;
- $x^{(i)}$ è l'input i-esimo;
- $y^{(i)}$ è il valore osservato corrispondente;
- $f(x^{(i)}; W) = \hat{y}^{(i)}$ è la previsione della rete;
- $\mathscr{L}$ è la funzione di perdita, che misura la discrepanza tra $\hat{y}^{(i)}$ e $y^{(i)}$.

La funzione di costo complessiva può essere espressa come:

$$J(W) = \frac{1}{n} \sum_{i=1}^{n} \mathscr{L}\big(\hat{y}^{(i)}, y^{(i)}\big), \tag{3.11}$$

e il compito della rete è determinare i pesi ottimali W^* tali da minimizzare $J(W)$:

$$W^* = \arg\min_W J(W). \tag{3.12}$$

Il processo di apprendimento supervisionato permette di minimizzare l'errore medio tra le previsioni e i valori reali, ottimizzando i parametri del modello in modo iterativo fino alla convergenza.

3.4.1 Funzioni di perdita per la regressione

Nei problemi di regressione si impiegano tipicamente le seguenti funzioni di perdita:

- *Mean Squared Error* (MSE): misura la media dei quadrati degli scarti tra valori osservati e previsti,

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} \big(y^{(i)} - \hat{y}^{(i)}\big)^2. \tag{3.13}$$

- *Root Mean Squared Error* (RMSE): radice quadrata della MSE, utile per riportare l'errore nelle stesse unità della variabile dipendente,

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^{n} \big(y^{(i)} - \hat{y}^{(i)}\big)^2}. \tag{3.14}$$

- *Mean Absolute Error* (MAE): media degli scarti assoluti, più robusta agli outlier rispetto alla MSE,

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} \big|y^{(i)} - \hat{y}^{(i)}\big|. \tag{3.15}$$

La scelta dipende dal contesto e dai dati: la MSE è comoda analiticamente e penalizza maggiormente gli errori grandi; la MAE è preferibile in presenza di outlier rilevanti. Nel contesto della previsione di volatilità, MSE e MAE sono entrambe diffuse e complementari.

3.4.2 Backpropagation e calcolo del gradiente

Il processo di apprendimento avviene attraverso le seguenti fasi iterative:

- si forniscono i dati in input alla rete neurale per calcolare l'output (forward propagation);
- si valuta la funzione di costo $\mathscr{L}(\hat{y}, y)$, che misura la differenza tra il valore predetto e quello reale osservato;
- si propaga l'errore all'indietro nella rete (backpropagation), utilizzando la regola della catena (*chain rule*) per calcolare il gradiente della funzione di perdita rispetto ai parametri;
- si aggiornano i pesi e i bias utilizzando l'algoritmo della discesa del gradiente;
- si ripete il processo fino a quando la funzione di perdita non raggiunge un livello soddisfacente oppure si raggiunge un numero predefinito di iterazioni (epoche).

Analizziamo ora nel dettaglio la terza fase. Durante la backpropagation, l'errore calcolato in uscita viene propagato a ritroso attraverso i vari strati della rete neurale. L'obiettivo è determinare in che misura ciascun peso abbia contribuito all'errore complessivo. Questo processo si basa sul calcolo dei gradienti, ovvero delle derivate parziali della funzione di perdita rispetto ai singoli parametri della rete. Per i pesi appartenenti agli strati interni, tali derivate vengono ottenute applicando la regola della catena o chain rule.

Si consideri, a titolo esemplificativo, una rete neurale estremamente semplice, costituita da uno strato nascosto con un solo neurone e da uno strato di output, illustrata in Figura 3.8.

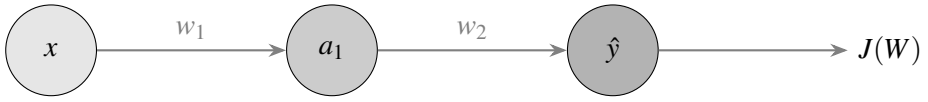


Figura 3.8: Flusso di derivazione nella backpropagation.

Nel diagramma sono indicati i seguenti elementi:

- x: input scalare della rete;
- w_1: peso che collega l'input x al neurone dello strato nascosto;
- a_1: combinazione lineare calcolata nel neurone nascosto data da $a_1 = b_1 + w_1 x$, dove b_1 è il termine di bias;
- w_2: peso che collega il neurone nascosto al neurone di output;
- $\hat{y}$: output predetto dalla rete;
- $J(W)$: funzione di perdita complessiva da minimizzare durante l'addestramento.

Il gradiente rispetto a w_2 è calcolato direttamente, poiché questo peso influenza l'output in modo immediato:

$$\frac{\partial J(W)}{\partial w_2} = \frac{\partial J(W)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2}. \tag{3.16}$$

Il calcolo del gradiente rispetto a w_1 è meno immediato, in quanto tale peso appartiene a uno strato precedente. In questo caso, si ricorre alla regola della catena per propagare il gradiente all'indietro. Ad esempio, considerando due funzioni composte $z = f(y)$ e $y = g(x)$, la loro derivata si calcola come:

$$\frac{dz}{dx} = \frac{df}{dy} \cdot \frac{dg}{dx}. \tag{3.17}$$

Allo stesso modo, nelle reti neurali, che sono funzioni composte da più strati, il gradiente della loss function rispetto a un peso in uno strato iniziale (come w_1) si ottiene seguendo la catena all'indietro:

$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a_1} \cdot \frac{\partial a_1}{\partial w_1}. \tag{3.18}$$

Il risultato di questa fase è quindi un insieme di gradienti, che vengono poi utilizzati dall'algoritmo di ottimizzazione per aggiornare i parametri della rete nella direzione di massima discesa della funzione di perdita.

3.4.3 Algoritmi di ottimizzazione

Una volta calcolati i gradienti della funzione di perdita rispetto ai parametri del modello, è possibile aggiornare i pesi della rete neurale. L'obiettivo è modificare ciascun peso in modo da ridurre progressivamente l'errore complessivo del modello. A tal fine, il gradiente fornisce un'informazione cruciale: la direzione in cui ogni peso deve essere modificato per diminuire la funzione di perdita. In particolare:

- se $\frac{\partial J(W)}{\partial w_i} > 0$, un aumento del peso w_i comporta un incremento della funzione di perdita, quindi è opportuno ridurre w_i;
- se $\frac{\partial J(W)}{\partial w_i} < 0$, un aumento di w_i riduce la perdita, pertanto conviene aumentare il valore di w_i;
- se il gradiente è prossimo a zero, il peso ha un'influenza trascurabile sull'errore, almeno in quella fase dell'addestramento.

L'algoritmo che utilizza queste informazioni per aggiornare i parametri della rete è noto come *Gradient Descent* (GD). Esso segue una procedura iterativa che può essere schematizzata come segue:

- inizializzazione casuale dei pesi, tipicamente da una distribuzione normale: $w \sim \mathcal{N}(0, \sigma^2)$;
- iterazione fino alla convergenza, ovvero finché la funzione di perdita non si stabilizza o si raggiunge un numero massimo di epoche;
- calcolo del gradiente della funzione di perdita rispetto ai pesi: $\frac{\partial J(W)}{\partial w}$;
- aggiornamento dei pesi nella direzione opposta al gradiente;
- restituzione dei pesi ottimizzati al termine dell'addestramento.

L'aggiornamento dei pesi avviene secondo la regola:

$$w \leftarrow w - \eta \cdot \frac{\partial J(W)}{\partial w}, \tag{3.19}$$

dove η è il *learning rate*, ossia il tasso di apprendimento che controlla l'ampiezza di ciascun aggiornamento. Una scelta appropriata di η è fondamentale per il buon funzionamento dell'algoritmo: un valore troppo piccolo comporta un apprendimento lento, mentre un valore eccessivo può causare instabilità e impedire la convergenza. Una strategia comune consiste nel partire con un η relativamente alto, riducendolo gradualmente man mano che l'algoritmo si avvicina al minimo della funzione di perdita.

Una metafora intuitiva per comprendere la discesa del gradiente è quella della "collina nella nebbia": immaginiamo di trovarci in cima a una collina avvolta dalla nebbia, senza visibilità. L'obiettivo è raggiungere il punto più basso possibile (il minimo della funzione di perdita), potendo percepire solo la pendenza locale. Ad ogni passo, ci muoviamo nella direzione in cui il terreno scende più rapidamente, proprio come fa l'algoritmo di GD, che aggiorna i pesi nella direzione opposta al gradiente. Un passo troppo lungo può far oltrepassare la valle (minimo), mentre un passo troppo corto rallenta il processo di ottimizzazione.

Durante l'addestramento, si effettuano più passaggi completi sull'intero training set: ogni epoca corrisponde a un ciclo di apprendimento in cui tutte le osservazioni del training set vengono utilizzate una volta per aggiornare i pesi del modello. Alla fine di ciascuna epoca si può calcolare la funzione di perdita complessiva $J(W)$, ossia l'errore medio tra i valori previsti e quelli reali, per valutare i progressi dell'apprendimento.

La Figura 3.9 fornisce una rappresentazione grafica delle strategie di discesa del gradiente.

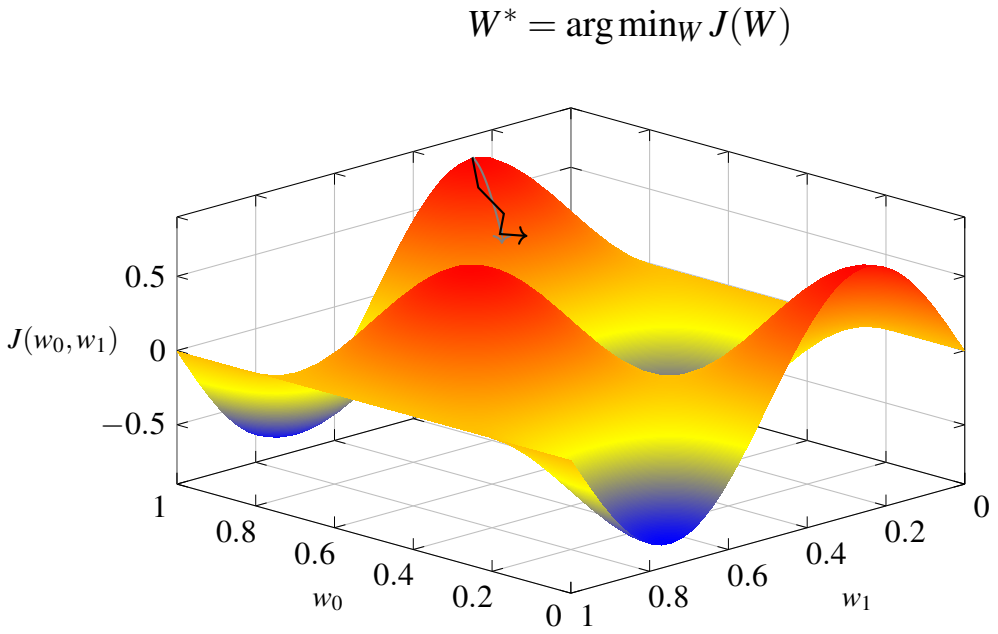


Figura 3.9: Confronto tra SGD e GD.

Il calcolo dei gradienti e l'aggiornamento dei pesi possono seguire diverse strategie [37]:

- *Batch Gradient Descent* (BGD): calcola il gradiente utilizzando l'intero training set a ogni iterazione. È accurato e stabile, ma risulta computazionalmente oneroso per dataset di grandi dimensioni.

- *Stochastic Gradient Descent* (SGD): aggiorna i pesi a ogni singola osservazione, rendendo l'apprendimento molto rapido ma più rumoroso, poiché il gradiente calcolato su un singolo dato può variare notevolmente. In Figura 3.9 si nota come l'SGD (nero) segua una traiettoria più rumorosa rispetto al GD (grigio).

- *Mini-batch Gradient Descent*: rappresenta un compromesso tra i due metodi precedenti. Il dataset è suddiviso in piccoli gruppi di dimensione B; per ciascuna osservazione k appartenente al batch si calcola il gradiente $\frac{\partial J_k(W)}{\partial w}$. Il gradiente utilizzato per l'aggiornamento dei pesi è quindi ottenuto come media dei gradienti calcolati sulle osservazioni del batch:

$$\frac{\partial J(W)}{\partial w} = \frac{1}{B} \sum_{k=1}^{B} \frac{\partial J_k(W)}{\partial w}. \tag{3.20}$$

Anche con i mini-batch, la funzione di perdita delle reti profonde può presentare numerosi minimi locali, punti sella o aree piatte. In questi casi, i metodi descritti possono rallentare, oscillare o bloccarsi in soluzioni subottimali [28]. Per superare tali problemi sono stati sviluppati algoritmi più sofisticati, che introducono due concetti chiave: l'adattività del learning rate e il *momentum*. Quest'ultimo consiste nell'aggiungere una frazione del gradiente precedente al gradiente corrente, in modo che i pesi si muovano con maggiore coerenza nella direzione della discesa, superando oscillazioni locali e minimi poco profondi. Su questo principio si basano diversi ottimizzatori avanzati largamente utilizzati nelle reti neurali profonde, tra cui:

- *Momentum*: accelera la discesa del gradiente tenendo conto della direzione degli aggiornamenti precedenti;
- *Nesterov Accelerated Gradient* (NAG): versione migliorata del momentum, che anticipa la direzione di aggiornamento;
- *Adagrad* e *Adadelta*: adattano automaticamente il learning rate durante l'addestramento;
- *RMSProp*: utilizza la media quadratica dei gradienti per ridurre l'ampiezza dei passi in direzioni instabili;
- *Adaptive Moment Estimation* (*Adam*): combina momentum e adattamento del learning rate per ogni parametro.

La scelta dell'ottimizzatore più adatto dipende spesso dalla specifica architettura, dal tipo di dati e dagli obiettivi di previsione.

La Figura 3.10 illustra un confronto qualitativo tra diversi algoritmi di ottimizzazione. Le visualizzazioni sono tratte da materiale divulgativo di Alec Radford e rielaborate a partire da Ruder [37], cui si rimanda per una trattazione più approfondita.

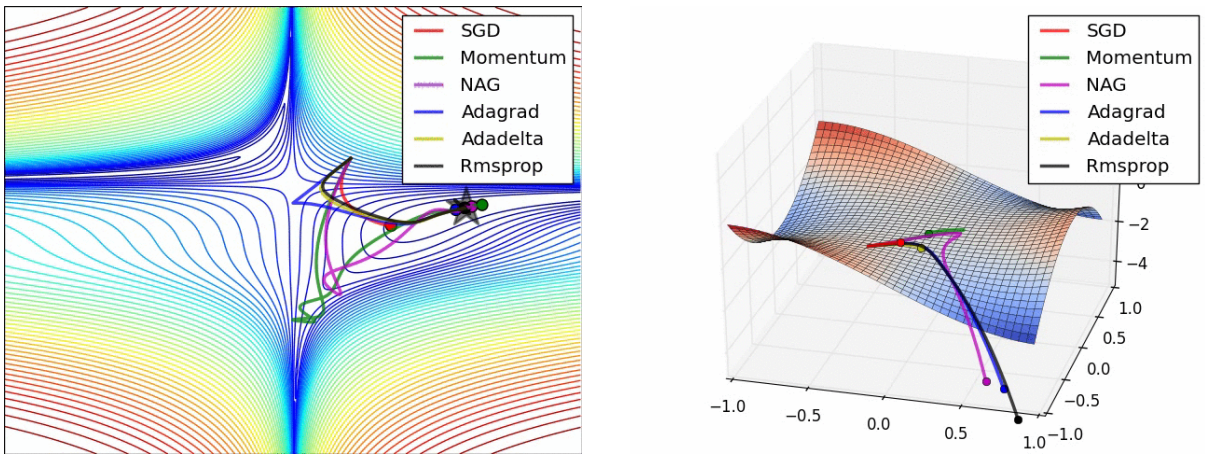


Figura 3.10: Confronto tra algoritmi di ottimizzazione.

3.5 Overfitting e tecniche di regolarizzazione

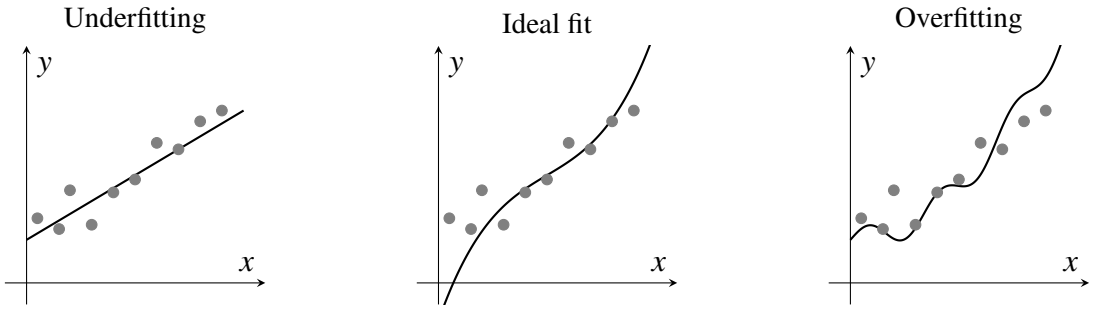


Figura 3.11: Underfitting, ideal fit e overfitting su un dataset fisso.

Uno degli aspetti più critici nell'addestramento delle reti neurali è rappresentato dal fenomeno dell'*overfitting*, che si verifica quando il modello si adatta in modo eccessivo ai dati di addestramento, includendo anche il rumore e le fluttuazioni casuali presenti nel training set. Questo comportamento compromette la capacità di generalizzazione, ossia l'abilità del modello di fornire previsioni affidabili su dati nuovi e mai osservati durante la fase di training [4, 16]. Come rappresentato in Figura 3.11, un modello può assumere tre diversi comportamenti:

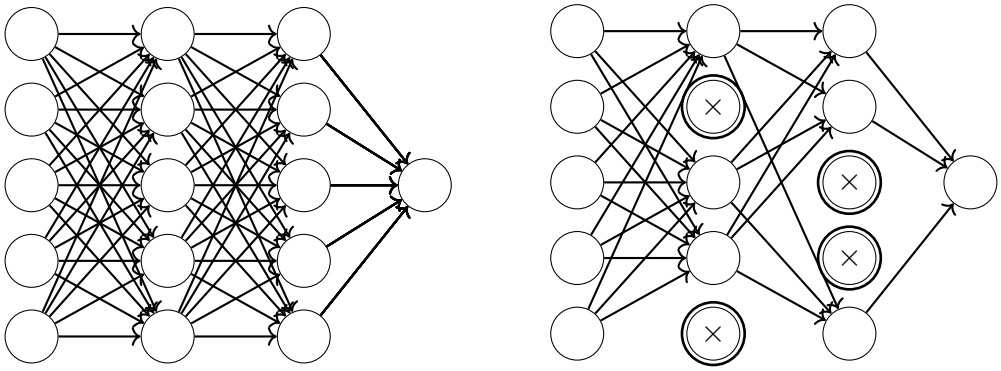
- *Underfitting*: il modello risulta troppo semplice per rappresentare adeguatamente la struttura dei dati. Ad esempio, una regressione lineare può non riuscire a catturare la non linearità presente.
- *Overfitting*: il modello è eccessivamente complesso e finisce per "memorizzare" i dati di training, adattandosi anche al rumore. Sebbene riesca ad apprendere i dati di addestramento, la sua capacità predittiva su dati nuovi risulta compromessa.
- *Ideal fit*: il modello riesce a catturare efficacemente i pattern sottostanti ai dati senza sovra-adattarsi a componenti casuali, ottenendo così una buona generalizzazione.

Per prevenire l'overfitting, si ricorre a tecniche di regolarizzazione, che hanno l'obiettivo di limitare la complessità del modello e favorire una migliore generalizzazione. Le principali strategie includono:

- *L1 regularization*: introduce un termine di penalizzazione nella funzione di perdita, proporzionale alla somma dei valori assoluti dei pesi del modello. La funzione di costo regolarizzata diventa:
$$\mathscr{L}_{L1} = \mathscr{L} + \lambda \sum_j |w_j| \tag{3.21}$$
dove λ è un iperparametro che controlla l'intensità della penalizzazione. Questa tecnica tende a indurre sparsità nei pesi del modello, spingendo molti di essi ad assumere valore esattamente zero. In pratica, il modello impara a concentrarsi sulle variabili più informative, escludendo quelle irrilevanti. È particolarmente utile in contesti ad alta dimensionalità o quando si desidera un modello più interpretabile.
- *L2 regularization*: aggiunge un termine di penalizzazione proporzionale alla somma dei quadrati dei pesi:
$$\mathscr{L}_{L2} = \mathscr{L} + \lambda \sum_j w_j^2 \tag{3.22}$$

dove λ è il coefficiente di regolarizzazione. A differenza della L1, questa tecnica non porta i pesi a zero, ma li riduce in modo omogeneo, limitando la complessità del modello e migliorandone la capacità di generalizzazione.

- *Dropout*: è una delle tecniche di regolarizzazione più diffuse. Durante la fase di addestramento, una frazione casuale di neuroni viene disattivata. In ciascuna iterazione, i neuroni esclusi non contribuiscono né al calcolo dell'output né alla propagazione del gradiente. Questo meccanismo impedisce alla rete di dipendere eccessivamente da specifici neuroni, favorendo un apprendimento più robusto e generalizzabile. L'effetto complessivo è simile a quello di addestrare più reti differenti e combinarne i risultati, riducendo il rischio di overfitting e migliorando le prestazioni su dati non visti [38].



Rete senza dropout

Rete con dropout (50%)

Figura 3.12: Dropout.

- *Early Stopping*: consiste nell'interrompere l'addestramento del modello non appena le prestazioni sul validation set iniziano a peggiorare, segnalando che il modello sta iniziando a sovra-adattarsi ai dati di training. Nelle fasi iniziali, sia la *training loss* sia la *validation loss* diminuiscono; tuttavia, dopo un certo punto, la validation loss comincia ad aumentare mentre la training loss continua a decrescere. Questo divario crescente indica l'insorgere dell'overfitting. L'early stopping interviene in questo momento, arrestando l'apprendimento prima che il modello perda capacità di generalizzazione. Tale tecnica è spesso utilizzata in combinazione con il dropout.

  La Figura 3.13 mostra l'andamento della perdita e dell'accuratezza (o *accuracy*) su training e validation set durante le epoche di addestramento. L'accuracy è una metrica comunemente impiegata nei problemi di classificazione e misura la proporzione di osservazioni correttamente classificate rispetto al totale. Nel grafico a sinistra, la perdita sul training set diminuisce costantemente, mentre quella sul validation set raggiunge un minimo intorno all'epoca 5 per poi aumentare, segnalando l'inizio dell'overfitting. Nel grafico a destra, l'accuratezza mostra un comportamento speculare: migliora fino all'epoca 5, dopodiché peggiora sul validation set. L'applicazione dell'*early stopping* consente di interrompere l'addestramento nel punto ottimale, evitando il degrado delle prestazioni su dati non visti (nel caso di regressione si monitora tipicamente la validation loss).

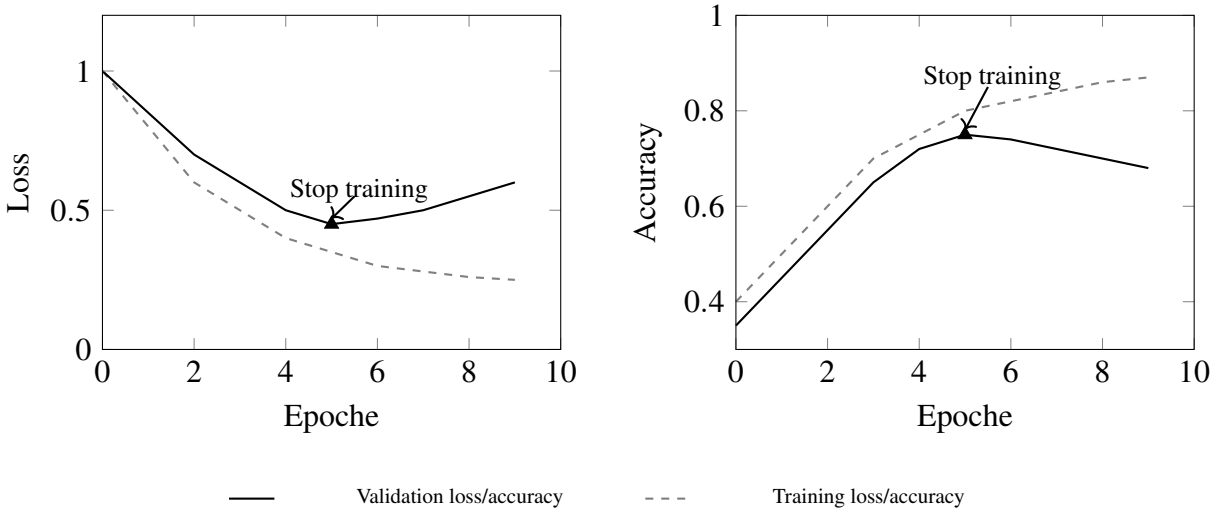


Figura 3.13: Early stopping.

3.6 I tensori e la rappresentazione dei dati

L'addestramento di una rete neurale richiede una rappresentazione strutturata e coerente dei dati in input. In questo contesto, il concetto di tensore riveste un ruolo fondamentale, poiché costituisce il formato standard attraverso cui i dati vengono organizzati e processati nei modelli di deep learning.

Un tensore è un array multidimensionale che memorizza i dati numerici utilizzati per addestrare e testare i modelli. Ogni tensore è caratterizzato da due proprietà chiave:

- *Rank*: indica il numero di dimensioni (o assi) del tensore;
- *Shape*: rappresenta la struttura lungo ciascun asse.

Ad esempio, un tensore di shape $(120, 12)$ è un tensore di rank pari a 2 (bidimensionale), composto da 120 righe e 12 colonne. Nel caso delle serie storiche finanziarie, i dati devono essere organizzati in modo compatibile con la struttura di input prevista dal modello neurale. A seconda dell'architettura utilizzata, la rappresentazione dei dati varia: nelle reti feedforward si utilizzano tensori di rank-2, con struttura $shape = (samples, features)$, dove ogni riga rappresenta un'osservazione e ogni colonna una variabile esplicativa (ad esempio, un dataset con 120 osservazioni e 12 variabili sarà rappresentato da un tensore di shape $(120, 12)$); nei modelli sequenziali (RNN, LSTM, CNN-1D), i dati vengono organizzati in tensori di rank-3, con struttura $shape = (samples, timesteps, features)$ in cui:

- *Samples*: rappresenta il numero di sequenze totali nel dataset;
- *Timesteps*: indica la lunghezza temporale di ciascuna sequenza (ad esempio, il numero di giorni o di osservazioni storiche considerate);
- *Features*: corrisponde al numero di variabili osservate in ciascun punto temporale (ad esempio, prezzo, volume o volatilità).

Un dataset composto da 250 giorni di contrattazioni, ciascuno suddiviso in 8 osservazioni orarie e con 2 variabili per osservazione (come prezzo e volume), sarà rappresentato da un tensore con shape $(250, 8, 2)$. Questa rappresentazione è essenziale per l'addestramento di modelli

in grado di catturare sia le dipendenze temporali all'interno delle sequenze (come le LSTM e le RNN), sia le relazioni locali tra le feature adiacenti nel tempo (come le CNN-1D). Anche architetture più avanzate, come N-BEATS, si basano su una rappresentazione tensoriale strutturata, che consente di modellare la dinamica temporale dei dati e di estrarre pattern complessi per la previsione di serie storiche finanziarie.

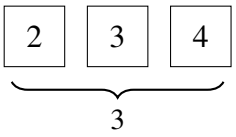
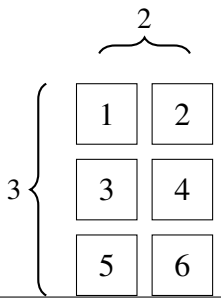
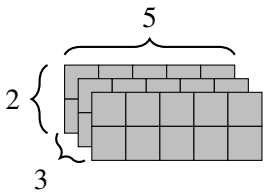
Tensore	*Shape*	*Esempio*
Scalare (Rank-0)	`()`	
Vettore (Rank-1)	`(3,)`	
Matrice (Rank-2)	`(3, 2)`	
Tensore (Rank-3)	`(3, 2, 5)`	

Tabella 3.1: Tensori con relative dimensioni.

In conclusione, la rappresentazione tensoriale fornisce la struttura comune necessaria per addestrare modelli neurali su dati finanziari. Il capitolo successivo approfondirà come tale rappresentazione venga utilizzata nelle architetture progettate per catturare le dipendenze temporali, come LSTM, GRU, CNN-1D e N-BEATS illustrando in che modo ciascun modello affronti i limiti delle reti feedforward e migliori la capacità di previsione della volatilità.

Capitolo 4

Architetture neurali per la previsione della volatilità

In questo capitolo vengono introdotte architetture neurali avanzate progettate per modellare esplicitamente la dimensione temporale dei dati, superando i limiti delle reti feedforward tradizionali. Le MLP rappresentano il punto di partenza naturale, poiché forniscono una solida *baseline* per catturare relazioni non lineari tra variabili, ma trattano ogni osservazione in modo indipendente e non possiedono alcuna forma di memoria. Questa limitazione le rende poco adatte a modellare le dipendenze temporali che caratterizzano la volatilità, fenomeno noto per la sua persistenza e per la presenza di dinamiche multi-orizzonte e pattern ricorrenti.

Per superare tali limiti, vengono considerate anche architetture di rete neurale che adottano meccanismi differenti per accumulare, aggiornare e trasmettere l'informazione nel tempo. Si procede analizzando in dettaglio le seguenti tipologie di modelli:

- *MLP*: rappresentano la struttura di base delle reti neurali e costituiscono un benchmark naturale per valutare il contributo introdotto dalla modellazione esplicita delle dipendenze temporali;
- *LSTM* e *GRU*: introducono uno stato interno di memoria e meccanismi di controllo che regolano il flusso informativo, consentendo di catturare dipendenze sia di breve sia di lungo periodo nelle serie temporali;
- *CNN-1D*: utilizzano filtri convoluzionali unidimensionali per individuare regolarità locali lungo l'asse temporale, combinando buona efficienza computazionale e capacità di estrazione di pattern dinamici;
- *N-BEATS*: adottano una struttura a blocchi residui dedicati alla ricostruzione e alla previsione della serie, offrendo un'architettura puramente feedforward ma particolarmente efficace e interpretabile nel contesto delle serie temporali univariate, come la volatilità realizzata.

L'obiettivo di questo capitolo è mostrare come ciascuna di queste architetture affronti i limiti delle reti feedforward tradizionali, introducendo differenti strategie di modellazione del tempo e come tali approcci possano essere impiegati per la previsione della volatilità realizzata.

4.1 Limiti delle reti feedforward e modellazione di dati sequenziali

Nell'analisi dei dati sequenziali, o *sequence modeling*, l'ordine temporale degli elementi è fondamentale per la comprensione del fenomeno osservato. In questo tipo di dati, le osservazioni non sono indipendenti e identicamente distribuite (i.i.d.), ma presentano forti dipendenze nel tempo [29]: l'informazione contenuta in un istante successivo dipende spesso da quella degli istanti precedenti, caratteristica rilevante nel contesto della previsione della volatilità, che presenta fenomeni di persistenza e autocorrelazione. Le architetture feedforward risultano pertanto inadatte a trattare in modo efficace dati di natura sequenziale, in quanto non sono progettate per modellare la struttura dinamica intrinseca nel tempo [3].

Per addestrare una rete neurale su una serie storica univariata, è necessario convertire i dati in una struttura supervisionata. Questo si ottiene tramite una finestra mobile (*rolling window*) [25] di lunghezza fissa T_x, che scorre lungo la serie e crea, per ciascun passo, una sequenza di input contenente i T_x valori passati e un valore target y_t da prevedere.

Ad esempio, fissando $T_x = 12$ e $T_y = 1$, possiamo costruire le seguenti matrici:

$$X = \begin{bmatrix} y_1 & y_2 & \cdots & y_{12} \\ y_2 & y_3 & \cdots & y_{13} \\ \vdots & \vdots & \ddots & \vdots \\ y_{132} & y_{133} & \cdots & y_{144} \end{bmatrix} \quad Y = \begin{bmatrix} y_{13} \\ y_{14} \\ \vdots \\ y_{145} \end{bmatrix}$$

Osserv.	y_{t-12}	y_{t-11}	y_{t-10}	y_{t-9}	y_{t-8}	y_{t-7}	y_{t-6}	y_{t-5}	y_{t-4}	y_{t-3}	y_{t-2}	y_{t-1}	y_t
1	1	2	3	4	5	6	7	8	9	10	11	12	13
2	2	3	4	5	6	7	8	9	10	11	12	13	14
3	3	4	5	6	7	8	9	10	11	12	13	14	15
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
130	130	131	132	133	134	135	136	137	138	139	140	141	142
131	131	132	133	134	135	136	137	138	139	140	141	142	143
132	132	133	134	135	136	137	138	139	140	141	142	143	144

Tabella 4.1: Esempio di costruzione del dataset supervisionato.

Si ottiene quindi un tensore X rank-2 di shape $(132, 12)$ e un vettore Y di shape $(132, 1)$. Successivamente, i dati vengono suddivisi in un training set e in un test set, mantenendo l'ordine temporale:

- $X_{\text{train}} \in \mathbb{R}^{120\times 12}$, $Y_{\text{train}} \in \mathbb{R}^{120\times 1}$;
- $X_{\text{test}} \in \mathbb{R}^{12\times 12}$, $Y_{\text{test}} \in \mathbb{R}^{12\times 1}$.

Il punto critico di questo approccio è che i valori forniti in input vengono trattati come variabili indipendenti. Sebbene siano ordinati nel vettore in modo da riflettere la sequenza temporale, la rete non possiede alcuna struttura interna che riconosca o modelli tale ordine. In altre parole, la rete è priva di memoria e non dispone di un meccanismo per rappresentare l'evoluzione temporale [19]. Di conseguenza, se l'ordine degli input viene modificato (ad esempio tramite uno *shuffle*), l'informazione temporale risulta completamente perduta. La rete

non è quindi in grado di cogliere pattern dinamici, quali persistenze, ricorrenze o dipendenze di lungo periodo. Ogni previsione è inoltre basata esclusivamente sull'input corrente, in assenza di qualsiasi meccanismo di accumulo o aggiornamento dello stato, poiché il modello non ha accesso alle osservazioni passate. L'ordine sequenziale dei *lags* è preservato solo per convenzione nella costruzione del dataset, non perché la rete ne comprenda la dimensione temporale. Pertanto, anche mantenendo la distinzione tra set di training e test, una feedforward non è in grado di sfruttare la continuità informativa tra osservazioni successive: ogni vettore di input è trattato come un'entità isolata e priva di relazione con il contesto temporale.

Questa limitazione motiva la necessità di architetture neurali specializzate nella modellazione sequenziale, come le RNN, che verranno trattate approfonditamente nella sezione successiva.

Si è visto nell'esempio che, nel caso della DNN, l'input ha dimensione $(n, 12)$, dove $n = 120$ rappresenta il numero di osservazioni nel training set. Ogni riga della matrice X_{train} corrisponde a una sequenza di 12 osservazioni consecutive, utilizzate come input per prevedere il valore futuro corrispondente in Y_{train}. Tuttavia, ciascun vettore di input è trattato dalla rete come una semplice riga indipendente, priva di una struttura temporale interna.

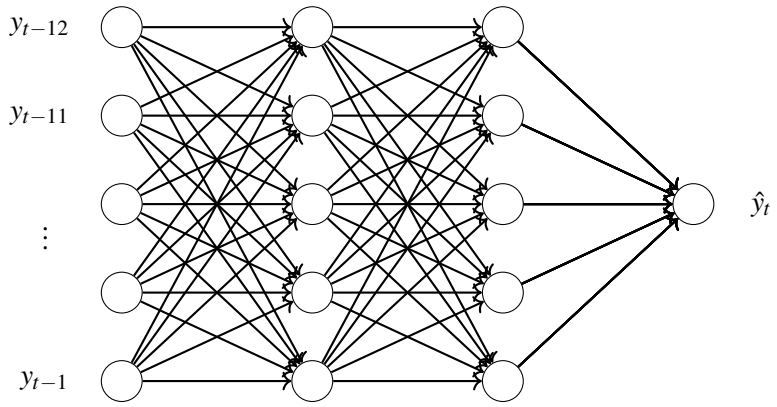


Figura 4.1: Architettura DNN: previsione $\hat{y}_t$ a partire dai valori passati.

Il modo in cui i dati vengono processati in una RNN è completamente diverso: la sequenza viene elaborata passo dopo passo, un valore alla volta. L'input ha dimensione $(n, 12, 1)$: ogni osservazione è una sequenza di 12 lag, ciascuno con una sola variabile (il valore y_t della serie al tempo t). In questo modo, la rete sfrutta la propria architettura ricorrente per aggiornare uno stato interno a ogni passo, trasportando informazione dal passato al futuro.

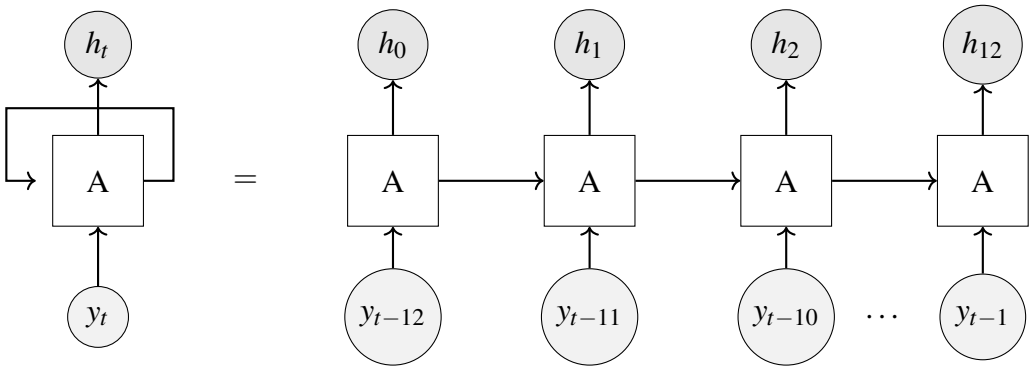


Figura 4.2: Architettura RNN per l'elaborazione ricorrente di una sequenza di input.

Questa differenza è cruciale: nelle RNN, l'informazione proveniente dai passi precedenti è trasmessa nel tempo tramite gli *hidden states*, che fungono da memoria dinamica della sequenza [12]. La capacità di memorizzazione di una rete ricorrente dipende principalmente da due fattori:

- *Sequence Length* (o *length*): indica il numero di passi temporali forniti in input alla rete. Una lunghezza maggiore consente di considerare un intervallo più ampio del passato, ma all'aumentare della profondità temporale emergono difficoltà computazionali, come il problema del vanishing gradient [3], che ostacola l'apprendimento di dipendenze di lungo periodo;
- *Hidden State Size* (o *depth*): rappresenta la dimensionalità dello stato nascosto in ciascun passo. Valori più elevati permettono di codificare informazioni più complesse, ampliando la capacità della rete di rappresentare strutture dinamiche e non lineari nella sequenza.

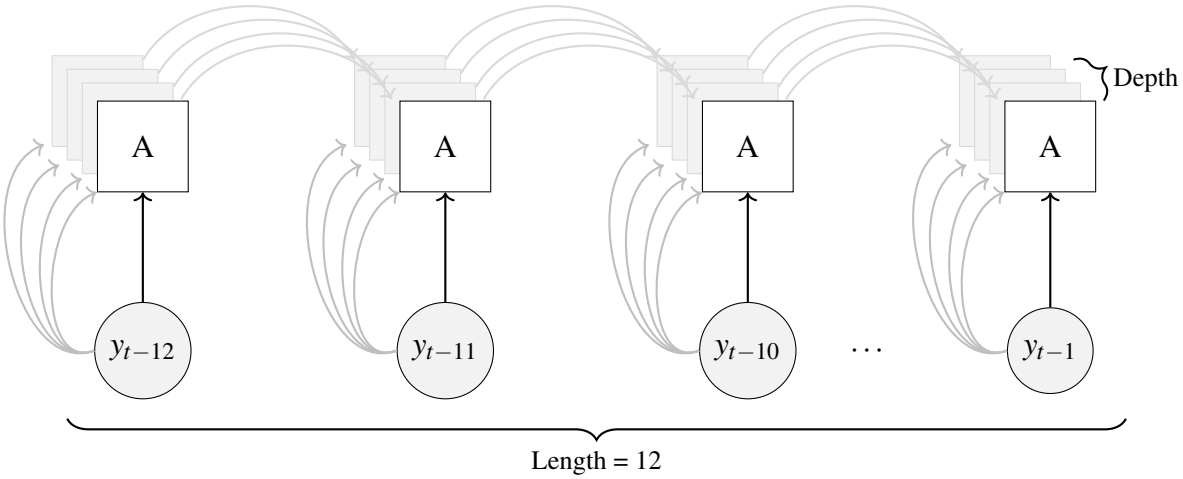


Figura 4.3: Rappresentazione *unrolled* di una RNN.

Nel contesto dell'elaborazione dei dati sequenziali, come quelli tipici delle serie temporali, una rete neurale efficace deve soddisfare i seguenti requisiti:

- preservare l'ordine: mantenere la sequenza temporale delle osservazioni per catturare correttamente la struttura dei dati;
- gestire le dipendenze a lungo termine: informazioni provenienti da istanti passati possono risultare determinanti per le previsioni future;
- gestire lunghezze di input variabili: a differenza delle reti feedforward, che richiedono input di dimensione fissa, un modello sequenziale deve poter processare sequenze di lunghezza variabile;
- condivisione dei parametri lungo la sequenza: l'architettura deve applicare gli stessi pesi a ogni passo temporale, riducendo la complessità del modello e migliorando la generalizzazione.

Le considerazioni esposte motivano l'introduzione delle RNN, architetture progettate per superare i limiti delle reti feedforward grazie alla presenza di uno stato interno in grado di mantenere memoria nel tempo. Nella sezione successiva verrà analizzata in dettaglio la loro struttura e il loro funzionamento.

DNN (Deep Neural Network)	RNN (Recurrent Neural Network)
Efficace per dipendenze di breve periodo	Progettata per dati sequenziali
Usa valori ritardati come input indipendenti	Processa una sequenza passo dopo passo nel tempo
Non mantiene esplicitamente l'ordine temporale tra le osservazioni	Mantiene una memoria attraverso uno stato interno
La sequenza temporale viene ricostruita tramite feature ingegnerizzate	Cattura in modo esplicito l'ordine e le dipendenze nei dati
Nessuna memoria interna: la dipendenza temporale è solo implicita	La capacità di memoria dipende dalla lunghezza della sequenza e dalla dimensione dello stato interno
Sensibile allo shuffle dei dati durante l'addestramento	L'ordine degli input è cruciale per la dinamica del modello

Tabella 4.2: Confronto tra reti neurali feedforward e ricorrenti.

4.2 Recurrent Neural Networks: struttura e funzionamento

L'elemento distintivo delle Recurrent Neural Networks, rispetto alle reti feedforward, è la capacità di trattare sequenze di dati mantenendo una forma di memoria interna. In ogni istante temporale, una cella ricorrente riceve in input non solo il dato corrente della sequenza, ma anche l'informazione proveniente dallo stato precedente. Questo meccanismo consente alla rete di "ricordare" il contesto e adattare il proprio comportamento in funzione degli elementi già elaborati.

In pratica, l'output prodotto da ciascuna cella viene utilizzato come input per la cella successiva nel tempo, creando un collegamento ricorrente tra i vari istanti temporali. Tale struttura può essere rappresentata in forma unrolled (srotolata lungo l'asse temporale), per evidenziare come le informazioni si propaghino attraverso i diversi *time step*:

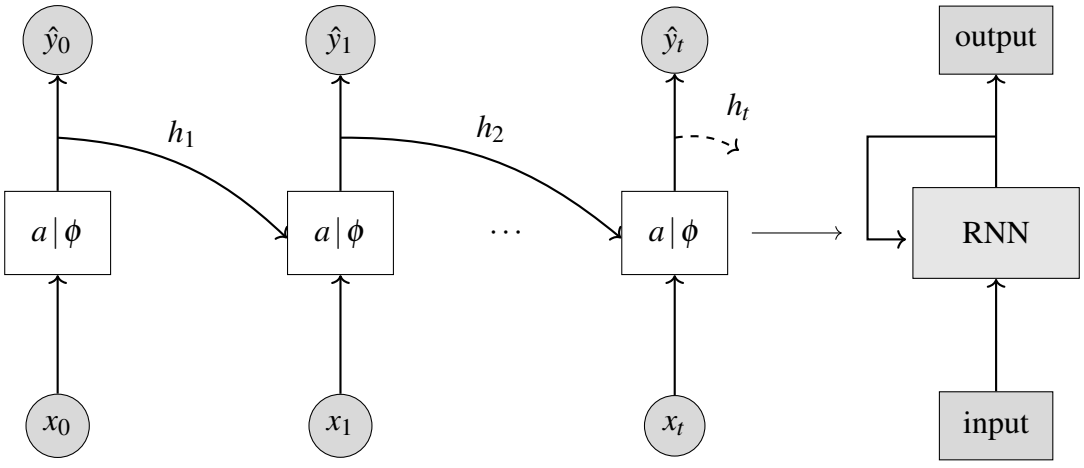


Figura 4.4: Sviluppo temporale di una rete neurale ricorrente.

Nel modello ricorrente si introduce uno stato nascosto h_t, anche detto memoria interna, che ha il compito di riassumere le informazioni accumulate fino all'istante t. Questo stato evolve

nel tempo in base sia al nuovo input x_t, sia allo stato precedente h_{t-1}:

$$h_t = f(x_t, h_{t-1}) \tag{4.1}$$

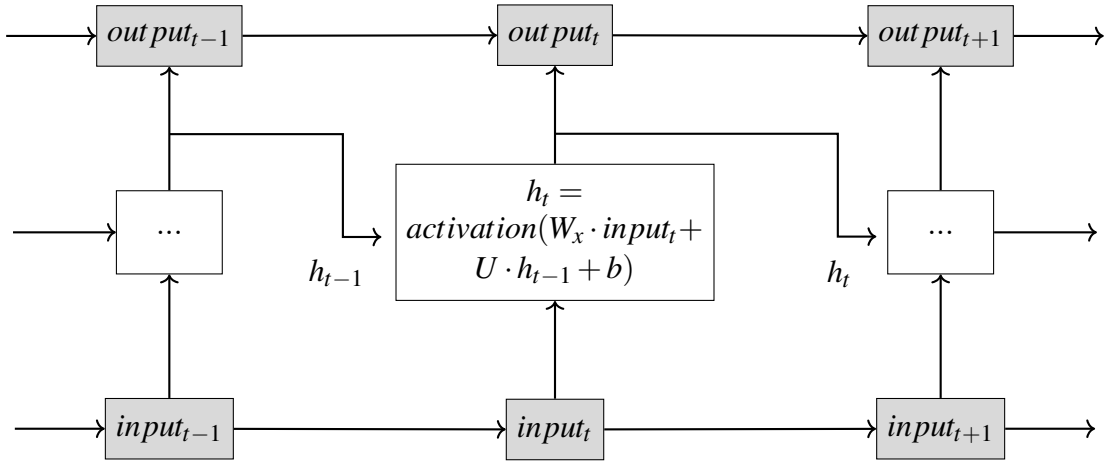


Figura 4.5: Interno di una cella ricorrente.

La dinamica della rete può essere formalizzata come:

$$h_t = \text{activation}(W_x x_t + U h_{t-1} + b) \tag{4.2}$$

dove:

- x_t: input al tempo t;
- h_{t-1}: stato nascosto (memoria interna) al tempo $t-1$;
- W_x: matrice dei pesi che collega l'input corrente allo stato nascosto;
- U: matrice dei pesi ricorrenti, che collega lo stato precedente a quello attuale;
- b: termine di bias;
- *activation*: funzione di attivazione non lineare, tipicamente tanh o ReLU.

A partire dallo stato nascosto h_t, si può calcolare l'output esterno della rete nel medesimo istante mediante una trasformazione lineare:

$$y_t = W_y h_t + c \tag{4.3}$$

dove:

- W_y: matrice dei pesi che proietta lo stato nascosto nello spazio dell'output;
- c: termine di bias associato all'output.

In alcune architetture, come nel caso *many-to-one*, l'output $\hat{y}_t$ è calcolato solo al termine della sequenza, a partire dall'ultimo stato nascosto h_t. In questi casi, la cella ricorrente calcola la successione degli stati $h_1, h_2, \ldots, h_t$, e l'output finale viene ottenuto proiettando h_t nello spazio dell'output tramite una trasformazione lineare.

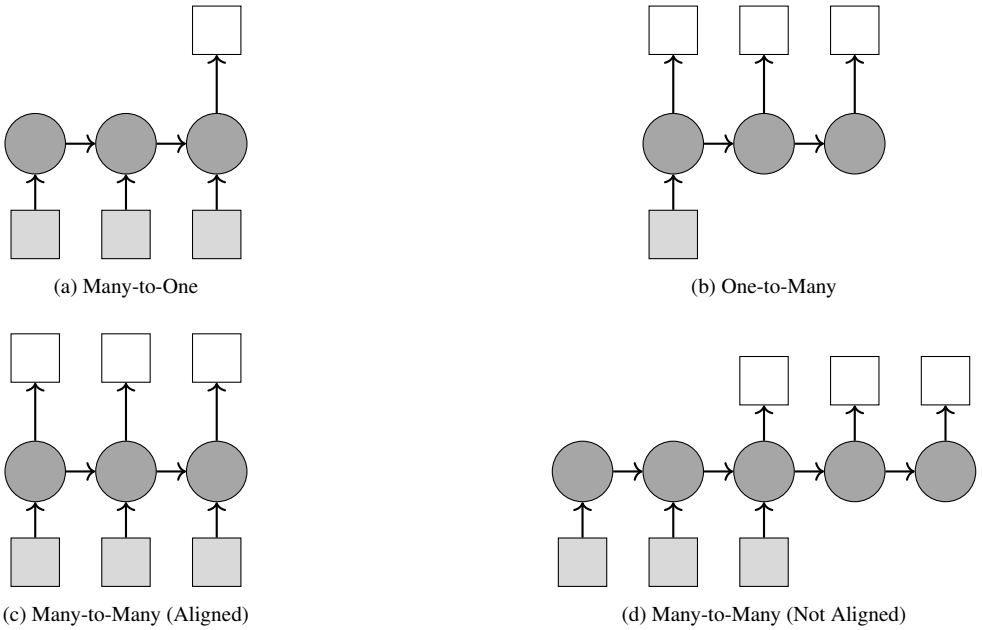
Nei modelli più generali, come quelli utilizzati per la classificazione o la generazione sequenziale, si mantiene invece una distinzione tra memoria interna e output, introducendo una funzione specifica per il calcolo di y_t.

Questa struttura permette alla rete di elaborare sequenze di lunghezza arbitraria, adattandosi dinamicamente alla storia passata della sequenza. Tuttavia, come verrà discusso nel prosieguo, le RNN standard presentano alcune limitazioni nell'apprendimento di dipendenze a lungo termine, che hanno motivato lo sviluppo di architetture più avanzate come le LSTM e le GRU.

4.3 Architetture delle RNN: strutture input-output

A seconda della natura del problema da risolvere, le RNN possono essere organizzate secondo diverse architetture di input-output. Le principali strutture sono le seguenti:

- *One-to-One*: rappresenta la struttura classica delle reti feedforward, in cui a un singolo input corrisponde un singolo output.
- *Many-to-One*: la rete riceve in input una sequenza temporale e produce un unico output.
- *One-to-Many*: un singolo input genera una sequenza di output.
- *Many-to-Many (aligned)*: in questo schema, input e output sono sequenze di uguale lunghezza [39] e ad ogni elemento della sequenza di input corrisponde un output specifico.
- *Many-to-Many (not aligned)*: sia gli input sia gli output sono sequenze, ma di lunghezza differente. In questo caso non esiste una corrispondenza temporale diretta tra input e output.



(a) Many-to-One

(b) One-to-Many

(c) Many-to-Many (Aligned)

(d) Many-to-Many (Not Aligned)

Figura 4.6: Configurazioni input-output nelle RNN.

Nel contesto della previsione di serie storiche finanziarie, l'architettura many-to-one è quella più comunemente adottata:

$$(x_{t-n}, \ldots, x_{t-1}, x_t) \longrightarrow \hat{y}_{t+1} \tag{4.4}$$

In questo caso:

- l'*input* è costituito da una sequenza di n osservazioni passate (ad esempio, gli ultimi n giorni di prezzi o di volatilità stimata);
- l'*output* rappresenta la previsione per il passo temporale successivo (ad esempio, la volatilità realizzata o il prezzo previsto al tempo $t+1$).

Questa configurazione è particolarmente adatta ai problemi di previsione della volatilità, poiché consente di modellare la dipendenza dinamica tra i valori passati e futuri, mantenendo esplicitamente la struttura temporale dei dati.

4.4 Apprendimento nelle RNN: Backpropagation Through Time

Le Recurrent Neural Networks sono in grado di elaborare e memorizzare efficacemente informazioni contenute in dati sequenziali grazie all'algoritmo di retropropagazione del gradiente, analogamente a quanto avviene nelle reti feedforward. Tuttavia, a causa della natura temporale dei dati trattati, le RNN richiedono un'estensione specifica di questo algoritmo, nota come *Backpropagation Through Time* (BPTT) [16].

La BPTT consente di aggiornare i pesi delle RNN calcolando la derivata della funzione di perdita complessiva rispetto ai parametri della rete, tenendo conto della struttura ricorrente distribuita lungo i vari time step. A ogni istante temporale t, la rete riceve un input x_t, aggiorna lo stato nascosto h_t, produce un output predetto $\hat{y}_t$ e calcola una perdita locale J_t, confrontando $\hat{y}_t$ con il target reale y_t.

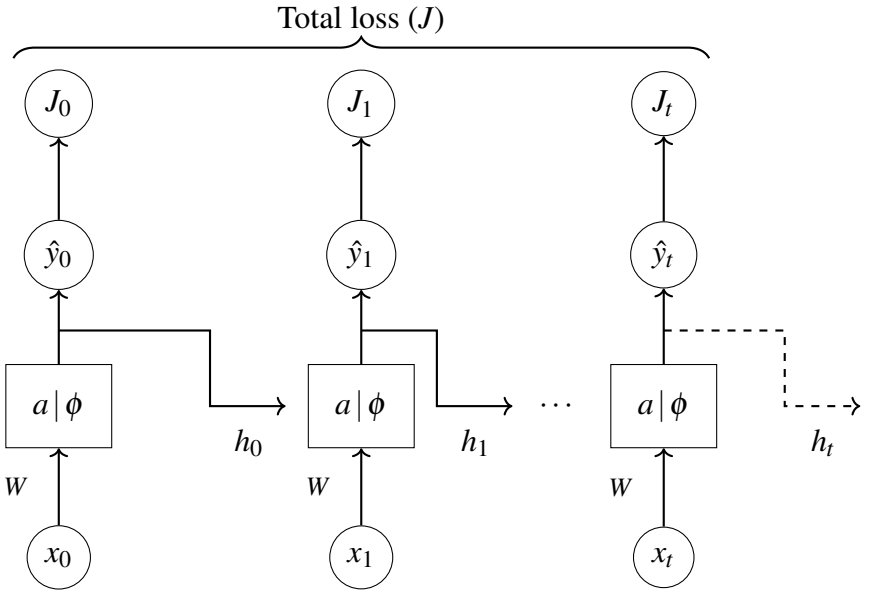


Figura 4.7: Backpropagation Through Time

La funzione di perdita totale J è quindi definita come la somma delle perdite locali nel tempo:

$$J = \sum_{k=0}^{T} J_k \tag{4.5}$$

Per aggiornare i pesi è necessario calcolare il gradiente della funzione di perdita rispetto ai parametri della rete:

$$\frac{\partial J}{\partial P} \quad \text{con} \quad P \in \{W_x, U, W_y\} \tag{4.6}$$

Ad esempio, il gradiente totale rispetto alla matrice dei pesi W_x (che collega l'input allo stato nascosto) è dato da:

$$\frac{\partial J}{\partial W_x} = \sum_{k=0}^{T} \frac{\partial J_k}{\partial W_x} \tag{4.7}$$

Ogni termine può essere esplicitato applicando la regola della catena. Ad esempio:

$$\frac{\partial J_0}{\partial W_x} = \frac{\partial J_0}{\partial \hat{y}_0} \cdot \frac{\partial \hat{y}_0}{\partial h_0} \cdot \frac{\partial h_0}{\partial W_x} \tag{4.8}$$

$$\frac{\partial J_1}{\partial W_x} = \frac{\partial J_1}{\partial \hat{y}_1} \cdot \frac{\partial \hat{y}_1}{\partial h_1} \cdot \frac{\partial h_1}{\partial W_x}, \quad \text{dove} \quad \frac{\partial h_1}{\partial W_x} = \frac{\partial h_1}{\partial h_0} \cdot \frac{\partial h_0}{\partial W_x} \tag{4.9}$$

Generalizzando, il gradiente della perdita al tempo t rispetto a W_x si può esprimere come:

$$\frac{\partial J_t}{\partial W_x} = \sum_{k=0}^{t} \frac{\partial J_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial h_k} \cdot \frac{\partial h_k}{\partial W_x} \tag{4.10}$$

Questo schema evidenzia come la retropropagazione attraversi l'intera catena temporale, coinvolgendo iterativamente tutti gli stati precedenti. Tale processo comporta un effetto numerico ben noto nella letteratura: la moltiplicazione ripetuta di derivate può causare instabilità nei gradienti. In particolare, si possono verificare due fenomeni principali:

- *Vanishing gradient*: quando i moduli delle derivate sono minori di 1, il gradiente si riduce progressivamente fino ad annullarsi. In questo caso, l'algoritmo non riesce ad aggiornare efficacemente i pesi relativi agli strati più lontani nel tempo;
- *Exploding gradient*: quando i moduli sono maggiori di 1, il gradiente può crescere esponenzialmente, causando instabilità numerica e difficoltà di convergenza durante l'ottimizzazione.

Questi problemi rappresentano una delle principali limitazioni delle RNN standard [35] e motivano l'introduzione di architetture più sofisticate, come le LSTM e le GRU, progettate per mitigare tali instabilità e favorire l'apprendimento di dipendenze temporali di lungo periodo.

4.5 Oltre le RNN: le motivazioni

Le RNN introducono la ricorrenza: ogni unità ricorrente mantiene uno stato interno che si aggiorna nel tempo, consentendo alla rete di catturare le relazioni dinamiche tra gli elementi della sequenza. Tuttavia, come evidenziato in precedenza, la propagazione del gradiente lungo la

sequenza può risultare instabile, soprattutto per serie di lunghezza elevata. In particolare, il problema del vanishing gradient limita la capacità delle RNN di apprendere dipendenze temporali di lungo periodo. Di conseguenza, queste reti tendono a conservare solo l'informazione relativa alla storia recente, perdendo progressivamente la memoria degli eventi più lontani nel tempo.

Nel corso degli anni sono stati proposti diversi approcci per mitigare tali problemi, agendo sia a livello di ottimizzazione sia di architettura:

- funzioni di attivazione meno soggette a saturazione, come la Leaky ReLU, che mantiene un gradiente diverso da zero anche per input negativi, mitigando i problemi di vanishing gradient tipici di funzioni fortemente saturanti come la sigmoid;
- inizializzazione dei pesi appropriata: fondamentale per prevenire, fin dalle prime iterazioni, l'annullamento o l'esplosione dei gradienti;
- algoritmi di ottimizzazione robusti: come Adam e RMSProp, più efficaci nella gestione di gradienti piccoli o rumorosi [24, 37];
- *batch normalization*: normalizzazione applicata ai minibatch, che stabilizza il flusso dei gradienti e accelera la convergenza;
- architetture con *gated cells*: che introducono percorsi informativi controllati, consentendo alla rete di decidere dinamicamente quali informazioni mantenere, aggiornare o dimenticare.

Tra queste soluzioni, le più influenti sono le LSTM, che potenziano la struttura delle RNN introducendo meccanismi interni di regolazione del flusso informativo. Queste reti sono in grado di apprendere dipendenze di lungo periodo, preservando nel tempo le informazioni rilevanti e attenuando le instabilità del gradiente.

Nel paragrafo successivo verrà analizzata in dettaglio la struttura interna delle LSTM, illustrandone il funzionamento e le componenti fondamentali.

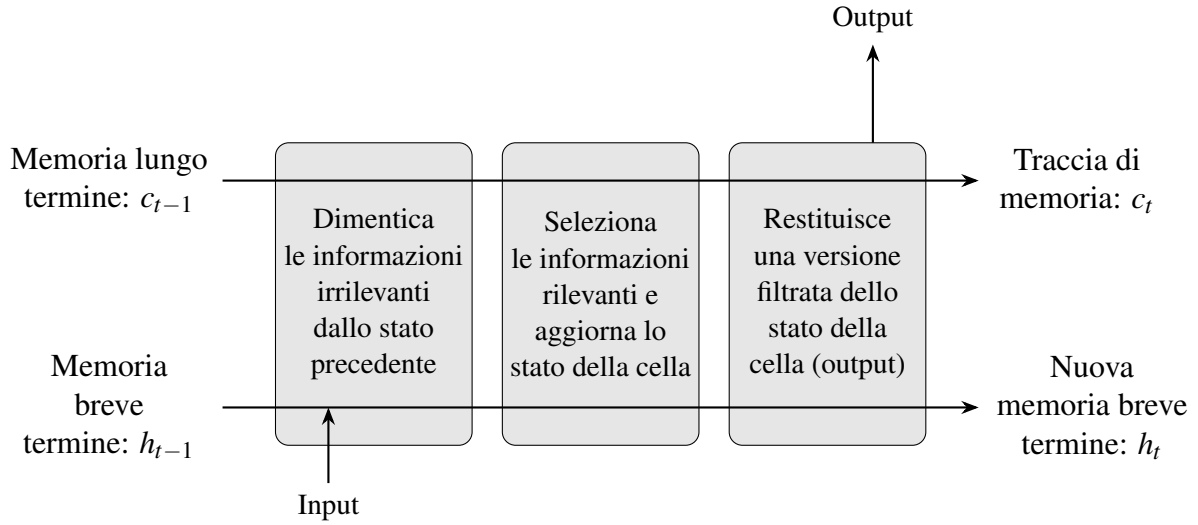


Figura 4.8: Funzionamento cella LSTM.

4.6 LSTM: Long Short-Term Memory

Le *Long Short-Term Memory* (LSTM) [19] rappresentano un'estensione delle Recurrent Neural Networks, progettate per superarne le principali limitazioni legate alla propagazione del gradiente. A differenza delle RNN tradizionali, le LSTM introducono una struttura interna più articolata in ciascuna unità ricorrente, dotandola di una cella di memoria e di tre componenti fondamentali, dette *gates* (porte), che regolano dinamicamente il flusso informativo all'interno della rete:

- *Forget gate*: decide quali informazioni dello stato precedente devono essere dimenticate;
- *Input gate*: seleziona le nuove informazioni da aggiungere alla memoria;
- *Output gate*: determina quali informazioni restituire come output della rete.

Ogni cella LSTM gestisce due stati distinti:

- lo stato nascosto h_t, che rappresenta la memoria a breve termine, analoga a quella delle RNN classiche;
- lo stato della cella c_t, che funge da memoria a lungo termine, capace di conservare informazioni rilevanti per molti passi temporali.

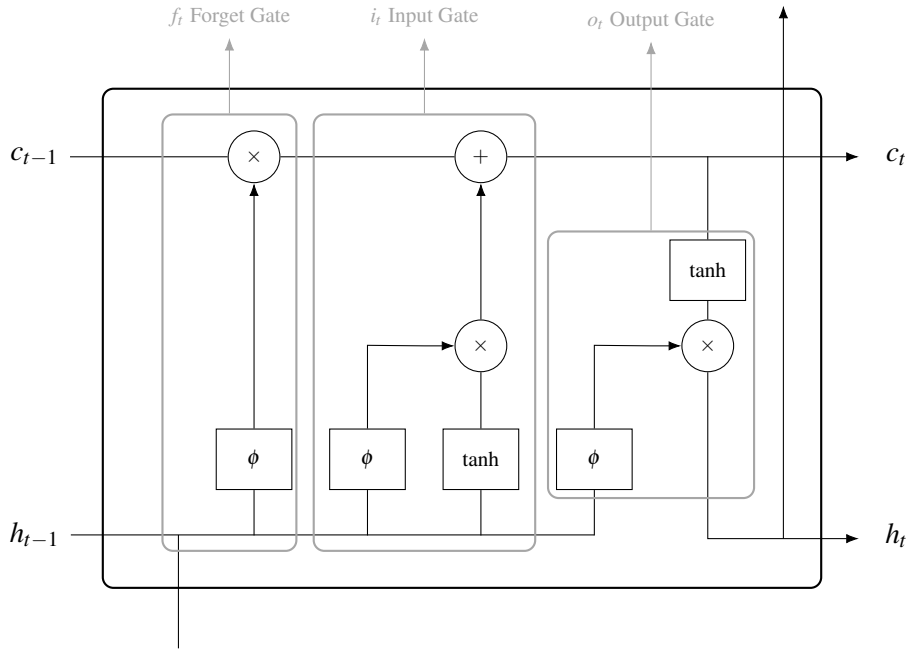


Figura 4.9: Struttura interna di una cella LSTM.

Forget gate

Il primo passo consiste nel decidere quali informazioni dello stato della cella precedente c_{t-1} devono essere mantenute. A tal fine, la forget gate calcola un vettore f_t tramite una funzione sigmoide:

$$f_t = \phi(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{4.11}$$

I valori di f_t, compresi tra 0 e 1, vengono moltiplicati elemento per elemento con c_{t-1}, determinando così quali informazioni dimenticare (valori prossimi a 0) e quali conservare (valori prossimi a 1).

Input gate

La seconda fase consiste nell'aggiornare la memoria della cella con nuove informazioni. Questo avviene in due passaggi distinti:

- calcolo del vettore di attivazione i_t tramite una funzione sigmoide:

$$i_t = \phi(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{4.12}$$

- calcolo del vettore dei valori candidati $\tilde{c}_t$ mediante una funzione tangente iperbolica:

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \tag{4.13}$$

Il prodotto $i_t \cdot \tilde{c}_t$ definisce le nuove informazioni da aggiungere allo stato della cella. Lo stato aggiornato è quindi:

$$c_t = f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t \tag{4.14}$$

Output gate

Infine, si determina lo stato nascosto h_t, ovvero l'output della cella, attraverso due operazioni:

- calcolo del gate di uscita:

$$o_t = \phi(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{4.15}$$

- calcolo dello stato nascosto:

$$h_t = o_t \cdot \tanh(c_t) \tag{4.16}$$

Il vettore h_t rappresenta l'output corrente della cella LSTM e viene trasmesso sia allo step temporale successivo, sia eventualmente al layer successivo della rete.

Proprietà delle LSTM

Le LSTM si distinguono per tre proprietà fondamentali:

- controllo dinamico del flusso informativo, grazie all'azione combinata delle tre porte che regolano selettivamente i percorsi di memoria;
- memoria a lungo termine efficiente, resa possibile dalla struttura dello stato della cella c_t e dal suo meccanismo di aggiornamento additivo;
- robustezza al vanishing gradient, favorita dalla presenza di percorsi diretti all'interno della cella che non attraversano attivazioni non lineari.

Grazie a queste caratteristiche, le LSTM risultano particolarmente efficaci nei compiti di sequence modeling, come la previsione di serie temporali, l'elaborazione del linguaggio naturale e la generazione di sequenze complesse.

4.7 GRU: Gated Recurrent Units

Le *Gated Recurrent Units* (GRU) [9], rappresentano una variante semplificata delle Long Short-Term Memory, progettate per ridurre la complessità computazionale e il numero di parametri, mantenendo al contempo la capacità di apprendere dipendenze temporali di lungo periodo. A differenza delle LSTM, che impiegano due stati distinti (hidden state h_t e cell state c_t), le GRU combinano tali informazioni in un unico vettore di stato h_t, semplificando la dinamica interna della rete. La struttura di una cella GRU include due gates principali:

- *Update gate* (ug_t): controlla quanto dello stato precedente deve essere mantenuto e quanto sostituito con le nuove informazioni;
- *Reset gate* (rg_t): regola quanta parte della memoria passata deve essere "dimenticata" prima di calcolare la nuova attivazione candidata.

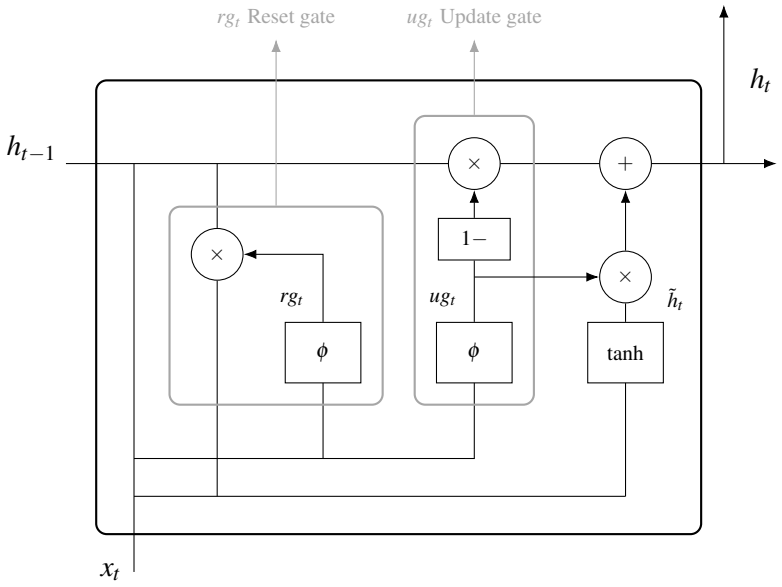


Figura 4.10: Struttura interna di una cella GRU.

La dinamica della cella può essere descritta dalle seguenti equazioni:

$$ug_t = \phi(W_u \cdot [h_{t-1}, x_t] + b_u) \tag{4.17}$$

$$rg_t = \phi(W_r \cdot [h_{t-1}, x_t] + b_r) \tag{4.18}$$

$$\tilde{h}_t = \tanh(W_h \cdot [rg_t \odot h_{t-1}, x_t] + b_h) \tag{4.19}$$

$$h_t = (1 - ug_t) \odot h_{t-1} + ug_t \odot \tilde{h}_t \tag{4.20}$$

dove $\phi(\cdot)$ indica la funzione sigmoide e $\odot$ rappresenta la moltiplicazione elemento per elemento. L'update gate ug_t agisce in modo analogo al forget gate e all'input gate combinati delle LSTM, riducendo così il numero di parametri e aumentando la stabilità del processo di addestramento.

Dal punto di vista applicativo, le GRU risultano particolarmente efficaci nel forecasting di serie temporali finanziarie, dove la lunghezza moderata delle sequenze e la presenza di relazioni non lineari rendono vantaggiosa una struttura più compatta e veloce da addestrare rispetto alle LSTM. Questa efficienza, unita alla capacità di catturare pattern dinamici di breve e medio periodo, rende le GRU una scelta appropriata per la previsione della volatilità realizzata.

4.8 CNN-1D: reti neurali convoluzionali monodimensionali

Le *Convolutional Neural Networks* (CNN) sono state originariamente sviluppate per l'elaborazione di immagini bidimensionali [26], ma la loro struttura si presta anche alla modellazione di sequenze temporali, nella forma monodimensionale o *one-dimensional* (1D). La CNN-1D applica un insieme di filtri convoluzionali a una finestra temporale di dati per estrarre pattern locali utili alla previsione della volatilità realizzata. Ogni filtro agisce come un rilevatore di schemi ricorrenti all'interno della sequenza: andamenti crescenti, decrescenti, brusche variazioni, combinazioni particolari tra volatilità e rendimento, e così via.

Principio di funzionamento

Una rete CNN-1D applica una serie di filtri (o *kernel*) che scorrono sulla sequenza di input, calcolando combinazioni lineari locali dei dati. Ogni filtro agisce su una finestra temporale di lunghezza prefissata e produce una nuova sequenza detta *feature map*, che rappresenta la risposta del filtro ai diversi segmenti del segnale.

Formalmente, dato un segnale in input

$$x = [x_1, x_2, \ldots, x_T] \in \mathbb{R}^T \tag{4.21}$$

e un filtro

$$k = [k_{-m}, \ldots, k_0, \ldots, k_m] \in \mathbb{R}^{2m+1}, \tag{4.22}$$

l'operazione di convoluzione discreta è definita come:

$$h(t) = (x * k)(t) = \sum_{\tau=-m}^{m} x(t-\tau)\, k(\tau), \qquad |k| = 2m+1. \tag{4.23}$$

dove $h(t)$ rappresenta l'output al tempo t.

In termini intuitivi, un filtro convoluzionale può essere visto come un rilevatore di pattern locali: produce un valore elevato quando il segmento di input corrisponde alla forma dei pesi del filtro (ad esempio, un picco improvviso o un tratto piatto della serie). Nel contesto della volatilità, ciò significa che una CNN-1D può identificare automaticamente configurazioni caratteristiche associate a periodi di alta o bassa variabilità dei rendimenti.

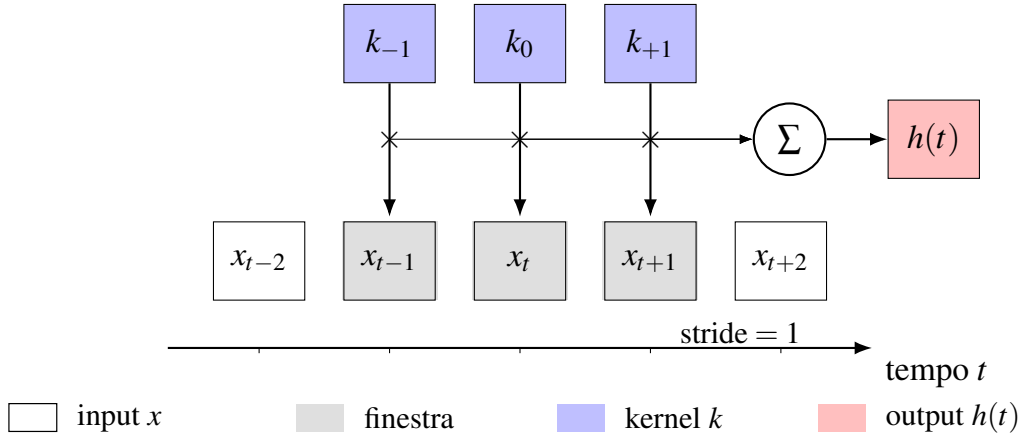


Figura 4.11: Architettura CNN-1D.

Apprendimento dei filtri

Durante l'addestramento, i pesi del filtro k vengono ottimizzati minimizzando una funzione di perdita $\mathscr{L}_h$, che misura la discrepanza tra l'output della rete e il valore osservato. Il gradiente utilizzato per aggiornare ciascun coefficiente $k(\tau_0)$ è calcolato come:

$$\frac{\partial \mathscr{L}_h}{\partial k(\tau_0)} = \sum_{t=1}^{T} \frac{\partial \mathscr{L}_h}{\partial h(t)} \sum_{\tau=-m}^{m} x(t-\tau) \frac{\partial k(\tau)}{\partial k(\tau_0)} \tag{4.24}$$

Poiché $\frac{\partial k(\tau)}{\partial k(\tau_0)} = 0$ per $\tau \neq \tau_0$, l'aggiornamento di ogni peso dipende solo dai punti dell'input effettivamente coinvolti nella finestra del filtro. Questo meccanismo è alla base dell'efficienza parametrica delle convoluzioni: gli stessi pesi vengono condivisi su tutte le posizioni dell'input, riducendo drasticamente il numero di parametri rispetto a una rete completamente connessa (MLP) [16].

Vantaggi principali

L'uso della convoluzione 1D offre tre vantaggi fondamentali rispetto alle reti feedforward tradizionali:

- efficienza dei parametri: i pesi del filtro sono condivisi per tutte le posizioni temporali, riducendo la complessità del modello e il rischio di overfitting.
- efficienza dei dati: un filtro addestrato per riconoscere un pattern (ad esempio, un picco di volatilità) sarà in grado di rilevarlo ovunque compaia nella sequenza temporale.
- migliore generalizzazione: grazie alla proprietà di equivarianza rispetto alle traslazioni temporali, il modello produce risposte simili quando un pattern viene spostato nel tempo, migliorando la capacità di generalizzare su dati non osservati.

Gestione dei bordi: il *padding*

Un problema comune nelle convoluzioni è la gestione dei bordi della sequenza (*boundary problem*). Quando il filtro viene applicato vicino agli estremi, alcuni valori dell'input mancano, impedendo il calcolo completo della finestra. Per ovviare a ciò, si introduce il *padding*, ossia l'aggiunta di valori artificiali (spesso zeri) all'inizio e alla fine della sequenza. Le strategie principali sono:

- *Valid padding*: non vengono aggiunti valori; l'output risulta più corto rispetto all'input.
- *Same padding*: vengono aggiunti $\frac{(\text{kernel size})-1}{2}$ valori ai bordi, in modo da mantenere la stessa lunghezza temporale.
- *Causal padding*: vengono aggiunti valori solo a sinistra, assicurando che ogni previsione al tempo t dipenda unicamente da osservazioni passate $(x_{t-1}, x_{t-2}, \dots)$, nel rispetto della causalità temporale.

Nel contesto della previsione della volatilità, il causal padding è particolarmente rilevante, poiché evita che il modello utilizzi informazioni future non disponibili al momento della previsione.

Riduzione dimensionale: *stride* e *pooling*

Applicare la convoluzione a ogni passo temporale produce un output della stessa lunghezza dell'input, aumentando i costi computazionali e limitando la capacità di astrazione del modello. Per controllare la dimensione delle feature maps e migliorare la robustezza, si introducono due meccanismi principali:

- *Stride*: indica di quanti passi si sposta il filtro lungo la sequenza. Con stride pari a 2, ad esempio, il filtro avanza ogni due osservazioni, riducendo la lunghezza dell'output e la risoluzione temporale della rappresentazione.
- *Pooling*: layer dedicato alla sintesi dell'informazione estratta dai filtri. Le principali varianti sono:
  - *Max pooling*: conserva il valore massimo all'interno della finestra, risultando efficace nell'individuare picchi locali di volatilità;
  - *Average pooling*: calcola la media delle attivazioni, contribuendo ad attenuare rumore e fluttuazioni transitorie.

Oltre alle forme di pooling locale, nelle applicazioni di previsione su serie temporali è frequentemente impiegato il *Global Average Pooling*, che aggrega ciascuna feature map calcolandone la media sull'intera dimensione temporale. Tale operazione consente di ottenere una rappresentazione compatta e stabile, riducendo il numero di parametri e limitando il rischio di overfitting.

Applicazioni al forecasting della volatilità

Nel contesto della previsione della volatilità realizzata, le reti CNN-1D permettono di catturare pattern temporali locali, come episodi di alta volatilità o transizioni di regime, mantenendo al contempo la coerenza temporale grazie al padding causale. I filtri convoluzionali fungono da estrattori automatici di caratteristiche, individuando configurazioni ricorrenti nei rendimenti passati o nella volatilità storica. Combinando più livelli convoluzionali, il modello costruisce rappresentazioni gerarchiche della dinamica dei mercati, dalle fluttuazioni giornaliere fino alle tendenze di più lungo periodo.

Nel complesso, le reti CNN-1D rappresentano un compromesso efficace tra semplicità computazionale e capacità di modellare la struttura temporale locale. Tuttavia, per cogliere dipendenze a più lungo orizzonte e componenti interpretabili di trend e stagionalità, si sono affermate architetture più avanzate come quella analizzata nella sezione successiva.

4.9 N-BEATS: un approccio avanzato per il forecasting

Le *Neural Basis Expansion Analysis for Interpretable Time Series Forecasting* (N-BEATS) [34] rappresentano una delle architetture più recenti ed efficaci per la previsione di serie temporali univariate. L'idea fondamentale di N-BEATS consiste nel costruire la previsione in modo progressivo, scomponendo l'andamento della serie storica in contributi successivi che spiegano porzioni sempre più raffinate del segnale osservato.

A differenza delle reti ricorrenti o convoluzionali, N-BEATS non utilizza meccanismi di memoria né operazioni di convoluzione. La rete è invece composta da una sequenza di moduli basati esclusivamente su strati completamente connessi, che operano in modo iterativo.

Ciascun modulo tenta di spiegare una parte dell'andamento passato della serie e, sulla base di questa informazione, fornisce un contributo alla previsione futura. La parte già spiegata viene progressivamente rimossa dall'input, permettendo ai moduli successivi di concentrarsi sulle componenti residue della dinamica temporale.

Questo meccanismo di correzione iterativa consente a N-BEATS di catturare strutture temporali complesse pur mantenendo un'architettura concettualmente semplice e facilmente interpretabile.

Struttura generale del modello

Il modello riceve in input una finestra storica di lunghezza L (*lookback window*) e produce una previsione su un orizzonte futuro di H passi (*forecast horizon*). L'architettura è organizzata in più *stack*, ciascuno dei quali contiene una sequenza di blocchi basati su reti neurali completamente connesse. Ogni blocco elabora l'informazione ricevuta e produce due risultati distinti:

- una componente detta *backcast*, che rappresenta la parte della serie passata che il blocco riesce a spiegare;
- un'altra componente detta *forecast*, che costituisce un contributo parziale alla previsione dei valori futuri.

Il backcast viene sottratto dall'input del blocco, generando un residuo che viene passato al blocco successivo. In questo modo, ciascun blocco opera su una versione sempre più "pulita" del segnale, migliorando progressivamente la qualità della previsione. La Figura 4.12 illustra schematicamente questo processo iterativo [34].

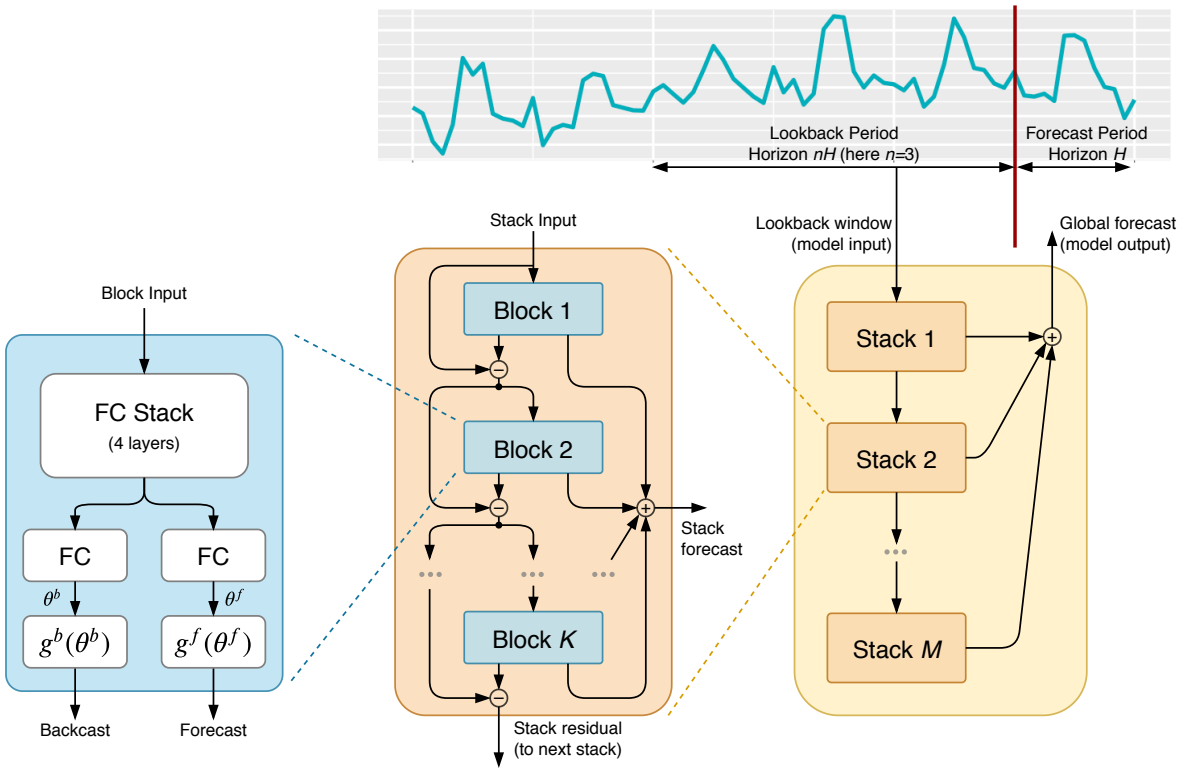


Figura 4.12: Architettura del modello N-BEATS. Fonte: [34].

Formalizzazione del meccanismo residuo

Indicando con $x_{t-L+1:t}$ la finestra storica iniziale e con $r^{(i,j)}$ il residuo in ingresso al blocco j dello stack i, ciascun blocco produce un backcast $b^{(i,j)} \in \mathbb{R}^L$ e un forecast parziale $f^{(i,j)} \in \mathbb{R}^H$. Il residuo viene aggiornato sottraendo la parte spiegata:

$$r^{(i,j+1)} = r^{(i,j)} - b^{(i,j)}, \tag{4.25}$$

mentre la previsione complessiva del modello è ottenuta sommando tutti i contributi forniti dai singoli blocchi:

$$\hat{y}_{t+1:t+H} = \sum_{i=1}^{S} \sum_{j=1}^{B_i} f^{(i,j)}. \tag{4.26}$$

Stack generici e interpretabili

Gli autori distinguono due principali configurazioni del modello:

- *stack generici*, in cui backcast e forecast sono appresi liberamente dal modello, massimizzando la flessibilità e l'accuratezza predittiva;
- *stack interpretabili*, in cui le uscite dei blocchi sono vincolate a basi predefinite che rappresentano componenti di trend e stagionalità, facilitando l'interpretazione della previsione.

Questa distinzione consente di bilanciare accuratezza e interpretabilità, rendendo N-BEATS adatto sia a contesti puramente previsivi sia ad analisi in cui è rilevante comprendere la struttura della dinamica temporale appresa.

Addestramento e applicazioni

Il modello viene addestrato minimizzando una funzione di perdita di tipo MSE, utilizzando l'ottimizzatore Adam e tecniche di early stopping per prevenire fenomeni di overfitting. Nel contesto della previsione della volatilità, N-BEATS si è dimostrato particolarmente efficace nel catturare pattern non lineari e variazioni di medio periodo, mantenendo al contempo stabilità numerica e semplicità di addestramento grazie alla sua natura puramente feedforward.

Capitolo 5

Confronto empirico dei modelli

Procediamo ora con il confronto empirico tra modelli parametrici e architetture neurali, con l'obiettivo di valutarne la capacità predittiva sulla volatilità nei mercati finanziari. L'analisi è condotta in primo luogo sull'S&P 500, che rappresenta il campione di riferimento; successivamente i risultati verranno estesi ad altre serie storiche per includere diverse tipologie di mercati finanziari.

La sezione presente descrive il dataset utilizzato, il periodo di analisi e le procedure adottate. Successivamente vengono presentati e discussi i risultati ottenuti dalle specificazioni GARCH e dai modelli neurali, includendo la diagnostica dei residui, la significatività dei parametri e le prestazioni predittive sia *in-sample* (IS) che *out-of-sample* (OOS).

5.1 Dataset e analisi preliminare

L'analisi viene condotta utilizzando la serie giornaliera *SPX Index* estratta da Bloomberg; in particolare, si considera il prezzo di chiusura dell'indice, rappresentato dalla variabile *PX_LAST*, che misura il livello dell'indice alla fine di ciascuna giornata di negoziazione. Il periodo di osservazione va dal 1° gennaio 1950 al 31 dicembre 2025, includendo fasi di mercato profondamente eterogenee. Ciò consente di valutare la robustezza dei modelli in contesti macrofinanziari molto differenti.

Il campione complessivo viene suddiviso secondo una struttura temporale che differisce in funzione della classe di modelli considerata. Per i modelli econometrici di tipo GARCH, che non richiedono una fase di validazione separata, l'analisi si basa su una suddivisione train/test pari all'80%–20% del campione, dove la prima parte è utilizzata per la stima dei parametri e la seconda per la valutazione OOS delle previsioni di volatilità.

Le architetture neurali, invece, richiedono una procedura di selezione degli iperparametri e di controllo della convergenza più articolata. A tal fine, il campione storico precedente al 2020 viene inizialmente suddiviso secondo uno schema train/validation 60%–20%, riservando il restante 20% del campione complessivo al test. Il training set viene impiegato per l'apprendimento dei pesi della rete, mentre il validation set è utilizzato per il tuning degli iperparametri e per l'applicazione di criteri di early stopping, al fine di mitigare fenomeni di overfitting.

Una volta individuata la configurazione ottimale, le reti neurali vengono riaddestrate sull'intero campione di stima pari all'80% iniziale e valutate sul restante 20%, coerentemente con quanto fatto per i modelli GARCH. In questo modo è garantita la piena comparabilità dei risultati.

La differenza metodologica tra le due classi di modelli riflette la loro natura intrinseca. I modelli GARCH sono caratterizzati da un numero limitato di parametri strutturali e non richie-

dono una procedura iterativa di validazione, mentre le reti neurali sono sensibili alla scelta degli iperparametri e necessitano di una fase di validazione esplicita per controllare la complessità del modello e la stabilità dell'apprendimento. In entrambi i casi, il periodo di test rimane completamente escluso dalla fase di stima e selezione, assicurando un confronto equo in termini di capacità predittiva.

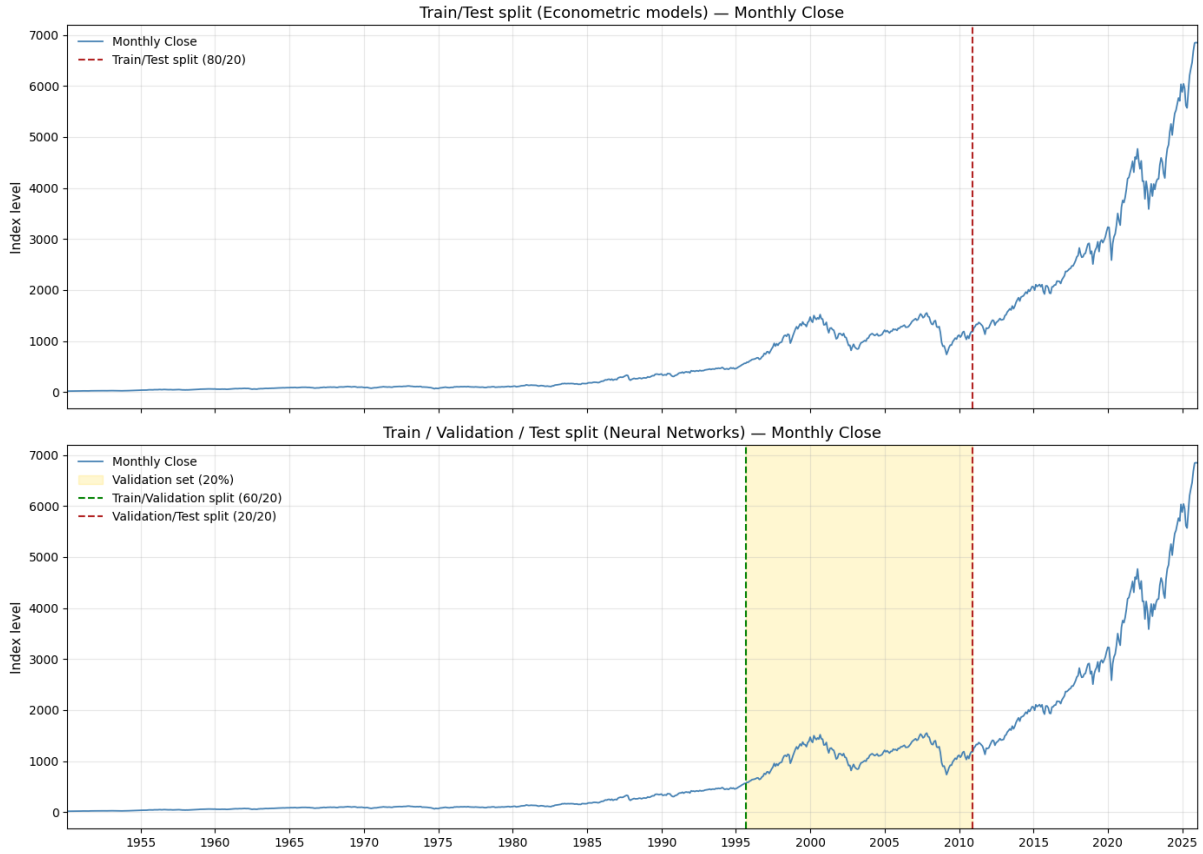


Figura 5.1: Suddivisione temporale del dataset per i modelli econometrici e neurali.

A partire dai prezzi di chiusura giornalieri $P_{d,m}$ vengono calcolati i rendimenti logaritmici giornalieri $ret_{d,m}$ secondo la relazione:

$$ret_{d,m} = \log(P_{d,m}) - \log(P_{d-1,m}), \tag{5.1}$$

dove $P_{d,m}$ indica il prezzo di chiusura dell'indice nel giorno d del mese m. Il rendimento mensile è quindi ottenuto aggregando i rendimenti giornalieri all'interno del mese:

$$ret_m = \sum_{d=1}^{n} ret_{d,m}, \tag{5.2}$$

dove n indica il numero di giorni di negoziazione nel mese m. La disponibilità dei dati giornalieri è infatti necessaria per la costruzione della volatilità realizzata mensile, che costituisce la variabile target dei modelli, come definito nell'Equazione (1.9).

Di conseguenza, sebbene l'informazione di partenza sia giornaliera, tutti i modelli stimati, sia le specificazioni GARCH sia le architetture neurali, sono allenati su dati a frequenza mensile (31-01-1950 – 31-12-2025). Le statistiche descrittive delle serie sono riportate nella Tabella 5.1.

	Panel A: Daily		Panel B: Monthly		
Statistica	P_d	ret_d	P_m	ret_m	RV_m
Osservazioni	19,121	19,121	912	912	912
Media	916.357	0.000315	919.319	0.006599	0.134707
Dev. std.	1,312.331	0.009953	1,320.186	0.041977	0.081833
Minimo	16.670	-0.228997	17.050	-0.245428	0.030814
1° quartile	89.560	-0.004060	89.333	-0.017521	0.086661
Mediana	272.310	0.000515	273.600	0.009986	0.113746
3° quartile	1,268.210	0.005078	1,271.815	0.034629	0.158194
Massimo	6,932.050	0.109572	6,849.090	0.151043	0.965484

Tabella 5.1: Statistiche descrittive dell'indice S&P 500 (SPX Index, *PX_LAST*)

Analisi della serie ai livelli

Prima di procedere alla modellazione della volatilità è necessario verificare le proprietà statistiche della serie dei prezzi ai livelli. La stazionarietà rappresenta infatti un requisito fondamentale per l'applicazione dei modelli GARCH, i quali assumono che la dinamica dei ritorni sia caratterizzata da media e varianza stabili nel tempo. A tale scopo analizzo dapprima l'*Autocorrelation Function* (ACF) e la *Partial Autocorrelation Function* (PACF) della serie P_m. Le funzioni ACF e PACF dei livelli presentano autocorrelazioni persistenti e lentamente decrescenti, evidenza tipica di una serie non stazionaria:

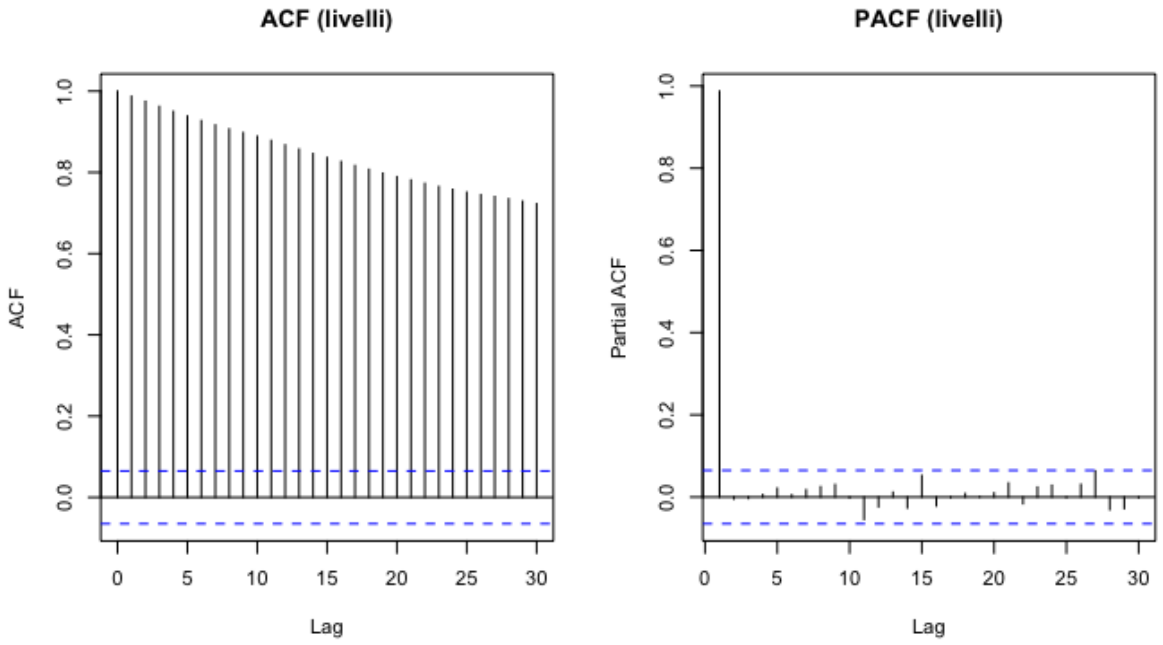


Figura 5.2: ACF e PACF della serie dei prezzi dell'S&P 500.

L'ACF mostra un decadimento molto lento, caratteristico della presenza di una radice unitaria. Per confermare questa evidenza applico l'*Augmented Dickey–Fuller* (ADF) test, con ipotesi nulla di non stazionarietà:

H_0 : la serie presenta una radice unitaria, H_1 : la serie è stazionaria.

Specificazione	Statistica	1% critico	5% critico	10% critico
Trend (`type = "trend"`)	4.823	−3.96	−3.41	−3.12
Drift (`type = "drift"`)	6.762	−3.43	−2.86	−2.57
Nessuna (`type = "none"`)	7.644	−2.58	−1.95	−1.62

Tabella 5.2: Risultati del test ADF sui livelli.

In tutte le specificazioni considerate (con trend, con drift e senza termine deterministico), la statistica ADF risulta meno estrema dei rispettivi valori critici, impedendo il rigetto dell'ipotesi nulla. Ciò conferma in modo formale che la serie ai livelli non è stazionaria e non può essere utilizzata direttamente nei modelli a varianza condizionata. La non stazionarietà dei prezzi motiva quindi la trasformazione in rendimenti.

Rendimenti logaritmici

La Figura 5.3 mostra la serie dei rendimenti logaritmici mensili ret_m relativi al periodo di training (definito secondo una suddivisione temporale 80–20).

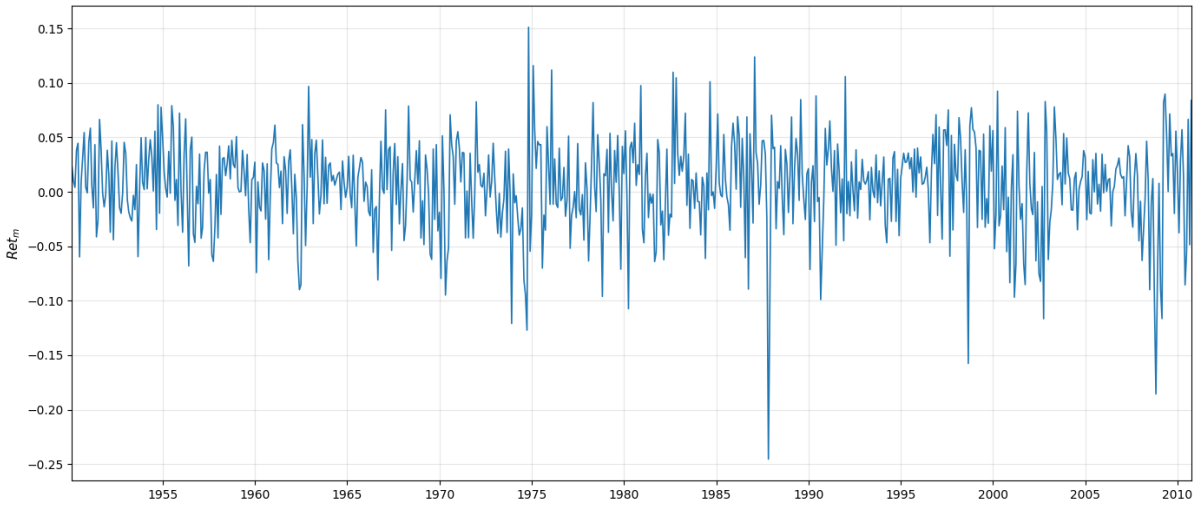


Figura 5.3: Rendimenti logaritmici mensili ret_m (training set).

Per analizzare la struttura dinamica dei rendimenti applico il *Teräsvirta Neural Network Test* (TNN Test), un test di linearità volto a verificare la presenza di componenti non lineari nella media condizionata della serie. Il test restituisce una statistica $\chi^2 = 3.5549$ con *p-value* pari a 0.1691, non consentendo di rifiutare l'ipotesi nulla di linearità ai livelli di significatività convenzionali. Questo risultato suggerisce che, per quanto riguarda la dinamica della media, una specificazione lineare risulta adeguata, motivando l'adozione di modelli ARMA di bassa dimensione per la componente di media.

Successivamente verifico l'ipotesi di normalità dei rendimenti mediante il *D'Agostino Normality Test*, che combina informazioni su skewness e kurtosis. I risultati indicano una marcata deviazione dalla normalità: la statistica omnibus risulta pari a 92.0434, con *p-value* inferiore a

$2.2 \cdot 10^{-16}$. In particolare, la skewness negativa (statistica $Z_3 = -6.7998$) evidenzia asimmetria a sinistra, mentre l'elevata kurtosis (statistica $Z_4 = 6.768$) segnala la presenza di code pesanti.

Nel complesso, mentre la dinamica della media appare compatibile con una struttura lineare, la distribuzione dei rendimenti presenta caratteristiche tipiche delle serie finanziarie, quali asimmetrie e leptocurtosi, che giustificano l'impiego di modelli GARCH per la volatilità e l'adozione di distribuzioni non gaussiane nella specificazione dell'errore.

Test	Statistica	p-value	Conclusione
Teräsvirta Neural Network Test	$\chi^2 = 3.5549$	0.1691	Linearità non rifiutata
D'Agostino Test (Omnibus)	92.0434	$< 2.2 \cdot 10^{-16}$	Non normalità
D'Agostino Test (Skewness)	$Z_3 = -6.7998$	$1.047 \cdot 10^{-11}$	Asimmetria significativa
D'Agostino Test (Kurtosis)	$Z_4 = 6.768$	$1.306 \cdot 10^{-11}$	Code pesanti

Tabella 5.3: Risultati dei test statistici si ret_m dell'S&P 500 (training set).

Per completare l'analisi preliminare sulla dinamica dei rendimenti esamino l'ACF e la PACF dei rendimenti logaritmici.

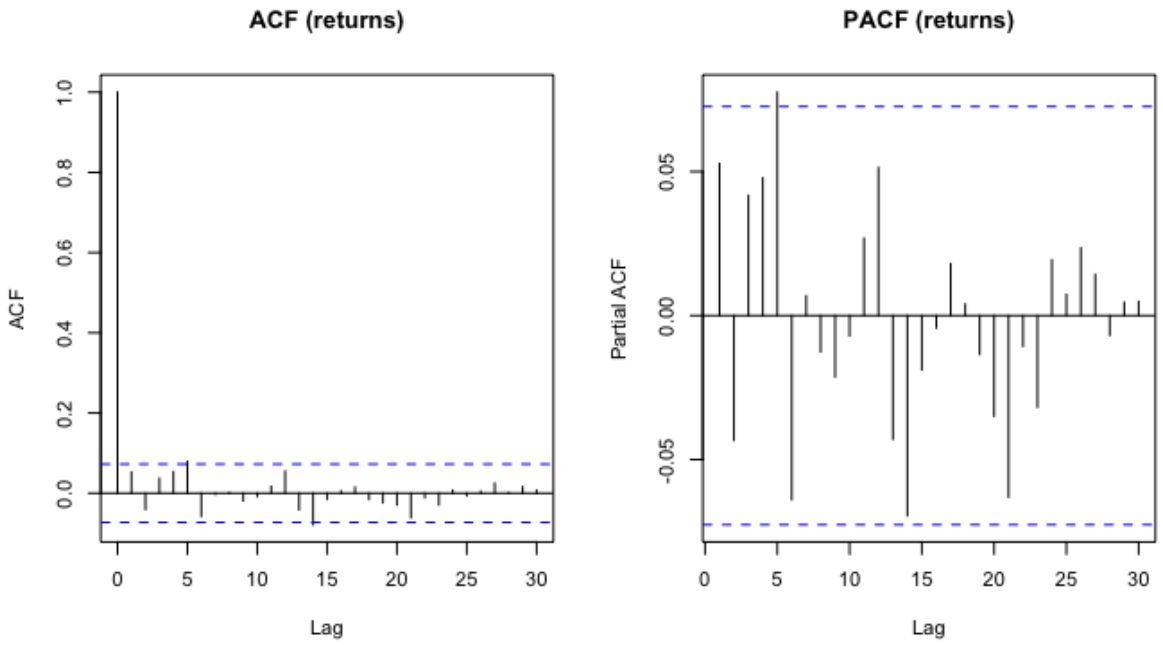


Figura 5.4: ACF e PACF della serie ret_m dell'S&P 500.

L'ACF mostra valori prossimi allo zero per tutti i ritardi considerati, con autocorrelazioni che rientrano quasi interamente all'interno delle bande di confidenza. Ciò indica l'assenza di dipendenza lineare significativa nei rendimenti e conferma che la serie può essere ragionevolmente approssimata come un processo a media zero privo di memoria lineare. Anche la PACF non evidenzia una struttura sistematica: eventuali picchi ai primi lag risultano isolati e di entità contenuta, senza suggerire una dinamica autoregressiva persistente nella media. Questa evidenza supporta ulteriormente l'adozione di specificazioni ARMA di bassa dimensione per la componente di media. Assenza di autocorrelazione lineare nella media, dinamica prevalentemente rumorosa, unite alla marcata non normalità documentata in precedenza, motivano lo

spostamento dell'attenzione verso la modellazione della varianza condizionata, affrontata nelle sezioni successive.

5.2 Stima e risultati dei modelli GARCH

5.2.1 Modello per la media

Prima di procedere alla stima dei modelli GARCH è necessario specificare la dinamica della media dei rendimenti. Una corretta modellazione della *mean equation* consente infatti di isolare in modo appropriato la componente di eteroschedasticità condizionata nella varianza, evitando che eventuali dipendenze di breve periodo nella media vengano assorbite impropriamente dalla dinamica della volatilità [7, 40].

La letteratura sul volatility forecasting evidenzia come la dinamica della media dei rendimenti finanziari svolga un ruolo secondario rispetto alla modellazione della varianza condizionata. Numerosi contributi mostrano che specificazioni semplici della mean equation, quali un processo autoregressivo di primo ordine oppure una media costante, risultano generalmente sufficienti per catturare eventuali dipendenze lineari di breve periodo, senza compromettere la qualità delle previsioni di volatilità [13, 2].

Alla luce di tali considerazioni, l'analisi prende avvio da una specificazione autoregressiva di primo ordine, AR(1), adottata come modello di riferimento iniziale per la dinamica della media. Successivamente, sulla base della significatività dei parametri stimati e di criteri di parsimonia, viene considerata anche una specificazione priva di dinamica autoregressiva, ARMA(0,0), corrispondente a una media costante.

Entrambe le specificazioni sono coerenti con la prassi consolidata nella letteratura e garantiscono un elevato grado di parsimonia. Questa scelta è in linea con l'obiettivo principale dell'analisi, che è la modellazione e la previsione della volatilità, piuttosto che della dinamica della media e consente di concentrare l'attenzione sulla specificazione della varianza condizionata.

5.2.2 Modello per la volatilità

Una volta specificata la dinamica della media, è necessario verificare che i residui del modello presentino le caratteristiche tipiche dei rendimenti finanziari, in particolare la presenza di dipendenza nella varianza condizionata. I modelli GARCH assumono infatti che l'eteroschedasticità sia contenuta nei residui della mean equation: qualora tale ipotesi trovi riscontro nei dati, l'adozione di un modello GARCH risulta appropriata.

Diagnostica dei residui: test BDS e test di non linearità

Procedo applicando il test *Brock–Dechert–Scheinkman* (BDS) ai residui del modello AR(1), per verificare l'ipotesi di indipendenza e identica distribuzione.

I risultati del BDS test, riportati in Tabella 5.4, indicano valori della statistica significativamente diversi da zero per diverse dimensioni di embedding e valori del parametro ε. I p-value risultano sistematicamente inferiori ai livelli di significatività convenzionali, portando al rigetto dell'ipotesi nulla di residui i.i.d. Tale evidenza segnala la presenza di dipendenza non lineare nei residui, non catturata dalla sola dinamica della media, ed è coerente con la presenza di eteroschedasticità condizionata.

Embedding		ε=0.0211	ε=0.0423	ε=0.0634	ε=0.0846
$m = 2$	Statistica	2.1012	2.3262	3.5732	4.7162
	p-value	0.0356	0.0200	0.0004	< 0.001
$m = 3$	Statistica	2.9062	3.0837	4.0982	5.1294
	p-value	0.0037	0.0020	< 0.001	< 0.001

Tabella 5.4: Risultati del BDS Test sui residui del modello per la media.

Per distinguere tra non linearità nella media e dipendenza nella varianza, applico inoltre il TNN Test ai residui. Il test restituisce una statistica $\chi^2 = 3.6189$ con p-value pari a 0.1637, non consentendo di rifiutare l'ipotesi nulla di linearità nella media condizionata dei residui. Questo risultato suggerisce che la componente di media risulta adeguatamente specificata e che la non linearità rilevata dal BDS test è riconducibile prevalentemente alla dinamica della varianza.

Statistica	Valore	df	p-value
χ^2	3.6189	2	0.1637

Tabella 5.5: Risultati del TNN Test sui residui del modello per la media.

A supporto di tale interpretazione, analizzo la struttura di autocorrelazione dei residui al quadrato $\hat{\varepsilon}_t^2$, che costituiscono un indicatore diretto della presenza di eteroschedasticità condizionata. La Figura 5.5 riporta le funzioni ACF e PACF di $\hat{\varepsilon}_t^2$, le quali mostrano autocorrelazioni significative e persistenti su numerosi ritardi. Questo comportamento è tipico delle serie finanziarie ed è indicativo di clustering della volatilità e dipendenza seriale nella varianza.

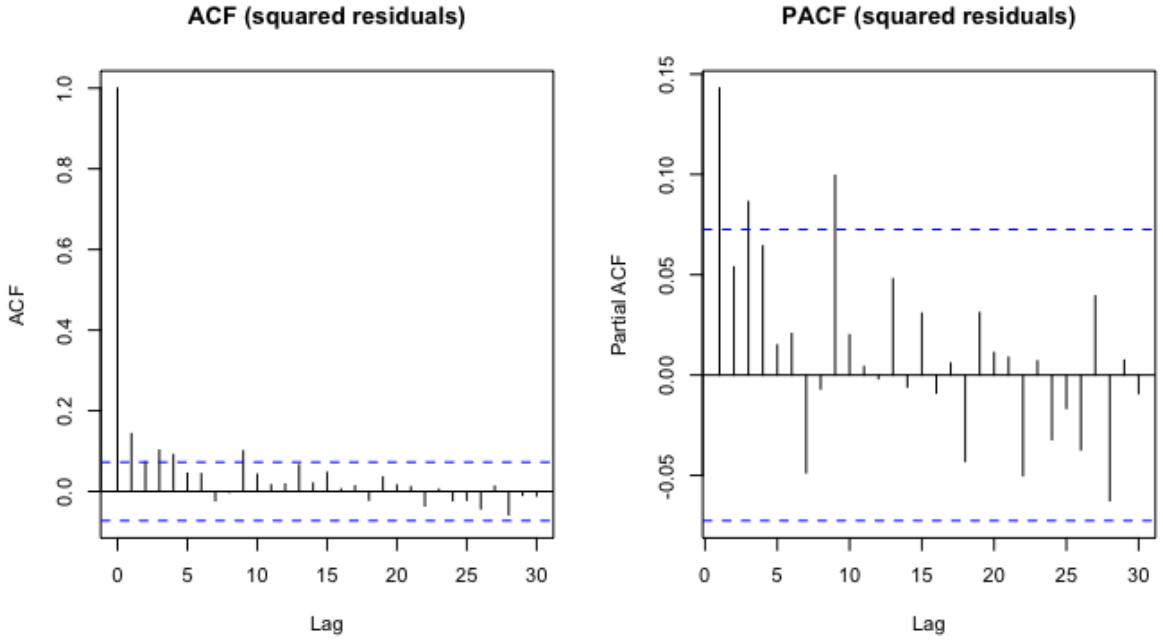


Figura 5.5: ACF e PACF dei residui al quadrato del modello AR(1).

Test di ARCH-LM

Per completare l'analisi diagnostica, verifico formalmente la presenza di effetti ARCH nei residui attraverso il test ARCH-LM, che valuta l'ipotesi nulla di assenza di autocorrelazione nella varianza condizionata.

Test	Statistica χ^2	df	p-value
ARCH-LM (lags = 10)	34.334	10	0.0001621

Tabella 5.6: Risultati del test ARCH-LM sui residui del modello per la media.

Il p-value porta al rigetto dell'ipotesi nulla del test. Questo risultato conferma in modo formale la presenza di eteroschedasticità condizionata nei residui.

Nel complesso, l'evidenza congiunta fornita da:

- il rigetto dell'ipotesi di residui i.i.d. nel BDS test,
- l'assenza di non linearità significativa nella media secondo il TNN test,
- la presenza di autocorrelazione persistente nei residui al quadrato,
- il rigetto dell'ipotesi nulla nel test ARCH-LM,

supporta in modo netto l'adozione di modelli GARCH per la modellazione della volatilità condizionata dei rendimenti.

5.2.3 Stima del modello GARCH(1,1)

Come punto di partenza viene stimato un modello GARCH(1,1) con innovazioni Normali, secondo la formulazione classica di Bollerslev [7]. Tale specificazione rappresenta una baseline standard nella letteratura, in quanto coniuga semplicità parametrica e capacità di riprodurre il clustering della volatilità osservato nei dati finanziari.

In una prima fase, la media condizionata dei rendimenti è modellata tramite un processo autoregressivo di primo ordine, AR(1), al fine di verificare se eventuali dipendenze lineari di breve periodo risultino rilevanti anche nella stima congiunta della varianza condizionata. La struttura del modello è quindi la seguente:

$$r_t = \mu + \phi_1 r_{t-1} + \varepsilon_t, \tag{5.3}$$

$$\sigma_t^2 = \omega + \alpha_1 \varepsilon_{t-1}^2 + \beta_1 \sigma_{t-1}^2, \tag{5.4}$$

dove $\varepsilon_t \mid \mathscr{F}_{t-1} \sim \mathscr{N}(0, \sigma_t^2)$. I risultati della stima sono riportati nella Tabella 5.7.

Parametro	Stima	Std. Error	t-value	p-value
μ	0.00625	0.00144	4.34	1.40×10^{-5}
ϕ_1	0.0276	0.0406	0.68	0.497
ω	8.37×10^{-5}	3.42×10^{-5}	2.44	0.014
α_1	0.1127	0.0258	4.36	1.29×10^{-5}
β_1	0.8482	0.0287	29.58	$< 10^{-16}$

Tabella 5.7: Stima del modello AR(1)–GARCH(1,1) con innovazioni Normali.

L'analisi dei coefficienti evidenzia che il termine autoregressivo ϕ_1 non risulta statisticamente significativo ($p = 0.497$), suggerendo che, una volta modellata la varianza condizionata, non permangano dipendenze lineari rilevanti nella media dei rendimenti. Questo risultato è coerente con quanto frequentemente riscontrato nella letteratura empirica sui modelli GARCH, dove la dinamica della media assume un ruolo secondario rispetto alla modellazione della volatilità. Alla luce della non significatività del coefficiente AR(1) e seguendo il principio di parsimonia, il modello viene pertanto ristimato adottando una specificazione della media priva di dinamica autoregressiva, corrispondente a una media costante, ARMA(0,0). La stima aggiornata è riportata nella Tabella 5.8.

Parametro	Stima	Std. Error	t-value	p-value
μ	0.00627	0.00140	4.47	7.85×10^{-6}
ω	8.52×10^{-5}	3.47×10^{-5}	2.45	0.014
α_1	0.1139	0.0260	4.38	1.16×10^{-5}
β_1	0.8464	0.0289	29.32	$< 10^{-16}$

Tabella 5.8: Stima del modello ARMA(0,0)–GARCH(1,1) con innovazioni Normali

Tutti i parametri della varianza condizionata risultano statisticamente significativi ($p < 0.05$) e soddisfano le condizioni teoriche del modello GARCH(1,1), ossia $\omega > 0$, $\alpha_1 \geq 0$ e $\beta_1 \geq 0$. La somma $\alpha_1 + \beta_1 = 0.9603$ indica un'elevata persistenza della volatilità, caratteristica tipica dei rendimenti finanziari, per cui gli shock alla varianza tendono a dissiparsi lentamente nel tempo.

Test	Statistica	df	p-value
Ljung–Box (residui std.)	11.075	10	0.352
Ljung–Box (residui std.^{2})	5.813	10	0.831
ARCH–LM	5.618	10	0.846

Tabella 5.9: Test diagnostici sul modello ARMA(0,0)–GARCH(1,1).

I test diagnostici condotti sul modello ARMA(0,0)–GARCH(1,1) con innovazioni Normali non evidenziano violazioni significative delle ipotesi del modello. In particolare, il test di Ljung–Box applicato ai residui standardizzati e ai residui standardizzati al quadrato non porta al rigetto dell'ipotesi nulla di assenza di autocorrelazione residua, né nella media né nella varianza.

Analogamente, il test ARCH–LM non rileva la presenza di effetti ARCH residui, indicando che la struttura GARCH(1,1) è sufficiente a catturare la dipendenza dinamica nella volatilità.

A supporto dell'analisi diagnostica, la Figura 5.6 riporta il QQ–plot dei residui standardizzati del modello considerato. Il grafico evidenzia una buona aderenza alla distribuzione teorica nella parte centrale, ma deviazioni sistematiche nelle code, in particolare nella coda sinistra, indicando la presenza di code pesanti rispetto all'ipotesi Gaussiana.

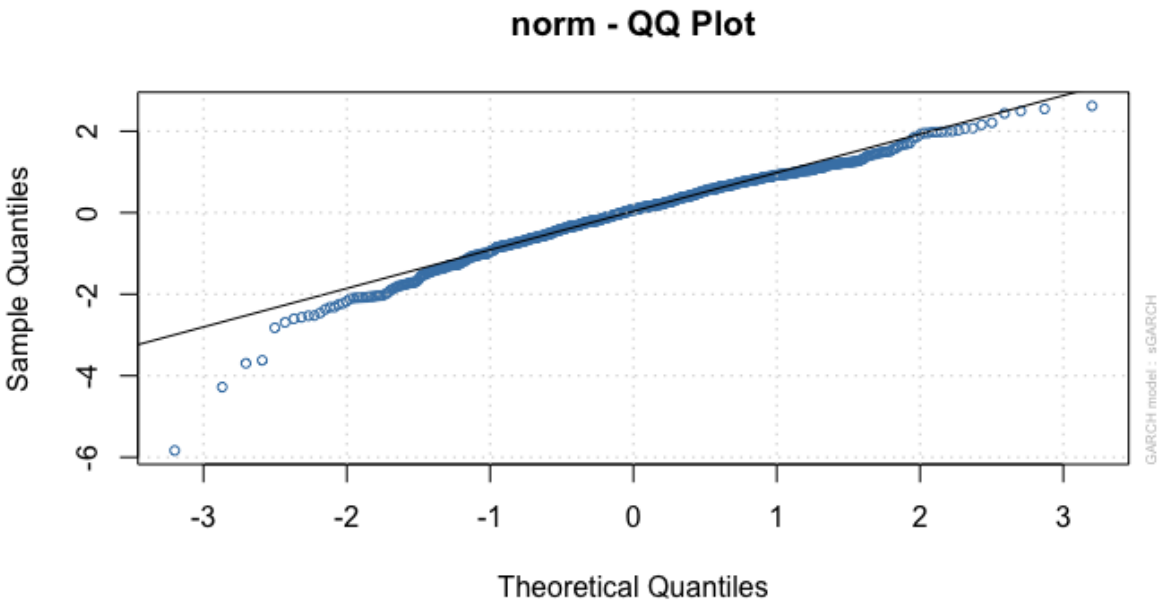


Figura 5.6: QQ–plot dei residui standardizzati del modello ARMA(0,0)–GARCH(1,1) con innovazioni Normali.

Nel complesso, tali evidenze suggeriscono che il modello ARMA(0,0)–GARCH(1,1) fornisce una descrizione adeguata della volatilità condizionata dei rendimenti e rappresenta una base solida per il confronto con distribuzioni alternative delle innovazioni e con successive estensioni asimmetriche del modello.

5.2.4 Distribuzioni alternative

Alla luce delle deviazioni dalla normalità evidenziate dall'analisi dei rendimenti e dal QQ–plot dei residui standardizzati del modello baseline, si procede a stimare il modello sGARCH(1,1) utilizzando distribuzioni delle innovazioni più flessibili, in grado di catturare code pesanti ed eventuali asimmetrie. In particolare, vengono considerate quattro distribuzioni alternative alla Normale:

- *Student-t*, che introduce una curtosi maggiore attraverso un parametro di forma;
- *Generalized Error Distribution* (GED), che consente una modulazione continua della pesantezza delle code;
- *Skewed-t*, che permette di modellare simultaneamente code pesanti e asimmetria;
- *Skewed-GED*, la più flessibile tra le distribuzioni considerate, in grado di catturare sia elevata curtosi sia skewness.

Per tutte le specificazioni viene mantenuta invariata la struttura del modello, al fine di garantire la piena comparabilità dei risultati. La Tabella 5.10 riporta le stime dei parametri, gli errori standard e le principali statistiche di sintesi.

	Normal	Student-t	GED	Skewed-t	Skewed-GED
Parametri della media e della varianza					
μ	0.00627^{***}	0.00781^{***}	0.00781^{***}	0.00628^{***}	0.00613^{***}
	(0.00140)	(0.00138)	(0.00142)	(0.00139)	(0.00136)
ω	$8.52\times10^{-5**}$	$1.10\times10^{-4**}$	$1.00\times10^{-4**}$	$9.00\times10^{-5**}$	$8.00\times10^{-5**}$
	(3.47×10^{-5})	(5.00×10^{-5})	(4.00×10^{-5})	(4.00×10^{-5})	(3.00×10^{-5})
α_1	0.11395^{***}	0.11603^{***}	0.11483^{***}	0.11710^{***}	0.11812^{***}
	(0.02599)	(0.03136)	(0.03024)	(0.02854)	(0.02773)
β_1	0.84637^{***}	0.82571^{***}	0.83714^{***}	0.83909^{***}	0.84493^{***}
	(0.02887)	(0.04195)	(0.03676)	(0.03399)	(0.02996)
shape	–	8.10330^{***}	1.48793^{***}	9.45539^{***}	1.59107^{***}
	–	(2.13548)	(0.10374)	(2.90837)	(0.11076)
skew	–	–	–	0.76954^{***}	0.76761^{***}
	–	–	–	(0.04562)	(0.04088)
Statistiche di sintesi del modello					
Log-likelihood	1297.68	1311.53	1307.25	1321.58	1318.96
Persistenza $\alpha_1+\beta_1$	0.9603	0.9417	0.9520	0.9562	0.9631
AIC	-3.5492	-3.5844	-3.5727	-3.6093	-3.6021
BIC	-3.5240	-3.5529	-3.5412	-3.5715	-3.5643

Nota: errori standard tra parentesi. $^{*}p<0.10$, $^{**}p<0.05$, $^{***}p<0.01$.

Tabella 5.10: Confronto tra modelli sGARCH con diverse distribuzioni delle innovazioni.

L'analisi comparativa evidenzia che l'impiego di distribuzioni non gaussiane porta a un netto miglioramento della log-verosimiglianza e dei criteri informativi (AIC e BIC) rispetto al modello con innovazioni Normali. In particolare, le distribuzioni Student-t e GED mostrano una migliore capacità di catturare la leptocurtosi dei rendimenti, mentre le loro versioni asimmetriche (Skewed-t e Skewed-GED) risultano ulteriormente più flessibili nel modellare anche la skewness.

Tra le specificazioni considerate, il modello con distribuzione Skewed-t presenta i valori più bassi di AIC e BIC, indicando il miglior compromesso tra qualità dell'adattamento e parsimonia. I parametri di forma e di asimmetria risultano statisticamente significativi, confermando empiricamente la presenza di code pesanti e asimmetria nei rendimenti finanziari, in linea con l'evidenza ampiamente documentata in letteratura [6, 17].

Tuttavia, un miglioramento delle prestazioni IS non implica necessariamente una superiore capacità predittiva OOS. Come sottolineato da diversi contributi, la massimizzazione della verosimiglianza può portare a fenomeni di overfitting, con una conseguente perdita di efficienza previsiva [18, 30].

Per questo motivo, nella fase successiva dell'analisi non ci si limita alla selezione del modello migliore in base ai criteri informativi, ma si procede a confrontare un'ampia classe di modelli GARCH, inclusi modelli asimmetrici, valutandone la capacità predittiva OOS.

5.2.5 Modelli GARCH asimmetrici

L'analisi viene ora estesa a modelli GARCH asimmetrici, in grado di catturare risposte differenziate della volatilità a shock positivi e negativi (leverage effect). In particolare, vengono considerate le specificazioni gjrGARCH, eGARCH e apARCH oltre al modello sGARCH come benchmark simmetrico. Ciascuna specificazione è stimata sotto diverse distribuzioni delle innovazioni. Il confronto tra modelli è condotto esclusivamente in termini di capacità predittiva

OOS, adottando come criterio di selezione principale l'RMSE calcolato rispetto alla volatilità realizzata. Questa scelta è coerente con l'obiettivo centrale del lavoro, che è l'identificazione della specificazione più accurata nel periodo di test, piuttosto che il semplice adattamento ai dati IS.

Modello	Dist.	RMSE_oos
gjrGARCH	std	0.082897
eGARCH	std	0.083327
eGARCH	ged	0.083762
apARCH	std	0.083954
eGARCH	sstd	0.083998
gjrGARCH	ged	0.084050
apARCH	ged	0.084199
apARCH	norm	0.084285
gjrGARCH	sstd	0.084293
apARCH	sstd	0.084321
eGARCH	norm	0.084400
eGARCH	sged	0.084446
apARCH	sged	0.084708
gjrGARCH	sged	0.085177
gjrGARCH	norm	0.085599
sGARCH	std	0.086984
sGARCH	ged	0.087251
sGARCH	sstd	0.087558
sGARCH	norm	0.087724
sGARCH	sged	0.087767

Tabella 5.11: Confronto tra modelli GARCH asimmetrici in base all'RMSE_oos.

I risultati (Tabella 5.11) mostrano che il modello con la migliore performance previsiva è il gjrGARCH(1,1) con innovazioni Student-t, che presenta il valore minimo di RMSE fuori dal campione. Seguono a breve distanza diverse specificazioni eGARCH e apARCH, suggerendo che l'introduzione di asimmetrie nella dinamica della volatilità contribuisce a migliorare la capacità predittiva rispetto al modello sGARCH simmetrico.

Nel complesso, i modelli sGARCH risultano sistematicamente meno performanti in termini di errore OOS, confermando che l'ipotesi di risposta simmetrica della volatilità agli shock è eccessivamente restrittiva nel contesto analizzato.

Alla luce di tali evidenze, il modello gjrGARCH(1,1)-Student-t viene selezionato come specificazione finale di riferimento. Nel paragrafo successivo ne viene analizzata in dettaglio la diagnostica, al fine di verificarne l'adeguatezza statistica.

Test diagnostici sul modello finale

L'obiettivo è verificare che i residui standardizzati del modello selezionato non presentino autocorrelazione residua né effetti ARCH non modellati e che la struttura asimmetrica del modello catturi adeguatamente la risposta differenziata della volatilità a shock positivi e negativi.

Parametro	Stima	Errore Std.	t-value	p-value
μ	0.00721	0.00140	5.134	2.83×10^{-7}
ω	0.00021	0.00011	1.949	0.0513
α_1	0.00000	0.04661	0.000	0.9999
β_1	0.78007	0.06713	11.620	$< 10^{-16}$
γ_1	0.18545	0.07803	2.377	0.0175
shape	8.43477	2.18096	3.867	1.10×10^{-4}

Tabella 5.12: Stima dei parametri del modello gjrGARCH(1,1) con innovazioni Student-t.

La stima dei parametri del modello evidenzia:

- una volatilità altamente persistente, con $\beta_1 \approx 0.78$;
- un parametro asimmetrico γ_1 positivo e statisticamente significativo ($p < 0.05$), che conferma la presenza di leverage effect;
- un parametro di forma della distribuzione Student-t significativo, indicativo di code pesanti nei residui;
- una dinamica della media adeguatamente catturata dal solo termine costante.

Test	Statistica	df	p-value
Ljung–Box Test			
Residui standardizzati	9.9382	10	0.4459
Residui standardizzati^2	4.3870	10	0.9282
ARCH–LM Test			
ARCH LM (lag 10)	4.2287	10	0.9364

Tabella 5.13: Test diagnostici sui residui del modello gjrGARCH(1,1)–Student-t.

Dalla Tabella 5.13 emerge che il test di Ljung–Box non rileva autocorrelazione residua né nei residui standardizzati né nei residui al quadrato, mentre il test ARCH–LM non evidenzia effetti ARCH residui di ordine superiore (p-value elevati per tutti i lag considerati).

Nel complesso, l'evidenza diagnostica conferma che il modello gjrGARCH(1,1) con innovazioni Student-t è correttamente specificato, cattura in modo efficace l'eteroschedasticità condizionata e l'asimmetria della volatilità, e rappresenta una specificazione robusta per l'analisi e la previsione della volatilità nel periodo OOS.

5.2.6 Analisi predittiva

Una volta selezionato il modello econometrico finale, procedo alla valutazione della sua capacità predittiva sulla volatilità. In questa fase è necessario disporre di una misura osservata con cui confrontare le previsioni del modello. Coerentemente con quanto discusso nel Capitolo 1, si adotta la variabile target RV_m.

Il modello viene stimato sulla serie completa dei rendimenti mensili, specificando l'argomento `out.sample = N` in `ugarchfit` per riservare le ultime N osservazioni alla fase di

test. Successivamente, la funzione `ugarchforecast` viene utilizzata per produrre previsioni ricorsive a un passo in avanti sull'intero periodo OOS.

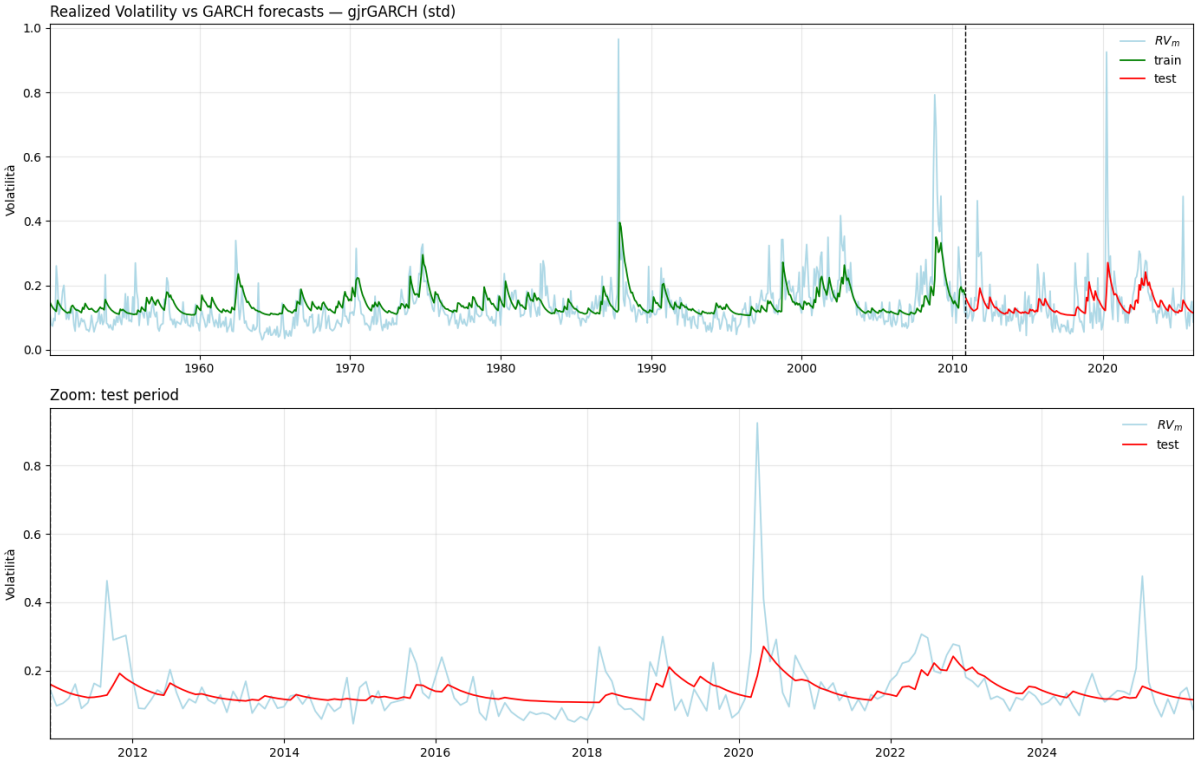


Figura 5.7: Confronto tra RV_m e previsioni del modello gjrGARCH(1,1)–Student-t.

Il confronto grafico mostra che il modello riproduce in modo coerente la dinamica di fondo della volatilità, catturando il clustering e i principali cambiamenti di regime. Come tipicamente osservato nei modelli GARCH, la volatilità condizionata stimata tende tuttavia a risultare più smussata rispetto alla volatilità realizzata: gli spike più estremi di RV_m vengono attenuati, mentre nei periodi di calma la previsione rimane più stabile. La performance predittiva viene valutata tramite RMSE e MAE, calcolate confrontando RV_m con la volatilità stimata OOS dal modello.

	RMSE_is	MAE_is	RMSE_oos	MAE_oos
gjrGARCH(1,1) – Student-t	0.06639	0.04380	0.08290	0.04830

Tabella 5.14: Metriche IS e OOS del gjrGARCH(1,1)–Student-t.

Come atteso, gli errori risultano inferiori nel periodo di training rispetto al periodo di test; l'incremento dell'errore OOS rimane tuttavia contenuto, suggerendo una buona capacità di generalizzazione del modello selezionato, che fornisce quindi una base econometrica solida per il successivo confronto con le architetture neurali.

5.3 Risultati dei modelli neurali

In questa sezione presento le specificazioni adottate per i modelli neurali, sintetizzando per ciascuno di essi la struttura dell'architettura e la procedura di addestramento.

MLP: rete feedforward basata su finestre temporali fisse

La MLP rappresenta la rete neurale feedforward più semplice: è priva di memoria interna e l'informazione fluisce in modo unidirezionale attraverso i layer; in altre parole, la MLP non è progettata per elaborare sequenze. Di conseguenza, affinché il modello possa sfruttare la dipendenza temporale tipica delle serie finanziarie, è necessario trasformare il dataset in un problema di regressione supervisionata mediante la costruzione di finestre temporali fisse (*sliding windows*).

Costruzione del dataset supervisionato

Le informazioni dinamiche vengono incorporate esplicitamente nel vettore di input: per ogni istante t, si costruisce una finestra contenente i valori passati delle due variabili RV_m e ret_m. Indicando con L il numero di ritardi utilizzati, la struttura dell'input risulta:

$$X_t = \left[\mathrm{RV}_{t-L}, \mathrm{ret}_{t-L}, \ldots, \mathrm{RV}_{t-1}, \mathrm{ret}_{t-1}\right], \qquad y_t = \mathrm{RV}_t, \tag{5.5}$$

La dimensione del vettore di input è quindi pari a $2L$ e la finestra scorre lungo la serie temporale eliminando a ogni passo il valore più vecchio e includendo quello più recente. Questo meccanismo consente alla MLP di catturare dipendenze di breve periodo pur in assenza di una memoria ricorrente interna. La Tabella 5.15 riporta la forma originale del dataset, mentre la 5.16 mostra la struttura risultante per un numero limitato di ritardi.

Date	RV_m	ret_m
1950–01–31	0.109106	0.023140
1950–02–28	0.078142	0.009921
1950–03–31	0.073541	0.004057
⋮	⋮	⋮
2025–10–31	0.134849	0.022433
2025–11–30	0.150288	0.001299
2025–12–31	0.086307	-0.000524

Tabella 5.15: Struttura del dataset di partenza (S&P 500).

	Lag $t-2$		Lag $t-1$		Target t
Tempo	RV_{t-2}	ret_{t-2}	RV_{t-1}	ret_{t-1}	RV_t
$t=3$	0.109106	0.023140	0.078142	0.009921	0.073541
$t=4$	0.078142	0.009921	0.073541	0.004057	0.097691
$t=5$	0.073541	0.004057	0.097691	0.038019	0.090642
$t=6$	0.097691	0.038019	0.090642	0.044645	0.260658
$t=7$	0.090642	0.044645	0.260658	-0.059793	0.202575

Tabella 5.16: Dataset supervisionato 2D con $L=2$.

Si osservi inoltre che, a seconda della posizione della finestra nella serie temporale, l'osservazione target associata ($y_t = \mathrm{RV}_t$) può appartenere tanto al periodo IS quanto al periodo OOS:

il modello utilizza sempre i $2L$ ritardi passati per costruire l'input, ma la previsione risultante è classificata come train o test in funzione dell'istante t a cui la finestra è riferita. Questo comportamento è illustrato nella Figura 5.8, riportata di seguito.

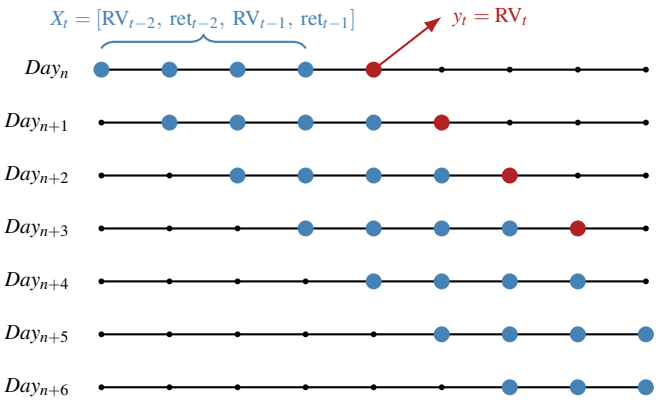


Figura 5.8: Schema della finestra temporale utilizzata per costruire gli input della MLP.

Architettura e procedura di addestramento

La rete MLP è implementata tramite l'API *Sequential* di Keras (TensorFlow), adottando una struttura volutamente semplice e parsimoniosa. L'architettura prevede un layer di input di dimensione $2L$, corrispondente alla finestra temporale dei ritardi di RV_m e ret_m, un singolo layer nascosto completamente connesso e un nodo di output lineare dedicato alla previsione della volatilità realizzata al tempo t.

Il layer nascosto utilizza la funzione di attivazione ReLU, scelta per la sua semplicità computazionale e per la buona stabilità numerica nei modelli feedforward poco profondi. Il nodo di output adotta invece un'attivazione lineare, coerente con la natura continua della variabile target.

L'addestramento avviene minimizzando la funzione di perdita Mean Squared Error mediante l'ottimizzatore *Adam*. Il modello viene allenato per un numero massimo di 250 epoche nella fase di selezione degli iperparametri e fino a 1000 epoche nella stima finale OOS. In ogni epoca, il training set è suddiviso in mini-batch e i pesi della rete vengono aggiornati in modo sequenziale, mantenendo l'ordine temporale dei dati (`shuffle = False`) per evitare qualunque forma di leakage informativo.

Durante la fase di tuning degli iperparametri, l'andamento dell'apprendimento è monitorato tramite early stopping basato sulla *validation loss*, con una `patience` pari a 5 epoche. Ciò significa che, qualora la validation loss non migliori per 5 epoche consecutive, il processo di training viene interrotto e vengono automaticamente ripristinati i pesi corrispondenti al minimo della curva di validazione (`restore_best_weights = True`).

Sia le variabili in input sia la variabile target vengono scalate tramite una trasformazione `MinMaxScaler` nel range $[-1,1]$, stimata esclusivamente sul campione di training. Questa scelta garantisce stabilità numerica durante l'ottimizzazione e previene qualsiasi forma di anticipazione informativa.

Le previsioni finali vengono successivamente riportate nella scala originale della volatilità realizzata mediante l'inversa della trasformazione di scaling. L'intero processo di addestramento è reso pienamente riproducibile fissando i *seed* delle librerie numeriche e del framework di deep learning.

Ricerca degli iperparametri e selezione modello

Definita l'architettura di base e la procedura di addestramento, la configurazione ottimale della MLP viene individuata tramite una ricerca su una griglia ristretta di iperparametri. In particolare, l'attenzione è posta su tre elementi chiave: la dimensione della finestra temporale L (lags), il numero di neuroni del layer nascosto e la batch size, ossia la dimensione dei sottoinsiemi del training set utilizzati per l'aggiornamento dei pesi a ogni iterazione dell'ottimizzatore. Lo spazio degli iperparametri esplorato è definito come segue:

$$L \in \{1,2,3,4\}, \qquad \text{hidden} \in \{2,4,6,8\}, \qquad \text{batch size} \in \{32,64,128,n\},$$

dove n indica il caso limite di batch gradient descent, in cui l'intero training set viene utilizzato per ciascun aggiornamento dei pesi.

La selezione degli iperparametri avviene in due fasi. In una prima fase viene condotta una grid search con batch size fissata a 32, valutando tutte le combinazioni di L e del numero di neuroni nel layer nascosto. In una seconda fase, fissata la coppia (L, hidden) ottimale, si procede alla selezione della batch size confrontando diverse strategie di aggiornamento dei pesi. Il criterio di selezione è l'RMSE OOS calcolato sul validation set, coerentemente con l'obiettivo di massimizzare la capacità predittiva e non il semplice adattamento IS. La combinazione ottimale individuata risulta:

$$L = 1, \qquad \text{hidden} = 8, \qquad \text{batch size} = 32.$$

L'architettura finale è quindi una MLP con un solo ritardo temporale e un layer nascosto composto da 8 neuroni. Poiché ciascun input include le due variabili RV_{t-1} e ret_{t-1}, la dimensione del vettore di input è pari a $2L = 2$. Il numero complessivo di parametri addestrabili risulta:

$$N_p = (2+1) \times 8 + (8+1) \times 1 = 33,$$

confermando la natura estremamente parsimoniosa del modello. Il numero totale di configurazioni esplorate nella prima fase della grid search è pari a $4 \times 4 = 16$, a cui si aggiungono ulteriori valutazioni sulla batch size. Data la ridotta dimensionalità dello spazio degli iperparametri e la semplicità dell'architettura, tale ricerca risulta sufficiente e non richiede l'impiego di metodologie più complesse quali *random search* o ottimizzazione bayesiana, tipicamente riservate a reti profonde o a spazi di ricerca molto ampi.

L'andamento della funzione obiettivo durante l'addestramento della configurazione ottimale è riportato in Figura 5.9, che mostra l'evoluzione della training loss e della validation loss. L'early stopping interviene dopo un numero limitato di epoche, segnalando una convergenza rapida e stabile e l'assenza di fenomeni evidenti di overfitting.

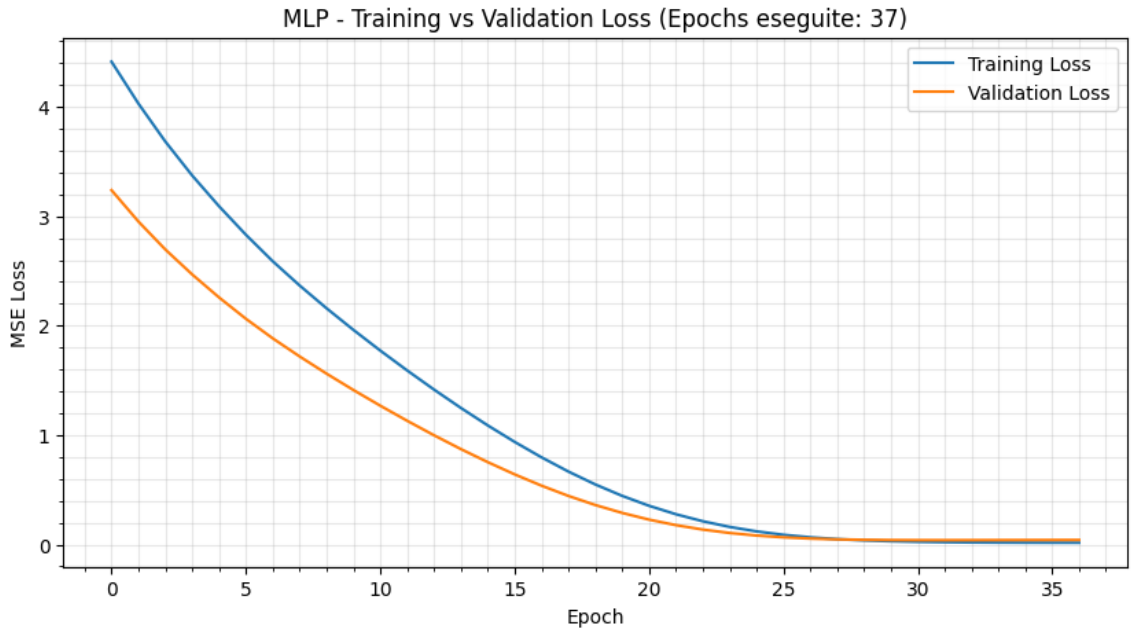


Figura 5.9: Andamento di training loss e validation loss durante l'addestramento della MLP.

Nel complesso, la procedura adottata consente di individuare una configurazione semplice, stabile e facilmente interpretabile.

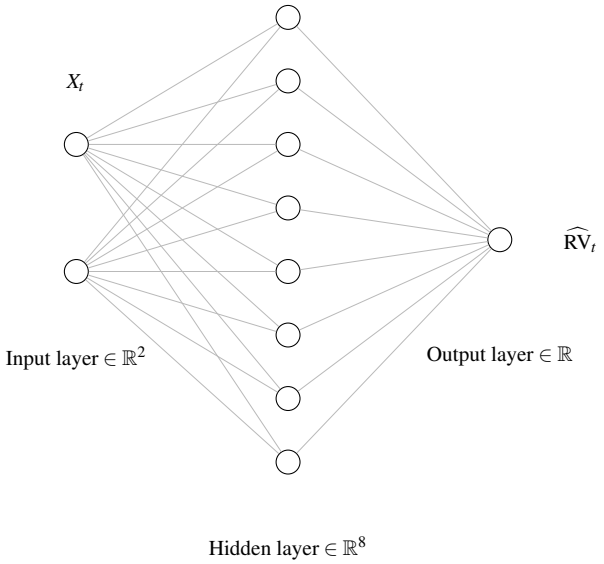


Figura 5.10: Architettura della MLP ottimale selezionata.

LSTM: rete ricorrente con memoria interna

A differenza della MLP, la LSTM appartiene alla famiglia delle reti neurali ricorrenti ed è progettata per modellare direttamente la dipendenza temporale nei dati. La presenza di uno stato interno e dei meccanismi di gating (input, forget e output gate) consente alla rete di conservare o dimenticare selettivamente l'informazione, mitigando il problema del vanishing gradient tipico delle RNN standard. Nel contesto della volatilità finanziaria, questa proprietà è particolarmente rilevante poiché la serie RV_m mostra persistenza e clustering. La differenza principale

rispetto alla MLP risiede quindi nella forma dell'input: la MLP riceve un vettore 2D, mentre la LSTM elabora una sequenza 3D (samples, timesteps, features).

Costruzione del dataset in formato 3D

Indicando con L il numero di ritardi (timesteps) e con d il numero di variabili (features), si consideri per semplicità espositiva il caso $d = 2$ e $L = 2$. Ogni input X_t è quindi una matrice $X_t \in \mathbb{R}^{L\times d} = \mathbb{R}^{2\times 2}$ e l'intero dataset supervisionato assume la forma $X \in \mathbb{R}^{n_{\text{samples}}\times 2\times 2}$. Il target associato alla sequenza $\{t-2, t-1\}$ è la volatilità realizzata al tempo t: $y_t = \text{RV}_t$. La Tabella 5.17 mostra la conversione nella rappresentazione tridimensionale utilizzata come input della rete.

Sample	Input LSTM $(L \times d)$	Target
$t = 3$	$\begin{bmatrix} 0.109106 & 0.023140 \\ 0.078142 & 0.009921 \end{bmatrix}$	0.073541
$t = 4$	$\begin{bmatrix} 0.078142 & 0.009921 \\ 0.073541 & 0.004057 \end{bmatrix}$	0.097691

Tabella 5.17: Dataset supervisionato 3D con $L = 2$.

Architettura e procedura di addestramento

L'architettura adottata prevede un unico layer LSTM con u unità, dotato di funzione di attivazione tanh, seguito da un layer denso finale con output lineare, utilizzato per la stima di $\widehat{\text{RV}}_t$.

Al fine di garantire una comparazione *fair*, la procedura di addestramento è mantenuta coerente con quella adottata per la MLP (e, analogamente, per i modelli successivi). In particolare, l'addestramento avviene mediante ottimizzatore Adam e funzione obiettivo MSE, con apprendimento eseguito senza rimescolamento delle osservazioni (`shuffle = False`). Le variabili di input e il target sono scalati nell'intervallo $[-1, 1]$ tramite `MinMaxScaler`; le previsioni finali vengono quindi riportate nella scala originale mediante trasformazione inversa. Il processo di apprendimento è monitorato attraverso una procedura di *early stopping* basata sulla *validation loss*, con `patience` pari a 5 epoche e ripristino automatico dei pesi corrispondenti al minimo della curva di validazione (`restore_best_weights = True`).

Ricerca degli iperparametri e selezione modello

La configurazione ottimale viene individuata tramite una ricerca a griglia sugli iperparametri principali:

$$L \in \{1,2,3,4\}, \qquad u \in \{2,4,6,8\},$$

selezionando la combinazione con minima RMSE OOS. Fissati L e u, la batch size viene scelta confrontando un insieme ristretto di valori candidati. Nel complesso, la selezione conduce a:

$$L = 4, \qquad u = 2, \qquad \text{batch size} = 64.$$

Con tali valori, il numero di parametri addestrabili della LSTM è:

$$N_{\text{LSTM}} = 4u(u + d + 1) = 4 \cdot 2\,(2 + 2 + 1) = 40,$$

a cui si aggiunge il layer denso finale:

$$N_{\text{Dense}} = u \cdot 1 + 1 = 3, \qquad N_p = 40 + 3 = 43.$$

La Figura 5.11 mostra le curve di training e validation loss del run diagnostico. Infine, il modello viene riaddestrato sul campione di stima complessivo (train+validation) e valutato sul periodo di test OOS, producendo le previsioni $\widehat{\text{RV}}_t$.

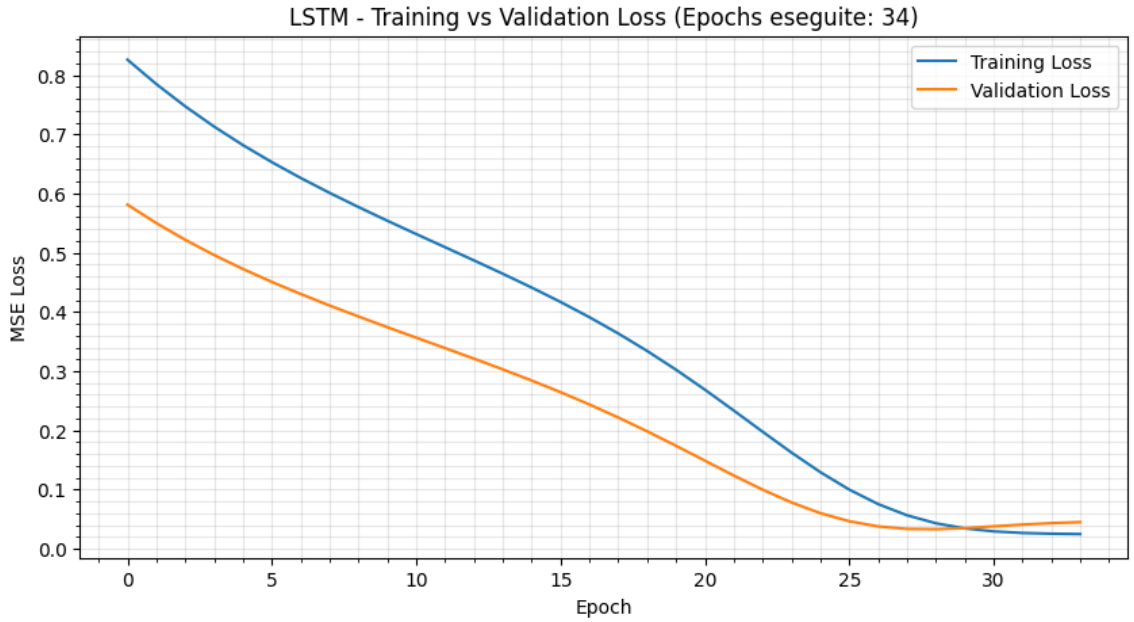


Figura 5.11: Andamento di training loss e validation loss durante l'addestramento della LSTM.

La Figura 5.12 riporta una rappresentazione schematica dell'architettura LSTM selezionata.

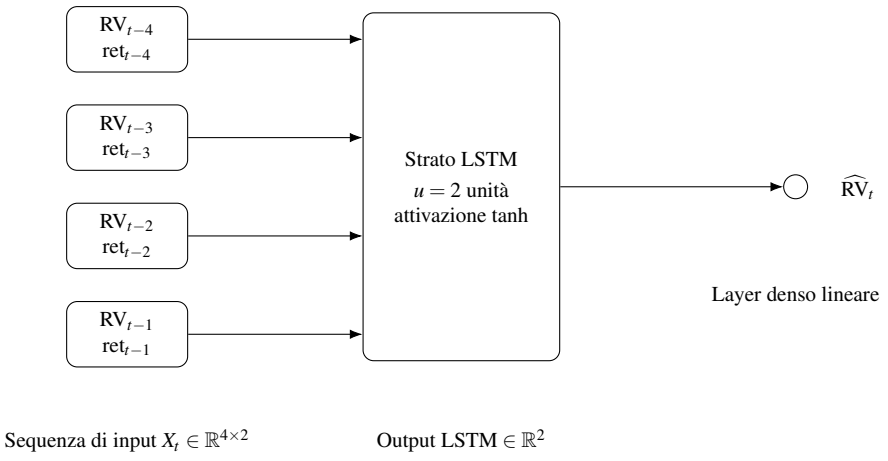


Figura 5.12: Architettura della LSTM ottimale selezionata.

GRU: struttura compatta con gated units

La GRU è un'architettura ricorrente con meccanismi di gating, progettata come alternativa più compatta alla LSTM. Rispetto a quest'ultima, la GRU utilizza due sole porte (reset e update) e un'unica rappresentazione dello stato interno, eliminando la separazione esplicita tra memoria

e stato nascosto. Questa semplificazione riduce il numero di parametri addestrabili, pur mantenendo la capacità di modellare dipendenze temporali persistenti e di mitigare il problema del vanishing gradient.

La costruzione del dataset supervisionato coincide con quella adottata per la LSTM (cfr. Sottosezione 5.3).

Architettura e procedura di addestramento

L'architettura GRU adottata è volutamente semplice e direttamente confrontabile con quella della LSTM. La rete è composta da un unico strato GRU con u unità ricorrenti, seguito da un layer denso finale con output lineare, impiegato per la previsione della volatilità $\widehat{\mathrm{RV}}_t$.

Al fine di garantire la piena confrontabilità tra i modelli, la procedura di addestramento è mantenuta identica a quella adottata per la LSTM (cfr. Sottosezione 5.3), sia in termini di ottimizzatore, funzione obiettivo, strategia di scaling dei dati, sia per quanto riguarda il monitoraggio dell'apprendimento tramite early stopping.

Ricerca degli iperparametri e selezione modello

La configurazione ottimale della GRU è individuata mediante una ricerca a griglia sugli iperparametri principali:

$$L \in \{1,2,3,4\}, \qquad u \in \{2,4,6,8\},$$

a cui segue la selezione della batch size confrontando un insieme ristretto di valori candidati. La combinazione ottimale, individuata sulla base dell'RMSE OOS è la seguente:

$$L = 4, \qquad u = 6, \qquad \text{batch size} = 32.$$

L'input della rete è costituito da sequenze $X_t \in \mathbb{R}^{4\times 2}$, mentre lo strato GRU produce un vettore in $\mathbb{R}^6$ che viene infine proiettato nello scalare $\widehat{\mathrm{RV}}_t$ tramite il layer denso lineare. Il numero di parametri addestrabili dello strato GRU è dato da:

$$N_{\mathrm{GRU}} = 3\,(d\,u + u^2) + 2 \cdot 3u = 3\,(2 \cdot 6 + 6^2) + 36 = 3\,(12 + 36) + 36 = 180.$$

A questi si aggiungono i parametri del layer denso lineare finale:

$$N_{\mathrm{Dense}} = u + 1 = 6 + 1 = 7.$$

Il numero totale di parametri addestrabili del modello GRU risulta pertanto:

$$N_p = 180 + 7 = 187.$$

In Figura 5.13 si riporta l'andamento della training e validation loss durante il run diagnostico.

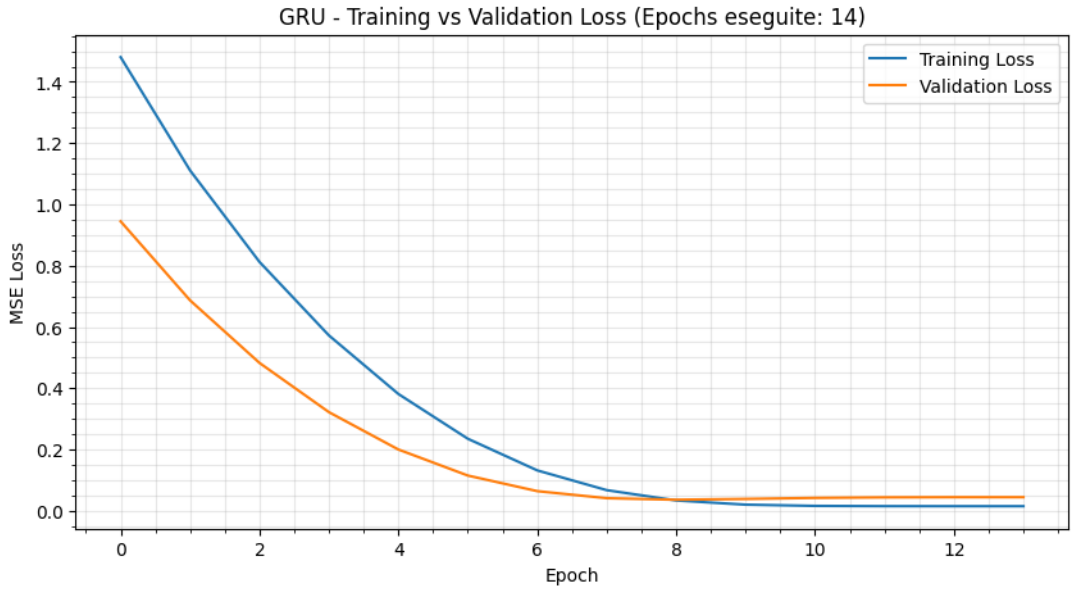


Figura 5.13: Andamento di training loss e validation loss per la GRU selezionata.

La Figura 5.14 riassume infine l'architettura della GRU ottimale.

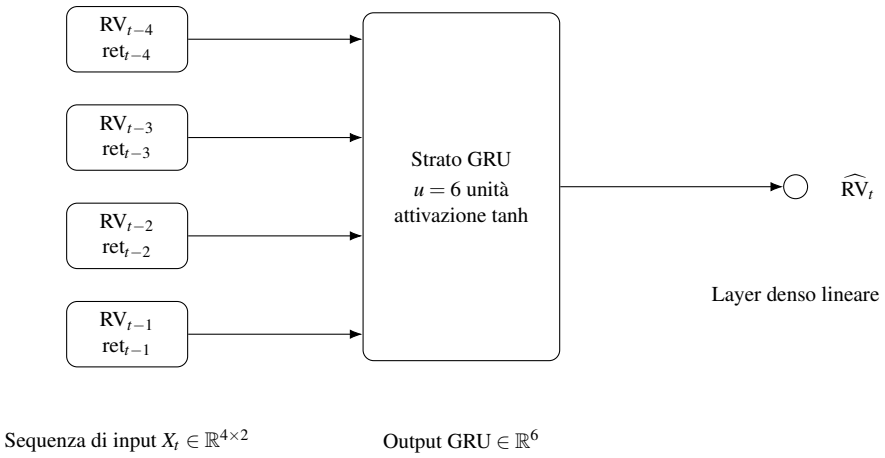


Figura 5.14: Architettura della GRU ottimale selezionata.

CNN-1D: rete convoluzionale monodimensionale

Le reti convoluzionali monodimensionali costituiscono un'alternativa compatta alle architetture ricorrenti per la modellazione di serie temporali. A differenza di LSTM e GRU, che sfruttano una memoria interna ricorrente, le CNN-1D estraggono pattern locali applicando operazioni di convoluzione lungo l'asse temporale. Questa caratteristica le rende particolarmente adatte a identificare configurazioni ricorrenti nei dati finanziari, quali picchi di volatilità, fasi di persistenza e transizioni di regime, mantenendo al contempo una struttura parametrica contenuta. Nel linguaggio delle CNN, la kernel size K determina l'ampiezza temporale della finestra analizzata da ciascun filtro mentre il numero di filters F indica quanti pattern distinti vengono appresi simultaneamente dal modello.

Struttura dell'input e convoluzione temporale

Il dataset supervisionato è lo stesso utilizzato per LSTM e GRU (cfr. Sottosezione 5.3). Per chiarire il funzionamento dell'operazione di convoluzione lungo l'asse temporale, si consideri un esempio numerico coerente con la struttura del dataset utilizzato nell'analisi empirica. Supponiamo di lavorare con una finestra temporale di ampiezza $L = 2$ e due variabili in input. Una singola osservazione in input assume quindi la forma:

$$X_t = \begin{bmatrix} \mathrm{RV}_{t-2} & \mathrm{ret}_{t-2} \\ \mathrm{RV}_{t-1} & \mathrm{ret}_{t-1} \end{bmatrix} = \begin{bmatrix} 0.1091 & 0.0231 \\ 0.0781 & 0.0099 \end{bmatrix}.$$

Con kernel size $K = 2$, il filtro convoluzionale opera sull'intera finestra temporale, producendo un singolo valore di output per ciascun filtro. Ogni filtro è caratterizzato da una matrice di pesi delle stesse dimensioni del kernel e da un termine di bias. A titolo esemplificativo, si consideri il seguente filtro:

$$W = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \qquad b = 0.$$

L'output convoluzionale si ottiene tramite il prodotto elemento per elemento seguito dalla somma:

$$W \odot X_t = \begin{bmatrix} 1 \cdot 0.1091 & 0 \cdot 0.0231 \\ 0 \cdot 0.0781 & -1 \cdot 0.0099 \end{bmatrix} = \begin{bmatrix} 0.1091 & 0 \\ 0 & -0.0099 \end{bmatrix},$$

$$0.1091 - 0.0099 = 0.0992.$$

Applicando la funzione di attivazione ReLU, si ottiene:

$$\mathrm{ReLU}(0.0992) = 0.0992.$$

Questo valore rappresenta l'attivazione del filtro per la finestra temporale considerata. Nel modello implementato vengono appresi $F = 16$ filtri distinti, ciascuno con pesi diversi, che analizzano la stessa finestra locale ma sono in grado di catturare pattern differenti, quali aumenti improvvisi della volatilità, fasi di persistenza o variazioni nei rendimenti.

Architettura e procedura di addestramento

L'architettura CNN-1D adottata prevede un layer *Conv1D* con F filtri, kernel fissa $K = 2$ e attivazione ReLU, seguito da un layer di *Global Average Pooling* (GAP), utilizzato per comprimere l'informazione estratta dai filtri in una rappresentazione compatta. La previsione $\widehat{\mathrm{RV}}_t$ è infine ottenuta tramite un layer denso con output lineare.

Con `padding="valid"`, la convoluzione opera esclusivamente su osservazioni passate, garantendo l'assenza di *look-ahead bias* nel forecasting. Gli altri parametri sono mantenuti coerenti con quelli adottati per i modelli precedentemente analizzati.

Ricerca degli iperparametri e selezione modello

La griglia di ricerca degli iperparametri per l'individuazione della configurazione ottimale è:

$$L \in \{1,2,3,4\}, \qquad F \in \{4,8,12,16\}, \qquad K = 2, \qquad \text{batch size} \in \{32,64,128\}.$$

La minimizzazione della RMSE OOS sul campione di validazione porta all'individuazione della seguente combinazione:

$$L = 2, \qquad F = 16, \qquad K = 2, \qquad \text{batch size} = 32.$$

Con tali valori, il numero di parametri addestrabili dello strato convoluzionale è:

$$N_{\text{Conv}} = F \cdot (K \cdot d + 1) = 16 \cdot (2 \cdot 2 + 1) = 80,$$

a cui si aggiungono:

$$N_{\text{Dense}} = F + 1 = 17.$$

Il numero complessivo di parametri addestrabili della CNN-1D è quindi:

$$N_p = 80 + 17 = 97.$$

La Figura 5.15 mostra l'andamento della training e validation loss nel run diagnostico.

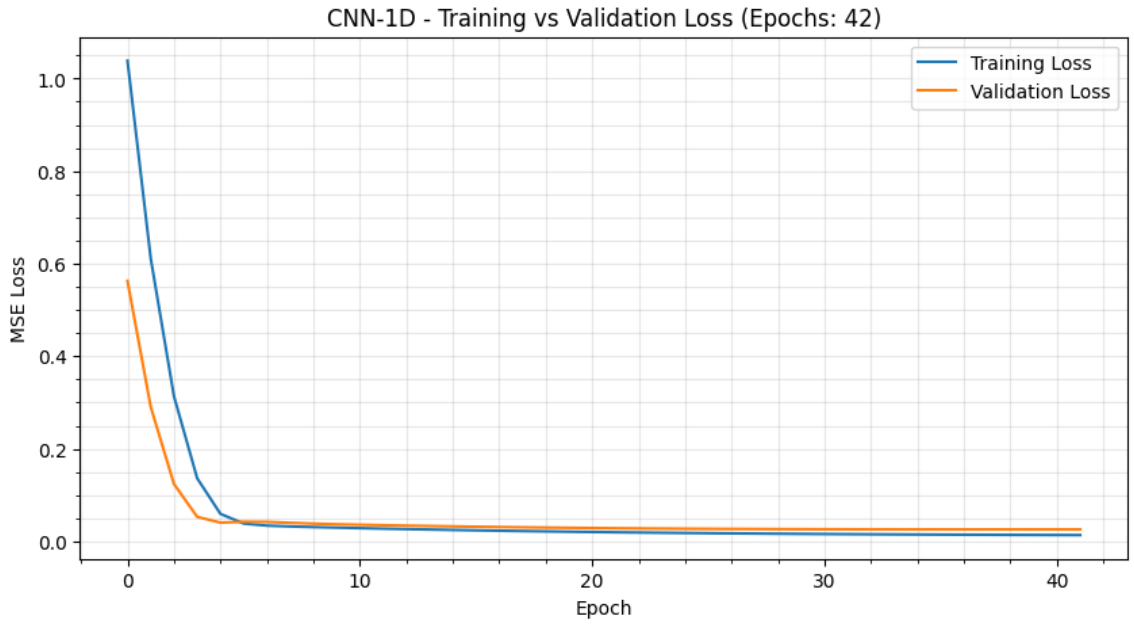


Figura 5.15: Andamento di training loss e validation loss durante l'addestramento della CNN-1D.

Infine, la Figura 5.16 riassume l'architettura della CNN-1D ottimale: la rete riceve in ingresso una sequenza di lunghezza $L = 2$ su due variabili (RV_m e ret_m), organizzata in un tensore $X_t \in \mathbb{R}^{2\times 2}$. La sequenza è processata da un layer Conv1D con $F = 16$ filtri e kernel size $K = 2$ (attivazione ReLU), quindi sintetizzata tramite GAP, e infine trasformata da un layer denso lineare per ottenere la previsione $\widehat{\text{RV}}_{m,t}$.

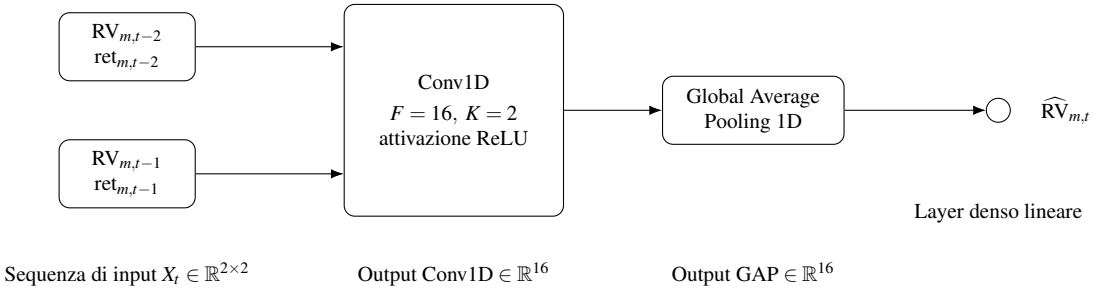


Figura 5.16: Architettura della CNN-1D ottimale selezionata.

N-BEATS: modello feedforward con stacking residuale

La N-BEATS è un'architettura completamente feedforward che costruisce la previsione attraverso una sequenza di blocchi collegati in modo residuale. Ogni blocco produce due componenti: un backcast, che approssima la parte di input "spiegata" dal blocco e viene sottratta ai residui, e un forecast, che viene sommato progressivamente alla previsione finale. In questo modo, i blocchi successivi operano su una rappresentazione residuale via via più "pulita", mantenendo al contempo una struttura compatta e direttamente confrontabile con le altre reti considerate.

Dataset supervisionato

A differenza delle architetture ricorrenti, N-BEATS non richiede una rappresentazione tridimensionale dei dati: l'input è infatti un vettore bidimensionale, analogo a quello utilizzato dalla MLP (cfr. Sottosezione 5.3).

Architettura implementata

Il modello è costruito in Keras per poter combinare operazioni di `Subtract` (sottrazione del backcast ai residui) e `Add` (aggregazione dei contributi di previsione). La specifica adottata prevede $n_{\text{stacks}} = 2$ blocchi in serie (*residual stacking*); in ciascun blocco sono presenti $n_{\text{layers}} = 1$ layer fully-connected con attivazione ReLU e $n_{\text{neurons}} = 2$ unità. Ciascun blocco termina con un layer lineare che produce il vettore latente θ, il quale definisce congiuntamente le componenti di backcast e forecast: la prima è utilizzata per aggiornare il residuo del segnale in ingresso, mentre la seconda contribuisce alla previsione finale. Con $L = 2$ e $d = 2$, si ha input size $= Ld = 4$ e horizon $= 1$. La dimensione del vettore θ è quindi:

$$\theta_{\text{size}} = \text{input size} + \text{horizon} = 4 + 1 = 5.$$

Ricerca degli iperparametri e selezione modello

La configurazione ottimale è individuata con una grid search sui due iperparametri principali (lags e neurons), mantenendo fissi $n_{\text{stacks}} = 2$ e $n_{\text{layers}} = 1$, e successivamente con una ricerca dedicata sulla batch size. Il criterio di selezione è RMSE OOS su:

$$L \in \{1,2,3,4\}, \qquad n_{\text{neurons}} \in \{2,4,6,8\}.$$

Fissati i valori migliori di L e n_{neurons}, la batch size viene scelta tra: $\{32,64,128,n\}$. La selezione conduce a:

$$L = 2, \qquad n_{\text{neurons}} = 2, \qquad \text{batch size} = 128, \qquad n_{\text{stacks}} = 2, \quad n_{\text{layers}} = 1.$$

Nel modello selezionato, ciascun blocco contiene:

- un Dense hidden: $(\text{input size} + 1) \cdot n_{\text{neurons}} = (4+1) \cdot 2 = 10$,
- un Dense θ: $(n_{\text{neurons}} + 1) \cdot \theta_{\text{size}} = (2+1) \cdot 5 = 15$.

Quindi, per ciascun blocco:

$$N_{\text{block}} = 10 + 15 = 25,$$

e con $n_{\text{stacks}} = 2$:

$$N_p = 2 \cdot 25 = 50 \quad \text{parametri addestrabili.}$$

La Figura 5.17 mostra che le curve decrescono in modo regolare e rimangono vicine lungo l'addestramento, indicando un processo stabile e privo di segnali evidenti di sovradattamento per l'architettura selezionata.

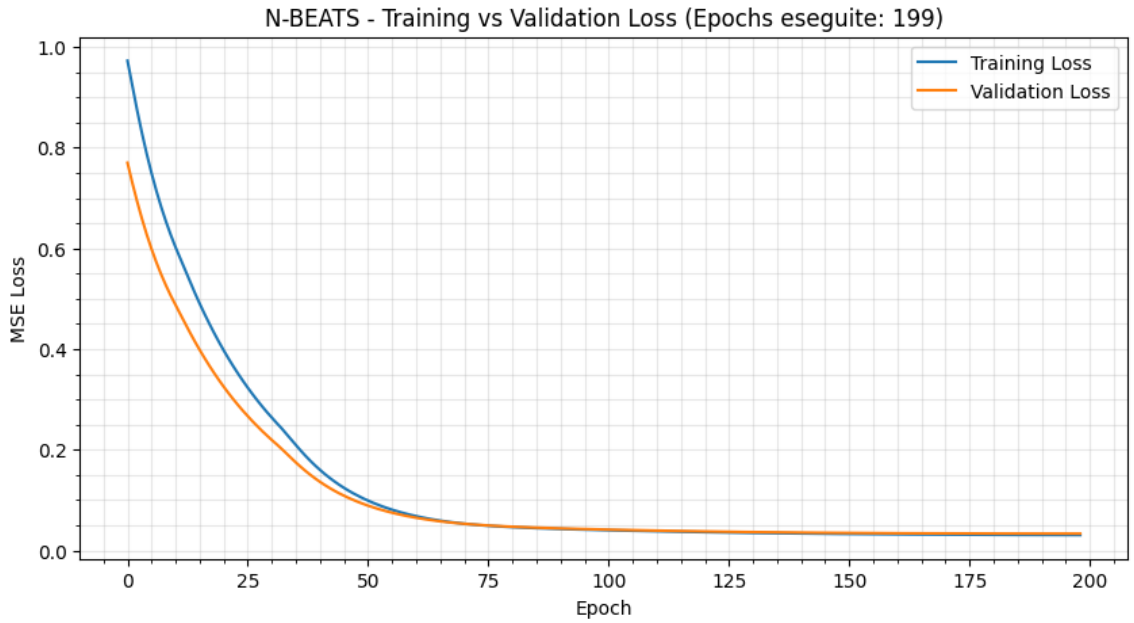


Figura 5.17: Andamento di training loss e validation loss durante l'addestramento del modello N-BEATS selezionato.

A completamento, la figura seguente riassume schematicamente la struttura del modello N-BEATS ottimale: due blocchi in cascata (uno per stack) con connessione residuale *backcast–residual* e somma dei forecast. L'input è una finestra $X_t \in \mathbb{R}^{2\times 2}$ appiattita in $x_t \in \mathbb{R}^4$; ciascun blocco ($n_{\text{layers}} = 1$, $n_{\text{neurons}} = 2$) produce un backcast $b^{(s)}$ e un forecast $f^{(s)}$, e la previsione finale è $\widehat{\text{RV}}_t = f^{(1)} + f^{(2)}$.

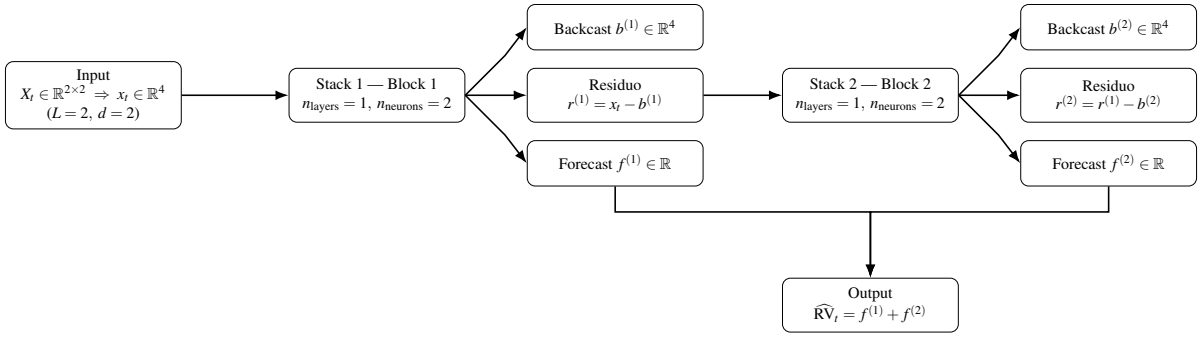


Figura 5.18: Architettura della N-BEATS ottimale selezionata.

5.4 Confronto complessivo e interpretazione dei risultati

Si confrontano dunque le prestazioni previsive del miglior modello econometrico individuato (gjrGARCH(1,1) con innovazioni Student-t) e delle architetture neurali.

Modello	RMSE_is	MAE_is	RMSE_oos	MAE_oos
GARCH	0.066387	0.043797	0.082897	0.048304
GRU	0.050831	0.029909	0.082669	0.046898
MLP	0.059554	0.038156	0.081947	0.050061
N-BEATS	0.057015	0.033382	0.081814	0.046967
CNN–1D	0.050892	0.030692	0.080968	0.046407
LSTM	0.055056	0.030779	**0.079304**	**0.044219**

Tabella 5.18: Confronto complessivo delle performance previsive IS e OOS (S&P 500).

Dalla Tabella 5.18 emerge in modo chiaro che tutte le architetture neurali superano il modello GARCH in termini di accuratezza predittiva fuori dal campione. In particolare, il gjr-GARCH(1,1), pur rappresentando una specificazione econometricamente solida e coerente con le caratteristiche stilizzate dei rendimenti finanziari, presenta la RMSE_oos più elevata tra i modelli considerati. Questo risultato segnala una minore capacità di adattamento alle dinamiche future della volatilità rispetto ai modelli basati su apprendimento automatico.

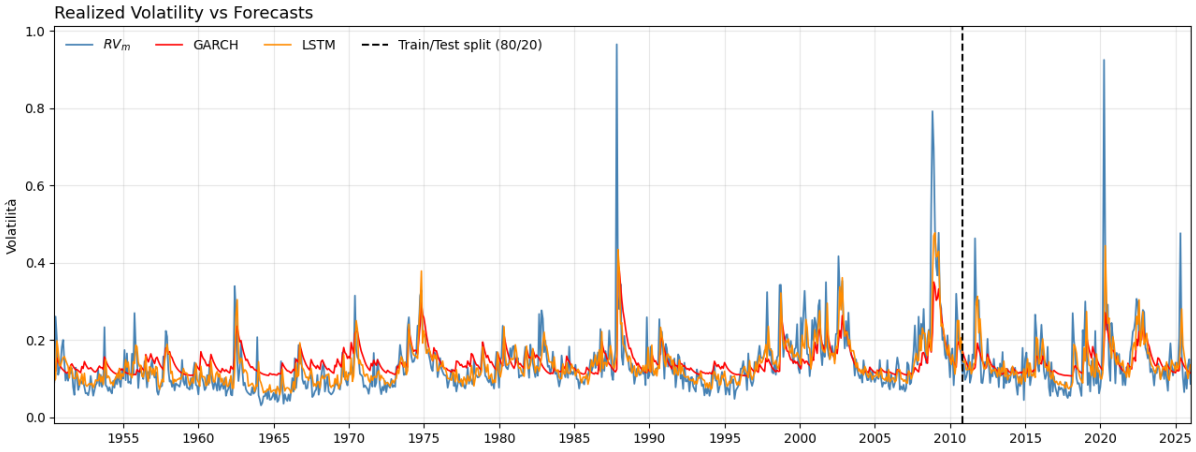
Tra le architetture neurali, la LSTM risulta il modello con le migliori prestazioni OOS, registrando il valore minimo sia di RMSE sia di MAE. Tale evidenza conferma il vantaggio derivante dalla presenza di uno stato interno e di meccanismi di gating, che consentono alla rete di catturare dipendenze temporali persistenti e non lineari nella dinamica della volatilità.

Le restanti architetture neurali mostrano performance complessivamente comparabili, ma con differenze rilevanti. La CNN-1D ottiene risultati molto vicini a quelli della LSTM, dimostrando che un'architettura convoluzionale compatta, basata sull'estrazione di pattern locali, può competere efficacemente con modelli ricorrenti più complessi. Il modello N-BEATS, pur essendo completamente feedforward, evidenzia una capacità predittiva superiore a quella della MLP e della GRU, a conferma dell'efficacia della decomposizione backcast–forecast e della struttura residuale nel modellare serie temporali finanziarie.

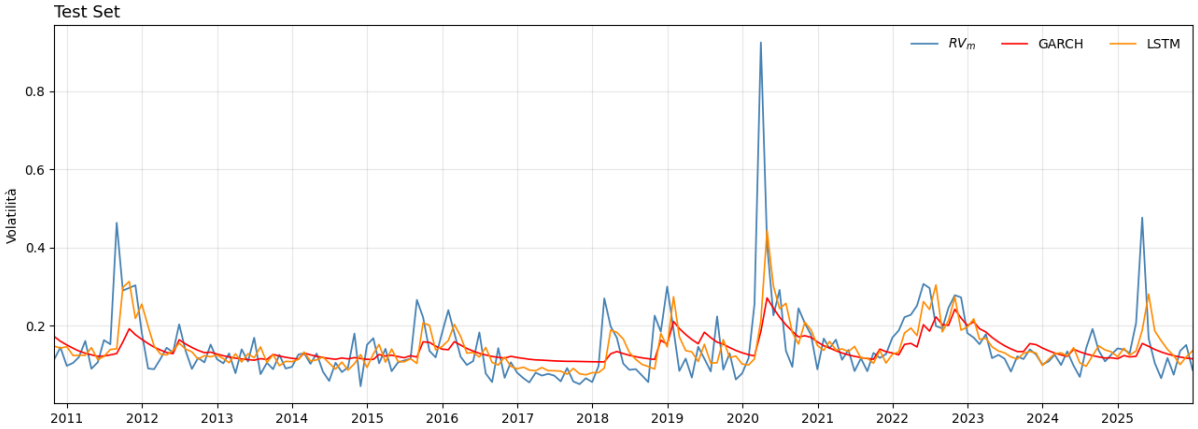
Dal punto di vista qualitativo, il confronto grafico (Figura 5.19) risulta pienamente coerente con le evidenze quantitative emerse dall'analisi delle metriche previsive. Entrambi i modelli sono in grado di riprodurre l'andamento di fondo della volatilità e di coglierne il clustering, ma mostrano differenze rilevanti nella risposta agli shock improvvisi.

In particolare, in corrispondenza dei picchi di volatilità più pronunciati, la LSTM mostra una maggiore reattività e un più rapido adattamento ai cambiamenti di regime, mentre il modello GARCH tende a produrre una dinamica più persistente e graduale, riflettendo la struttura parametrica della varianza condizionata e l'elevata inerzia tipica delle specificazioni GARCH.

Nel complesso, l'evidenza grafica suggerisce che, nel contesto analizzato, le architetture neurali presentano un vantaggio predittivo rispetto ai modelli GARCH tradizionali, in particolare nella capacità di adattarsi rapidamente a fasi di forte turbulenza dei mercati. Tale miglioramento in termini di accuratezza previsiva avviene tuttavia a fronte di una maggiore complessità computazionale e di una minore interpretabilità strutturale. Nel capitolo successivo, questi risultati vengono ulteriormente approfonditi ed estesi ad altre serie storiche finanziarie.



(a) Campione intero.



(b) Periodo di test.

Figura 5.19: Confronto tra RV_m e previsioni dei modelli GARCH ed LSTM (S&P 500).

5.5 Generalizzazione dei risultati

Dopo aver analizzato in dettaglio il caso dell'S&P 500, l'analisi viene estesa ad altre serie storiche finanziarie appartenenti a classi di attivo, settori economici e mercati geografici differenti, al fine di valutare la robustezza e la capacità di generalizzazione dei risultati ottenuti.

Simbolo	Classe di attivo	Periodo di analisi
SPX Index	Indice azionario	31-01-1950 – 31-12-2025
XAUUSD Curncy	Materia prima (commodity)	31-01-1975 – 31-12-2025
NKY Index	Indice azionario	31-01-1970 – 31-12-2025
GBPUSD Curncy	Cambio valutario (FX)	31-01-1971 – 31-12-2025
USGG10YR Index	Tasso di interesse (Treasury USA)	31-01-1971 – 31-12-2025
KO US Equity	Titolo azionario (consumer staples)	31-01-1968 – 31-12-2025
XOM US Equity	Titolo azionario (energia)	31-07-1980 – 31-12-2025
G IM Equity	Titolo azionario (finanziario/assicurativo)	31-01-1985 – 31-12-2025
7203 JP Equity	Titolo azionario (industriale/automotive)	30-09-1974 – 31-12-2025

Tabella 5.19: Serie finanziarie analizzate.

Le serie sono state selezionate con l'obiettivo di ampliare il più possibile il dominio di applicazione dell'analisi, includendo strumenti rappresentativi dei mercati azionari, valutari, delle materie prime e dei tassi di interesse, nonché titoli azionari appartenenti a settori industriali eterogenei. In particolare, per i titoli azionari vengono considerati: *The Coca-Cola Company* (KO US Equity) per il settore dei beni di consumo, *Exxon Mobil Corporation* (XOM US Equity) per il settore energetico, *Assicurazioni Generali S.p.A.* (G IM Equity) per il settore finanziario–assicurativo e *Toyota Motor Corporation* (7203 JP Equity) per il settore industriale, consentendo un confronto trasversale tra modelli su realtà aziendali e contesti di mercato differenti. La Tabella 5.19 riporta l'elenco completo delle *time series* considerate nell'analisi, tutte scaricate dalla piattaforma Bloomberg. Per l'analisi grafica delle serie storiche, nonché per il confronto dettagliato tra le previsioni dei modelli, si rimanda all'Appendice A.

Per ciascuna serie si applica la medesima pipeline di costruzione dei dati, basata su frequenza mensile, target di previsione pari alla volatilità realizzata RV_m e input costituito da RV_m e ret_m, nonché la stessa procedura di valutazione OOS, così da garantire la piena confrontabilità dei risultati tra mercati e classi di attivo differenti. Il modello GARCH riportato nella Tabella 5.20 corrisponde, in ciascun caso, alla migliore specificazione econometrica selezionata sulla base delle prestazioni previsive OOS tra le diverse varianti considerate.

Tabella 5.20: Prestazioni IS e OOS dei modelli per ciascuna serie.

Serie	Modello	RMSE_is	MAE_is	RMSE_oos	MAE_oos
XAUUSD Curncy	gjrGARCH-norm	0.076708	0.053699	0.047851	0.038017
	MLP	0.072764	0.046870	0.045890	0.033809
	N-BEATS	0.073574	0.046442	0.045808	0.033193
	GRU	0.067819	0.045716	0.044246	0.033679
	CNN-1D	0.064574	0.043991	0.042772	**0.032078**
	LSTM	0.065767	0.044832	**0.042622**	0.032300
NKY Index	N-BEATS	0.085115	0.057743	0.093667	0.064467
	eGARCH-sged	0.089433	0.064417	0.093439	0.058877
	MLP	0.080756	0.053403	0.087980	0.057362
	GRU	0.077113	0.050090	0.087608	0.057175
	LSTM	0.078155	0.051047	0.087085	0.056238
	CNN-1D	0.075752	0.049271	**0.086947**	**0.055511**
GBPUSD Curncy	CNN-1D	0.025776	0.018622	0.036102	0.020632
	gjrGARCH-norm	0.036642	0.029100	0.035934	0.023286
	MLP	0.028604	0.020351	0.034999	0.020709
	N-BEATS	0.036899	0.026559	0.034668	0.019594
	LSTM	0.027385	0.019599	0.034427	0.019581
	GRU	0.025963	0.018904	**0.033373**	**0.019437**
USGG10YR	apARCH-norm	0.064568	0.051887	0.050984	0.037941
	MLP	0.050793	0.035445	0.047001	0.029262
	N-BEATS	0.050944	0.035048	0.046511	0.028056
	CNN-1D	0.047449	0.033588	0.046129	0.026681
	LSTM	0.048250	0.034073	0.045857	0.027353
	GRU	0.048143	0.034115	**0.043841**	**0.026584**

(continua nella pagina successiva)

Serie	Modello	RMSE_is	MAE_is	RMSE_oos	MAE_oos
KO US Equity	sGARCH-sstd	0.089545	0.055268	0.081264	0.055201
	N-BEATS	0.087261	0.055212	0.079117	0.055193
	MLP	0.083905	0.051067	0.076847	0.050412
	GRU	0.081863	0.049538	0.076411	0.051321
	CNN-1D	0.080345	0.049685	0.076072	0.050229
	LSTM	0.080696	0.049019	**0.075613**	**0.049085**
XOM US Equity	gjrGARCH-norm	0.112883	0.062722	0.149316	0.085620
	MLP	0.100147	0.062391	0.132941	0.078013
	CNN-1D	0.086211	0.049206	0.105328	0.062748
	N-BEATS	0.087752	0.050229	0.104119	0.061931
	GRU	0.088998	0.050144	0.102895	0.065051
	LSTM	0.086459	0.048873	**0.097258**	**0.059307**
G IM Equity	apARCH-std	0.105092	0.077363	0.101026	0.074150
	MLP	0.096607	0.067387	0.090475	0.062596
	N-BEATS	0.096965	0.065559	0.088190	0.058843
	LSTM	0.091088	0.064085	0.086995	0.059317
	CNN-1D	0.087273	0.060294	0.085387	0.054820
	GRU	0.079496	0.056852	**0.084991**	**0.053873**
7203 JP Equity	CNN-1D	0.102655	0.072335	0.107276	0.076805
	N-BEATS	0.121744	0.079998	0.106819	**0.069673**
	sGARCH-sstd	0.119969	0.084581	0.104218	0.074091
	LSTM	0.107539	0.075504	0.103952	0.073404
	GRU	0.107290	0.075474	0.103443	0.073241
	MLP	0.112808	0.078312	**0.101528**	0.070749

Nel caso di XAUUSD, le reti ricorrenti e convoluzionali risultano particolarmente efficaci: LSTM e CNN-1D ottengono le migliori performance OOS. Questo risultato suggerisce che la volatilità dell'oro presenti pattern locali e dipendenze sequenziali che vengono catturate in modo efficiente da architetture in grado di modellare dinamiche non lineari e relazioni temporali persistenti.

Per l'indice azionario giapponese NKY, la CNN-1D emerge come modello più accurato, con un vantaggio contenuto ma stabile rispetto alle architetture ricorrenti. Tale evidenza è coerente con l'ipotesi che, in alcuni mercati azionari, pattern temporali di breve periodo possano essere sufficienti per migliorare la previsione della volatilità, rendendo le convoluzioni causali particolarmente competitive.

Nel caso del cambio GBPUSD, la GRU fornisce la migliore accuratezza fuori dal campione, seguita a breve distanza da LSTM e N-BEATS. La maggiore competitività relativa delle architetture ricorrenti nel contesto FX appare plausibile, in quanto la volatilità dei cambi riflette spesso dinamiche persistenti, transizioni di regime e shock macroeconomici che si propagano nel tempo.

Per quanto riguarda la serie USGG10YR, rappresentativa del mercato dei tassi di interesse, la GRU ottiene la migliore performance OOS, con un vantaggio netto rispetto al modello GARCH. Questo risultato suggerisce che, anche per serie macrofinanziarie legate alla dinamica dei rendimenti obbligazionari, architetture ricorrenti compatte siano in grado di model-

lare in modo più efficace la persistenza e le variazioni graduali della volatilità rispetto alle specificazioni econometriche tradizionali.

Per il titolo azionario KO US Equity, la LSTM risulta il modello più performante, con differenze moderate rispetto a CNN-1D e GRU. Anche in questo caso, la presenza di una memoria interna consente di catturare efficacemente la persistenza della volatilità e la risposta a shock idiosincratici tipici di imprese mature appartenenti al settore dei beni di consumo.

Passando ai titoli azionari aggiuntivi, per XOM US Equity (settore energetico) la LSTM fornisce la migliore accuratezza fuori dal campione, mentre GRU e N-BEATS mostrano prestazioni comparabili ma leggermente inferiori. La maggiore complessità e volatilità del settore energetico, fortemente influenzato da shock esogeni legati ai prezzi delle materie prime e a fattori geopolitici, sembra favorire modelli in grado di catturare dipendenze temporali persistenti piuttosto che strutture puramente locali.

Nel caso di G IM Equity, rappresentativo del settore finanziario–assicurativo europeo, le architetture neurali mantengono un chiaro vantaggio rispetto al GARCH, con GRU e CNN-1D che ottengono le migliori prestazioni OOS. Questo risultato indica che la volatilità del titolo riflette una combinazione di persistenza e pattern di breve periodo, coerente con la natura del business assicurativo e con l'esposizione a cicli finanziari e shock sistemici.

Infine, per 7203 JP Equity (Toyota Motor Corporation), i risultati mostrano una maggiore eterogeneità nel ranking dei modelli neurali. In questo caso, l'MLP ottiene la migliore performance OOS, mentre GRU e LSTM seguono a breve distanza. Tale evidenza suggerisce che, per questo titolo industriale, una modellazione non lineare relativamente semplice possa essere sufficiente a catturare la dinamica della volatilità, mentre architetture più complesse non garantiscono un vantaggio netto in termini di generalizzazione.

In sintesi, l'estensione dell'analisi a mercati, settori e contesti geografici differenti conferma che l'approccio basato su reti neurali offre un vantaggio predittivo robusto rispetto ai modelli GARCH nel contesto considerato. Al tempo stesso, la variabilità del ranking tra architetture di deep learning evidenzia che non esiste un modello universalmente dominante: la scelta architetturale ottimale dipende dalle proprietà dinamiche della serie, dalla struttura degli shock e dal contesto economico di riferimento.

Conclusione

In questa tesi è stato analizzato il problema della previsione della volatilità realizzata concentrandosi sul confronto tra modelli econometrici e diverse reti neurali.

I risultati ottenuti mostrano come, nel contesto considerato, le architetture neurali tendano a fornire prestazioni previsive superiori rispetto ai modelli tradizionali, soprattutto in termini di accuratezza out-of-sample. Tale evidenza emerge in modo relativamente robusto su più mercati, pur con differenze nel ranking delle singole architetture, a conferma del fatto che non esiste un modello universalmente dominante.

L'analisi mette inoltre in luce come i modelli econometrici mantengano un ruolo rilevante dal punto di vista interpretativo e strutturale: essi continuano a offrire una rappresentazione parsimoniosa e teoricamente fondata della dinamica della volatilità condizionata, pur mostrando limiti nella capacità di adattamento rapido a cambiamenti di regime e a pattern non lineari complessi. Le reti neurali, al contrario, risultano più flessibili e reattive, ma tale flessibilità si accompagna a una minore interpretabilità dei meccanismi interni che guidano la previsione.

Nel complesso, i risultati suggeriscono che il vantaggio delle architetture neurali non debba essere interpretato come una sostituzione definitiva dei modelli econometrici, bensì come un'evidenza empirica a favore di un approccio complementare. In particolare, la scelta del modello più appropriato dipende dal trade-off tra accuratezza previsiva, interpretabilità e stabilità del modello, nonché dalle caratteristiche specifiche della serie temporale analizzata.

Il lavoro apre a diverse possibili estensioni future, tra cui l'inclusione di un insieme informativo più ricco negli input delle reti neurali (ad esempio variabili macroeconomiche, indicatori di mercato o misure di volatilità implicita), nonché l'analisi di architetture ibride in grado di combinare la struttura dei modelli econometrici con la flessibilità delle reti neurali.

In conclusione, questa tesi fornisce un contributo empirico al dibattito sul confronto tra modelli econometrici e approcci di machine learning nella previsione della volatilità, mostrando come le reti neurali rappresentino uno strumento promettente, ma sicuramente non esaustivo, per l'analisi di un fenomeno complesso e multidimensionale come la dinamica della volatilità nei mercati finanziari.

Bibliografia

[1] Torben G. Andersen e Tim Bollerslev. «Answering the Skeptics: Yes, Standard Volatility Models Do Provide Accurate Forecasts». In: *International Economic Review* 39.4 (1998), pp. 885–905.

[2] Torben G. Andersen et al. «Volatility and Correlation Forecasting». In: *Handbook of Economic Forecasting* 1 (2006), pp. 777–878.

[3] Yoshua Bengio, Patrice Simard e Paolo Frasconi. «Learning Long-Term Dependencies with Gradient Descent Is Difficult». In: *IEEE Transactions on Neural Networks* 5.2 (1994), pp. 157–166.

[4] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[5] Fischer Black. «Studies of Stock Price Volatility Changes». In: *Proceedings of the American Statistical Association, Business and Economic Statistics Section* (1976), pp. 177–181.

[6] Tim Bollerslev. «A Conditionally Heteroskedastic Time Series Model for Speculative Prices and Rates of Return». In: *The Review of Economics and Statistics* 69.3 (1987), pp. 542–547.

[7] Tim Bollerslev. «Generalized Autoregressive Conditional Heteroskedasticity». In: *Journal of Econometrics* 31.3 (1986), pp. 307–327.

[8] Tim Bollerslev, George Tauchen e Hao Zhou. «Expected Stock Returns and Variance Risk Premia». In: *Review of Financial Studies* 22.11 (2009), pp. 4463–4492.

[9] Kyunghyun Cho et al. «Learning Phrase Representations Using RNN Encoder–Decoder for Statistical Machine Translation». In: *arXiv preprint arXiv:1406.1078* (2014). URL: `https://arxiv.org/abs/1406.1078`.

[10] Rama Cont. «Empirical Properties of Asset Returns: Stylized Facts and Statistical Issues». In: *Quantitative Finance* 1.2 (2001), pp. 223–236.

[11] Zhuanxin Ding, Clive W. J. Granger e Robert F. Engle. «A Long Memory Property of Stock Market Returns and a New Model». In: *Journal of Empirical Finance* 1.1 (1993), pp. 83–106.

[12] Jeffrey L. Elman. «Finding Structure in Time». In: *Cognitive Science* 14.2 (1990), pp. 179–211.

[13] Robert F. Engle. «Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation». In: *Econometrica* 50.4 (1982), pp. 987–1007.

[14] Christian Francq e Jean-Michel Zakoian. *GARCH Models: Structure, Statistical Inference and Financial Applications*. John Wiley & Sons, 2019.

[15] Lawrence R. Glosten, Ravi Jagannathan e David E. Runkle. «On the Relation Between the Expected Value and the Volatility of the Nominal Excess Return on Stocks». In: *The Journal of Finance* 48.5 (1993), pp. 1779–1801.

[16] Ian Goodfellow, Yoshua Bengio e Aaron Courville. *Deep Learning*. MIT Press, 2016.

[17] Bruce E. Hansen. «Autoregressive Conditional Density Estimation». In: *International Economic Review* 35.3 (1994), pp. 705–730.

[18] Peter R. Hansen e Asger Lunde. «A Forecast Comparison of Volatility Models: Does Anything Beat a GARCH(1,1)?» In: *Journal of Applied Econometrics* 20.7 (2005), pp. 873–889.

[19] Sepp Hochreiter e Jürgen Schmidhuber. «Long Short-Term Memory». In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

[20] Kurt Hornik, Maxwell Stinchcombe e Halbert White. «Multilayer Feedforward Networks Are Universal Approximators». In: *Neural Networks* 2.5 (1989), pp. 359–366.

[21] John C. Hull. *Opzioni, Futures e altri derivati*. 11a edizione. Milano: Pearson, 2022.

[22] J.P. Morgan. *RiskMetrics Technical Document*. J.P. Morgan, 1996.

[23] Ioannis Karatzas e Steven E. Shreve. *Brownian Motion and Stochastic Calculus*. Graduate Texts in Mathematics. New York: Springer, 1988.

[24] Diederik P. Kingma e Jimmy Ba. «Adam: A Method for Stochastic Optimization». In: *arXiv preprint arXiv:1412.6980* (2014).

[25] Guokun Lai et al. «Modeling Long- and Short-Term Temporal Patterns with Deep Neural Networks». In: *Proceedings of the 41st International ACM SIGIR Conference* (2018), pp. 95–104.

[26] Yann LeCun et al. «Gradient-Based Learning Applied to Document Recognition». In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.

[27] Moshe Leshno et al. «Multilayer Feedforward Networks with a Nonpolynomial Activation Function Can Approximate Any Function». In: *Neural Networks* 6.6 (1993), pp. 861–867.

[28] Hao Li et al. «Visualizing the Loss Landscape of Neural Nets». In: (2018).

[29] Zachary C. Lipton, John Berkowitz e Charles Elkan. «A Critical Review of Recurrent Neural Networks for Sequence Learning». In: *arXiv preprint arXiv:1506.00019* (2015).

[30] Asger Lunde e Peter R. Hansen. «A Consistency Test for Models of Conditional Mean and Variance». In: *Journal of Econometrics* 128.2 (2005), pp. 475–513.

[31] Alexander J. McNeil, Rüdiger Frey e Paul Embrechts. *Quantitative Risk Management: Concepts, Techniques and Tools*. Revised Edition. Princeton University Press, 2015.

[32] Robert C. Merton. «On Estimating the Expected Return on the Market: An Exploratory Investigation». In: *Journal of Financial Economics* 8.4 (1980), pp. 323–361.

[33] Daniel B. Nelson. «Conditional Heteroskedasticity in Asset Returns: A New Approach». In: *Econometrica* 59.2 (1991), pp. 347–370.

[34] Boris N. Oreshkin et al. «N-BEATS: Neural Basis Expansion Analysis for Interpretable Time Series Forecasting». In: *International Conference on Learning Representations*. 2020.

[35] Razvan Pascanu, Tomas Mikolov e Yoshua Bengio. «On the Difficulty of Training Recurrent Neural Networks». In: *Proceedings of the International Conference on Machine Learning* (2013), pp. 1310–1318.

[36] Ser-Huang Poon e Clive W. J. Granger. «Forecasting Volatility in Financial Markets: A Review». In: *Journal of Economic Literature* 41.2 (2003), pp. 478–539. DOI: 10.1257/002205103765762743.

[37] Sebastian Ruder. «An Overview of Gradient Descent Optimization Algorithms». In: *arXiv preprint arXiv:1609.04747* (2016).

[38] Nitish Srivastava et al. «Dropout: A Simple Way to Prevent Neural Networks from Overfitting». In: *Journal of Machine Learning Research* 15.1 (2014), pp. 1929–1958.

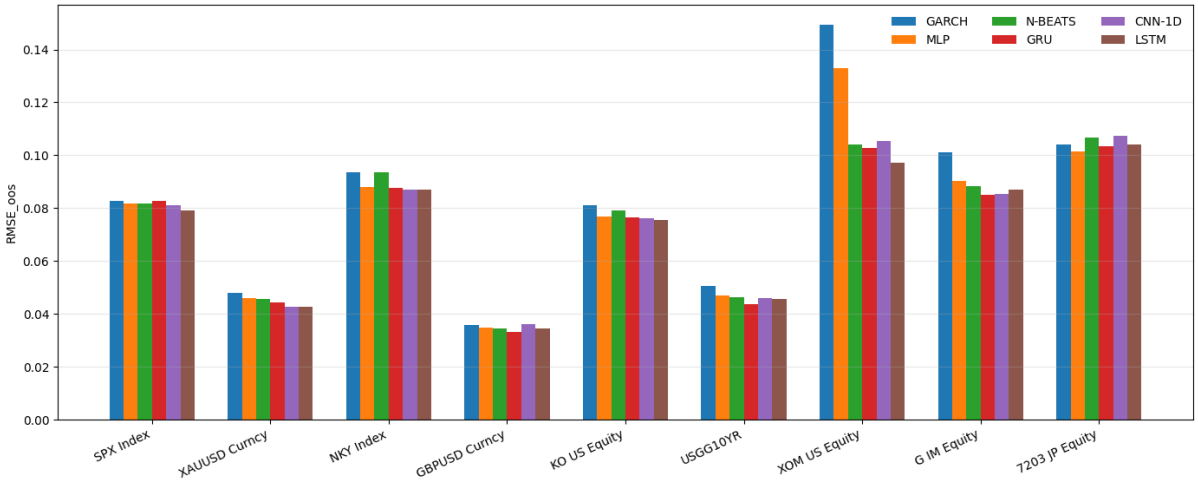
[39] Ilya Sutskever, Oriol Vinyals e Quoc V. Le. «Sequence to Sequence Learning with Neural Networks». In: *arXiv preprint arXiv:1409.3215* (2014).

[40] Ruey S. Tsay. *Analysis of Financial Time Series*. 3rd. Wiley, 2010.

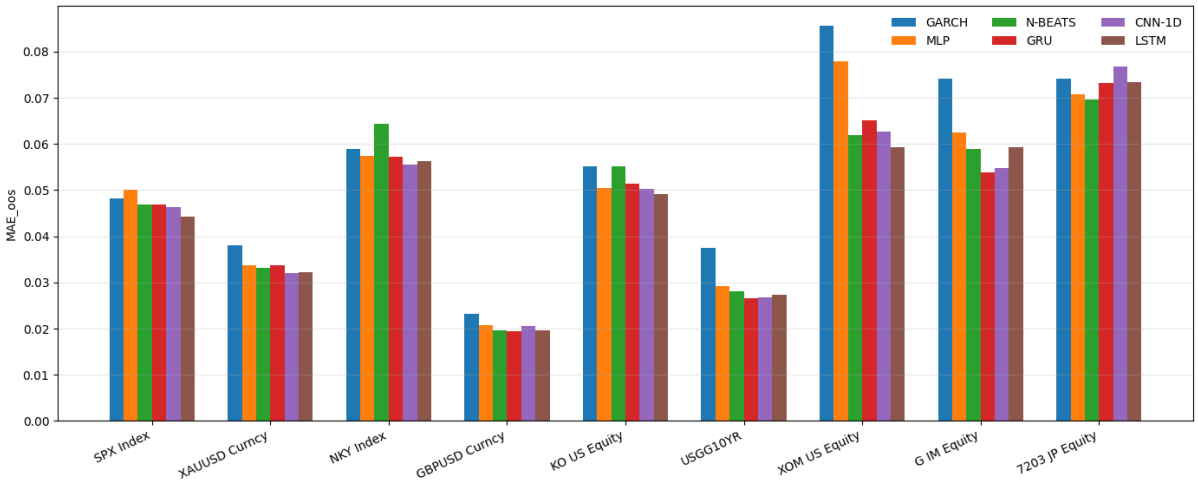
[41] Jean-Michel Zakoian. «Threshold Heteroskedastic Models». In: *Journal of Economic Dynamics and Control* 18.5 (1994), pp. 931–955.

A. Grafici

Confronto complessivo dei modelli



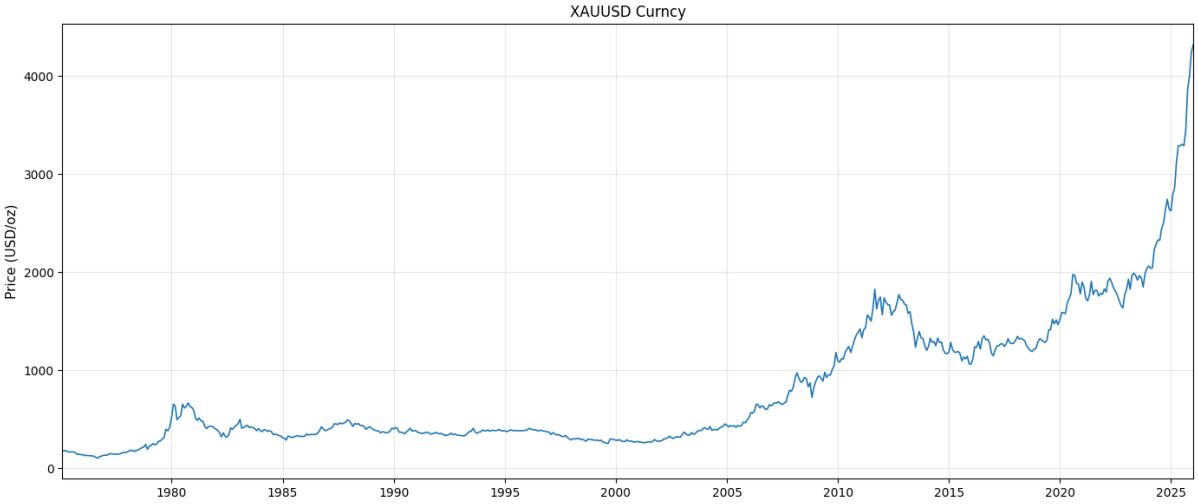
(a) Prestazioni previsive in termini di RMSE_oos (S&P 500).



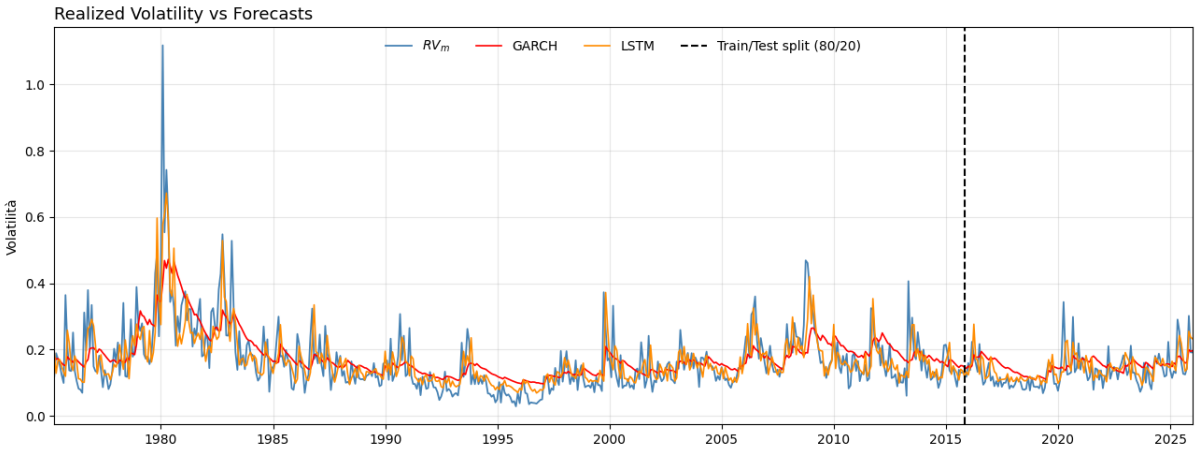
(b) Prestazioni previsive in termini di MAE_oos (S&P 500).

Figura A.1: Confronto delle metriche OOS per le diverse serie e architetture considerate.

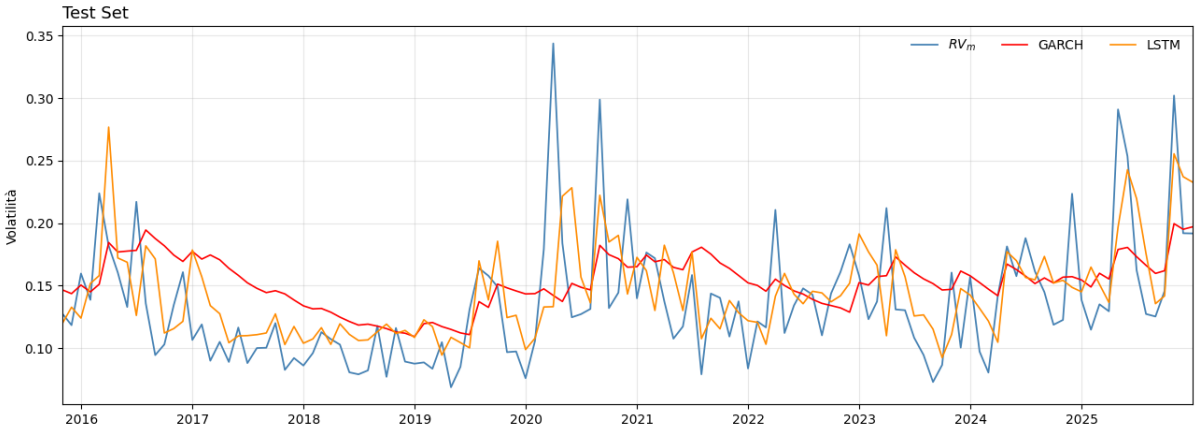
XAUUSD Curncy



(a) Prezzo di chiusura.



(b) Campione intero.



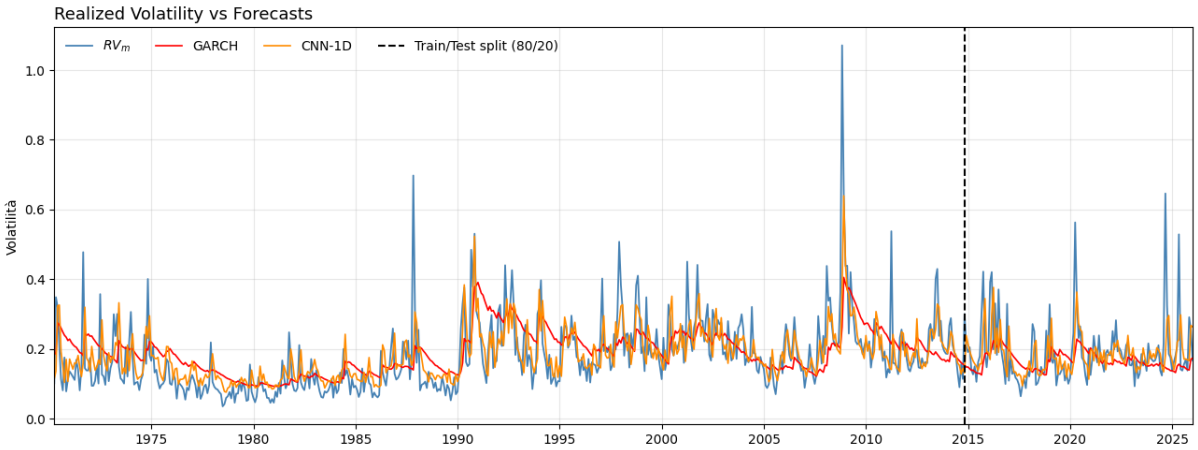
(c) Periodo di test.

Figura A.2: Risultati grafici per XAUUSD Curncy.

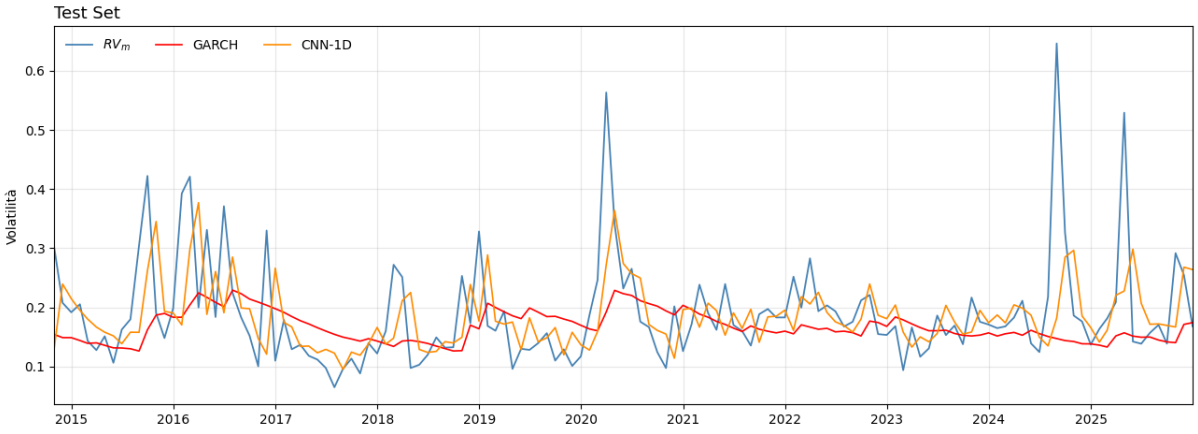
NKY Index



(a) Prezzo di chiusura.



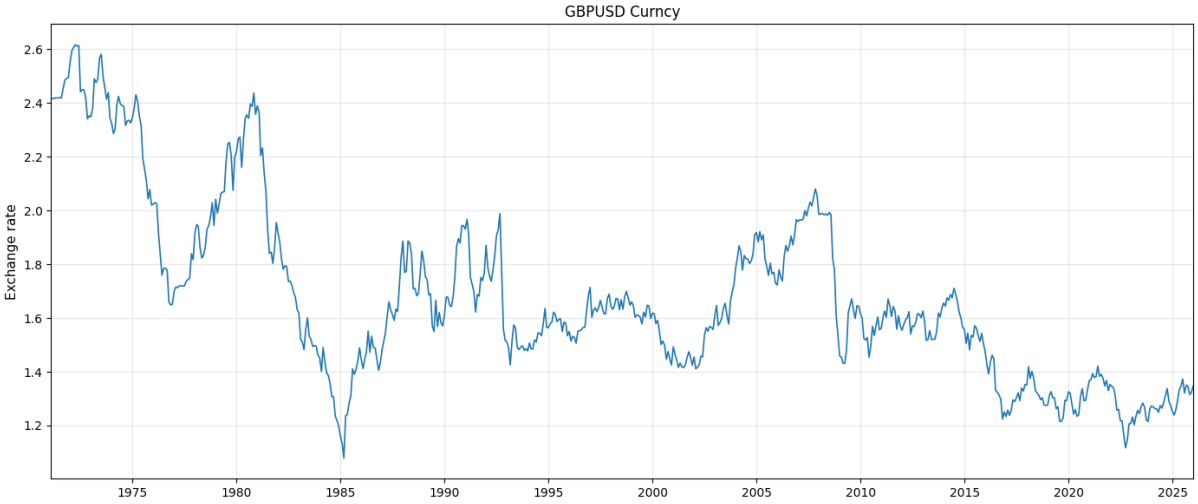
(b) Campione intero.



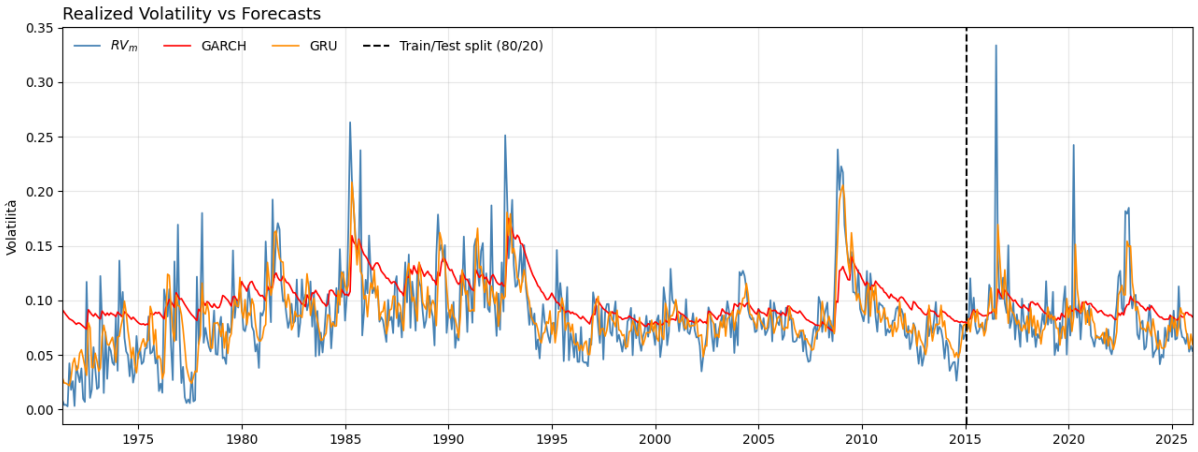
(c) Periodo di test.

Figura A.3: Risultati grafici per NKY Index.

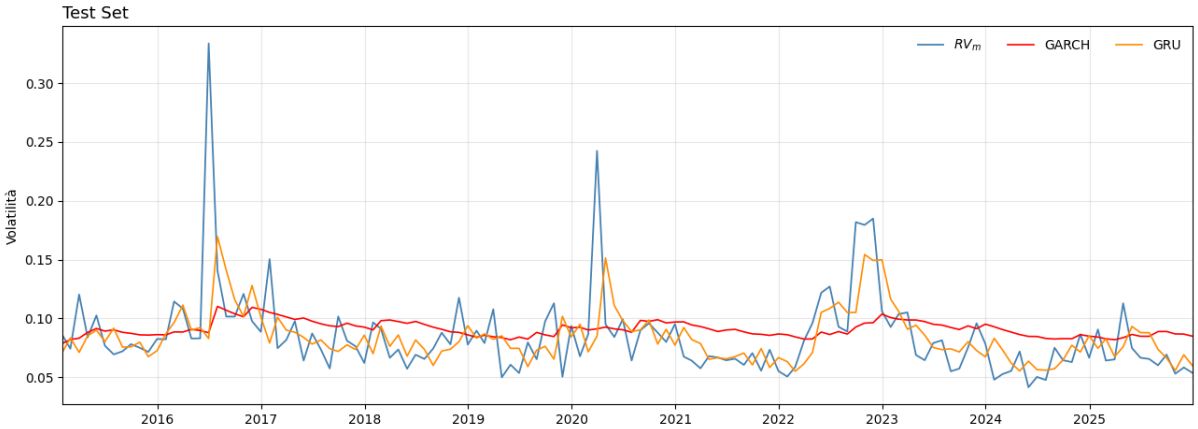
GBPUSD Curncy



(a) Prezzo di chiusura.



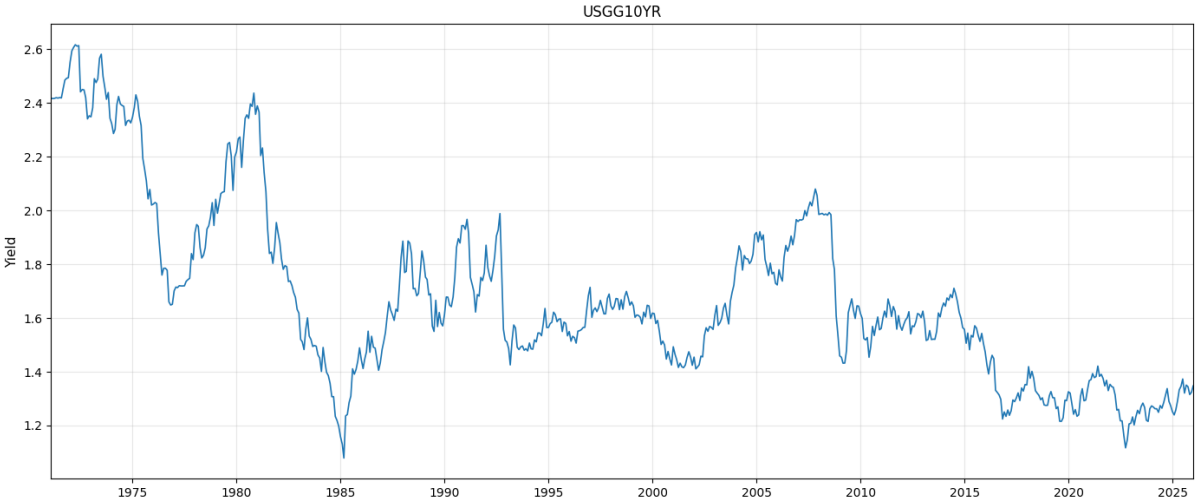
(b) Campione intero.



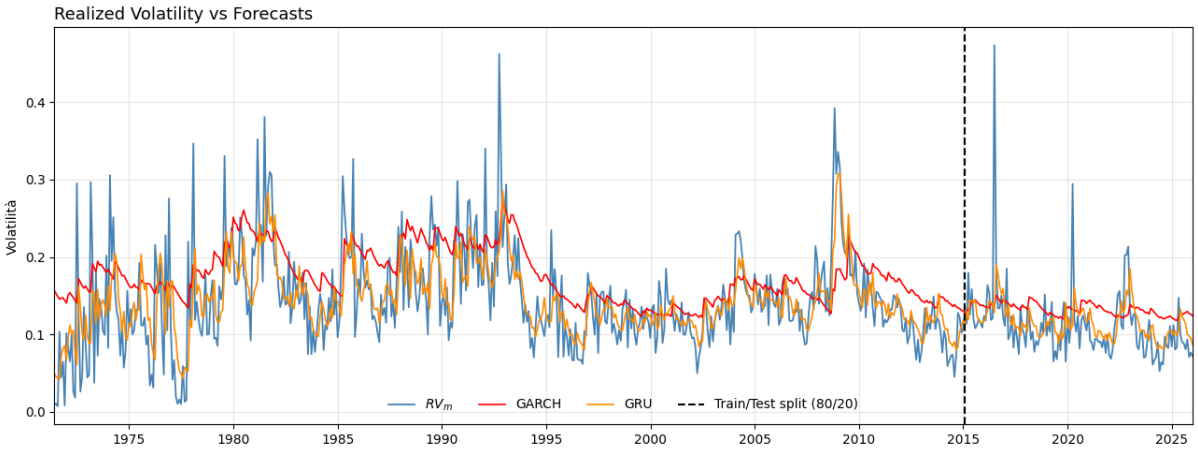
(c) Periodo di test.

Figura A.4: Risultati grafici per GBPUSD Curncy.

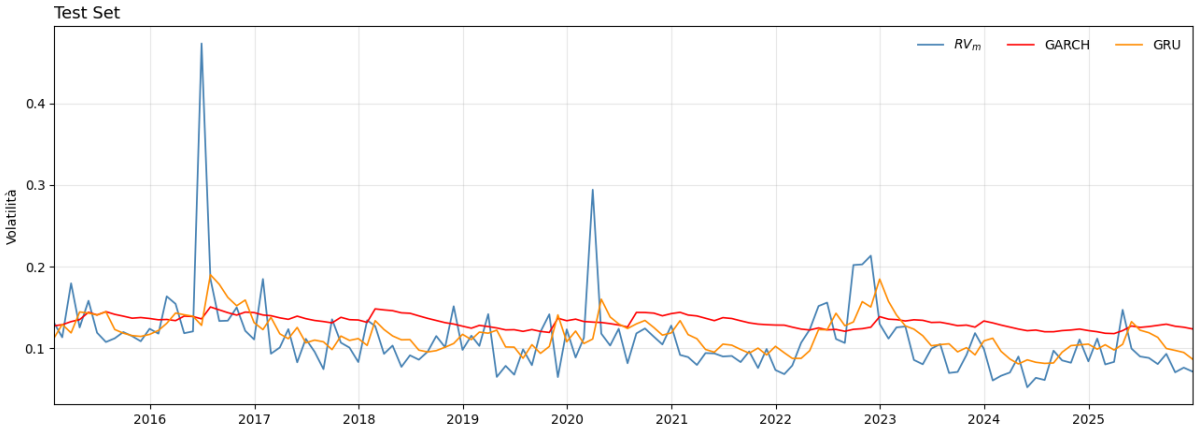
USGG10YR



(a) Rendimento a scadenza.



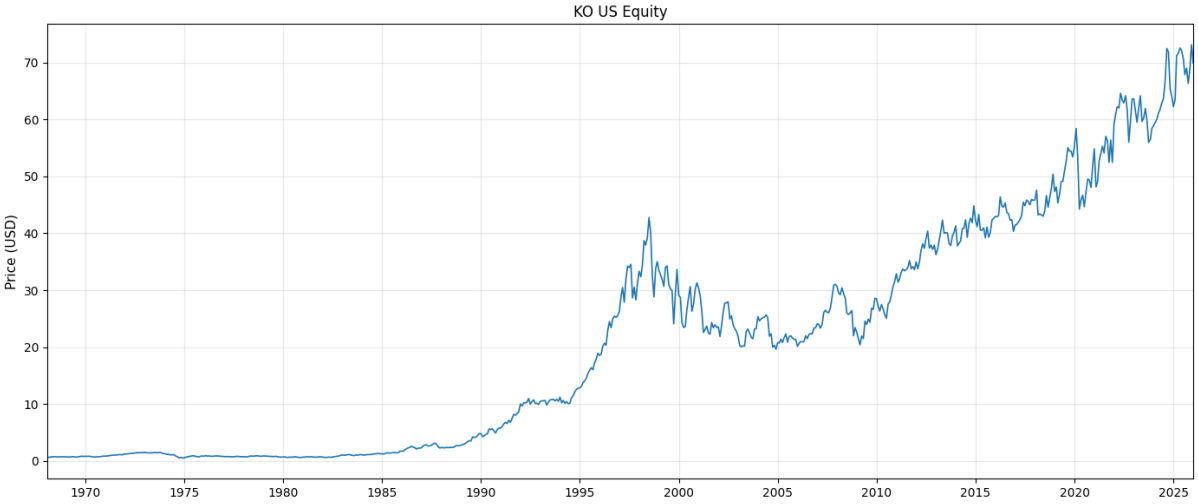
(b) Campione intero.



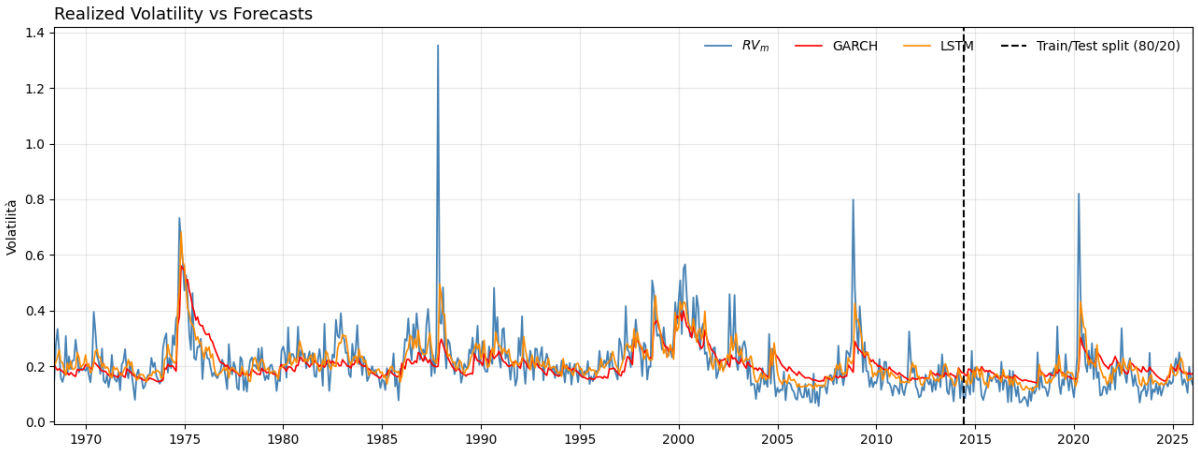
(c) Periodo di test.

Figura A.5: Risultati grafici per USGG10YR.

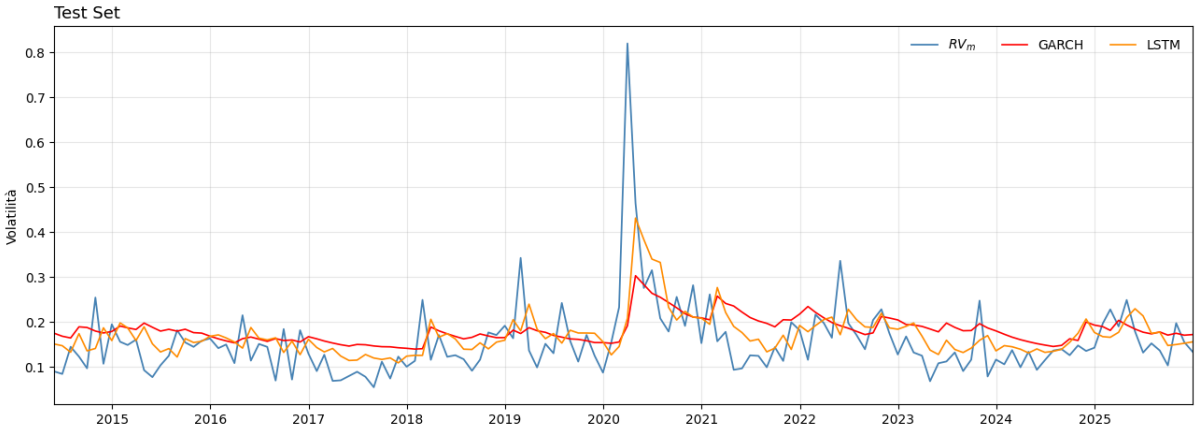
KO US Equity



(a) Prezzo di chiusura.



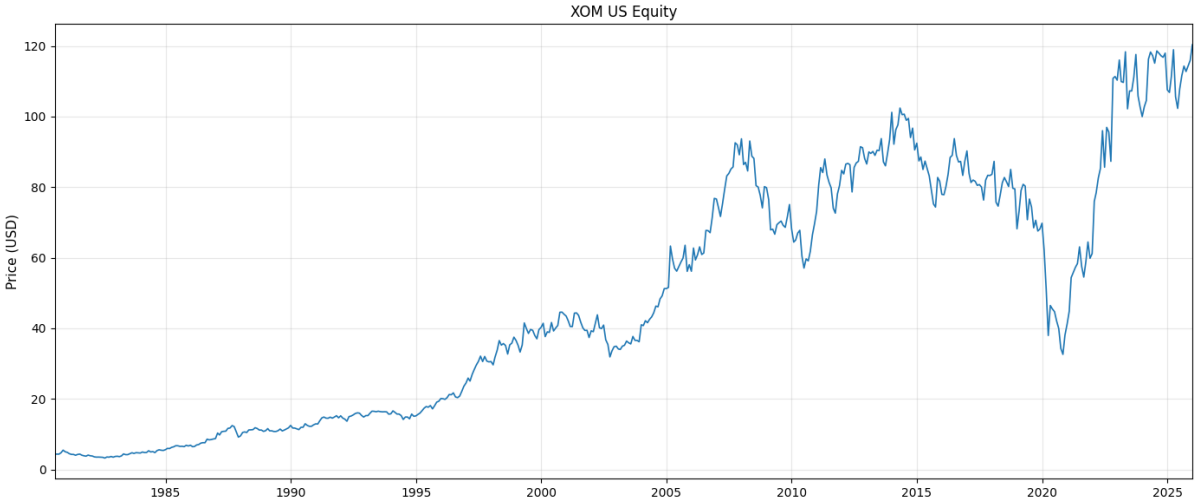
(b) Campione intero.



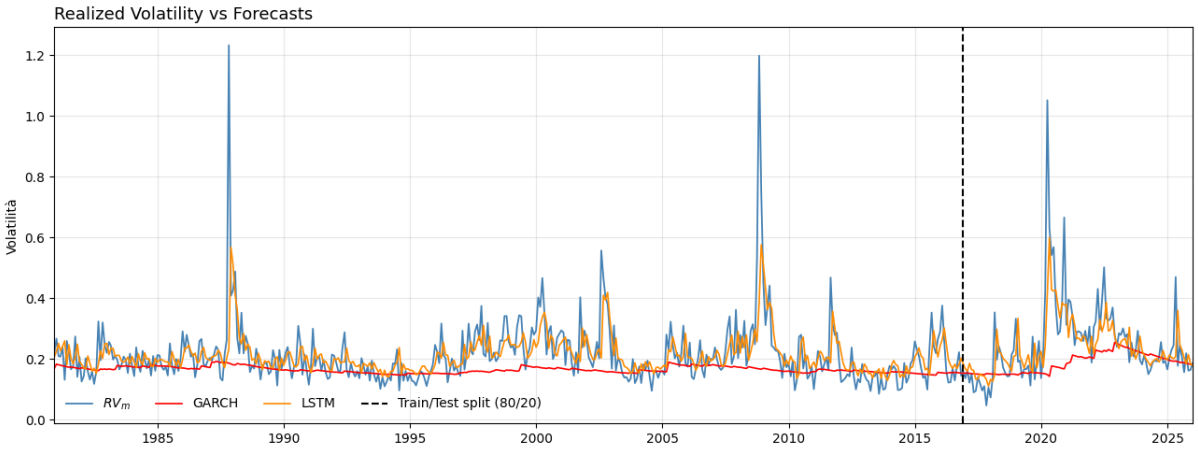
(c) Periodo di test.

Figura A.6: Risultati grafici per KO US Equity.

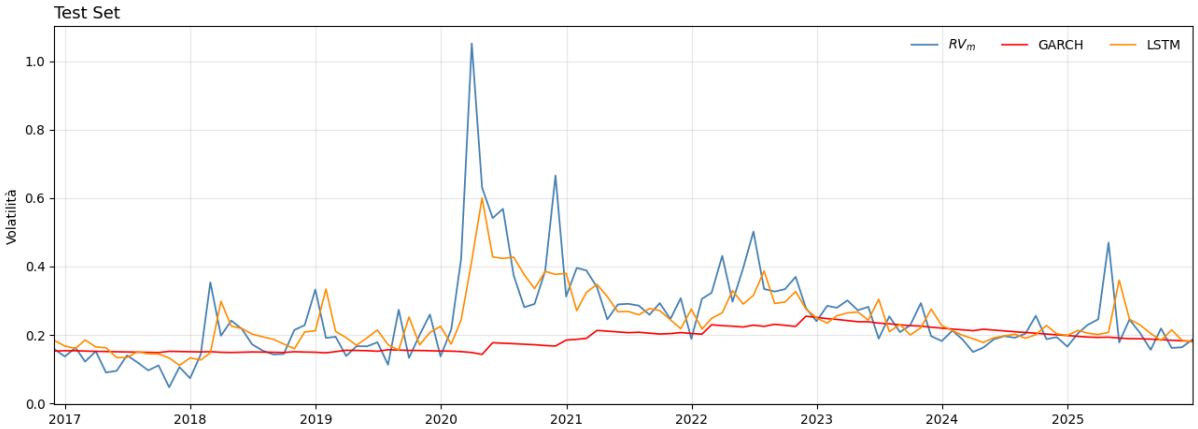
XOM US Equity



(a) Prezzo di chiusura.



(b) Campione intero.



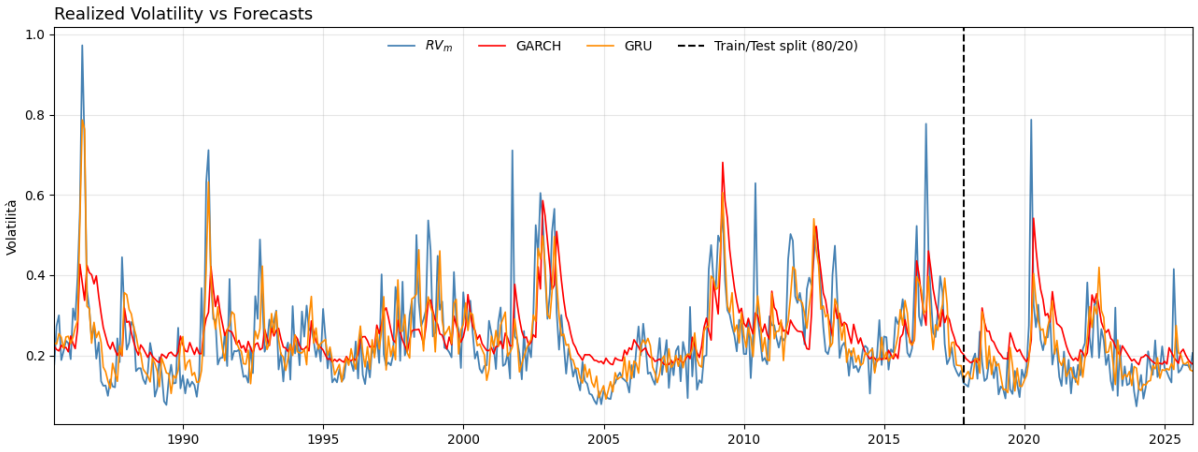
(c) Periodo di test.

Figura A.7: Risultati grafici per XOM US Equity.

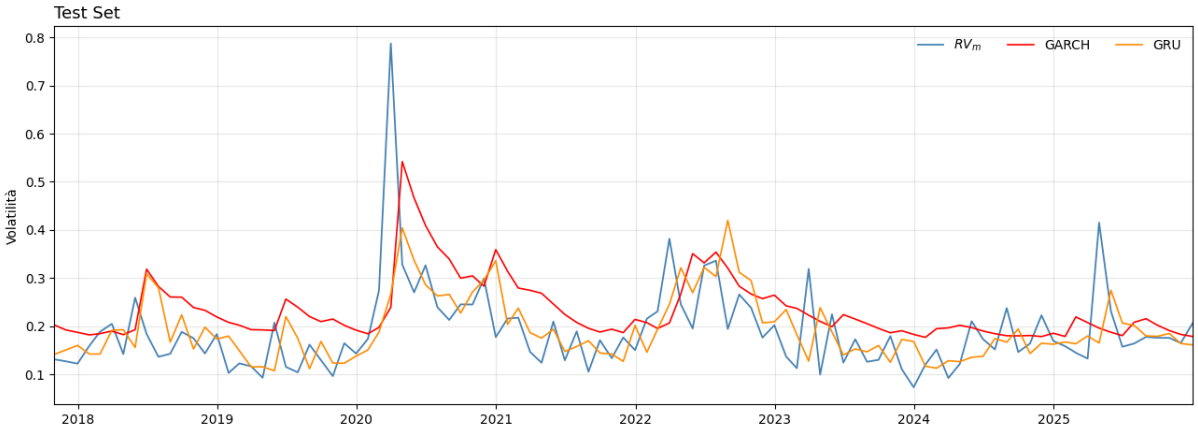
G IM Equity



(a) Prezzo di chiusura.



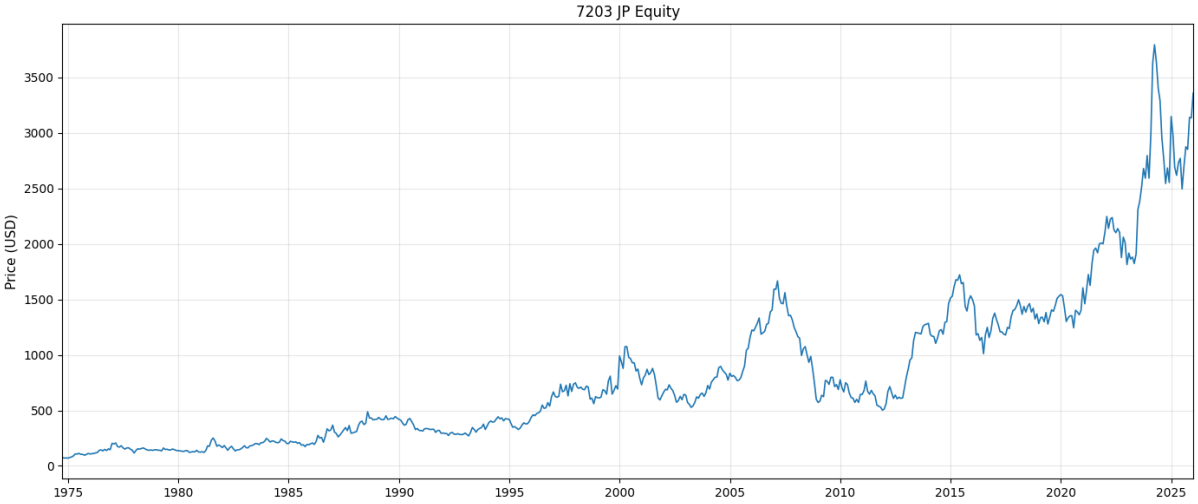
(b) Campione intero.



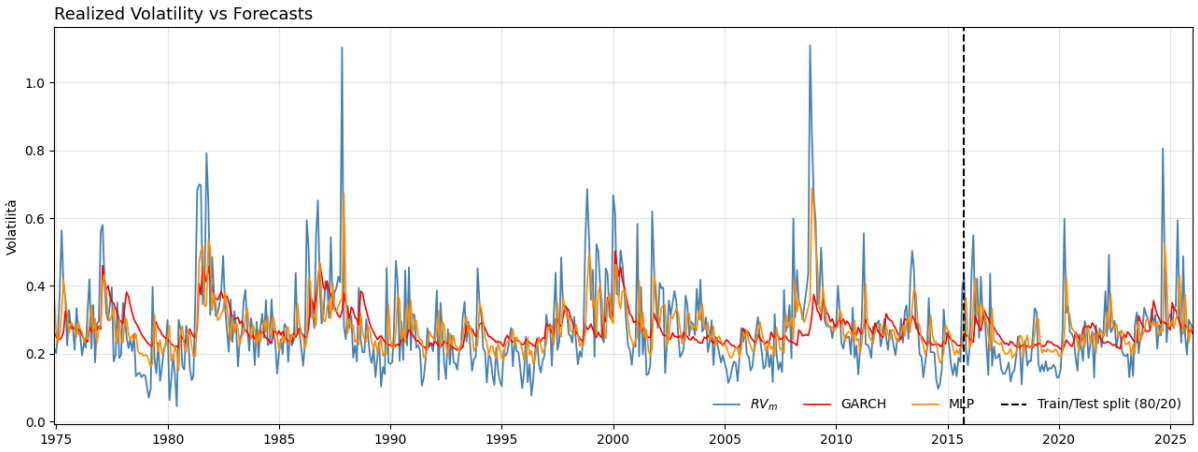
(c) Periodo di test.

Figura A.8: Risultati grafici per G IM Equity.

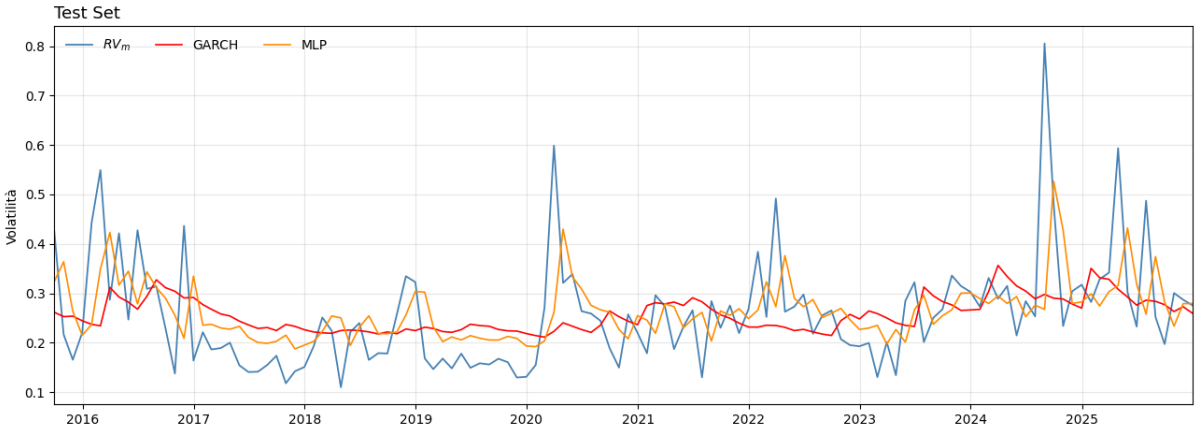
7203 JP Equity



(a) Prezzo di chiusura.



(b) Campione intero.



(c) Periodo di test.

Figura A.9: Risultati grafici per 7203 JP Equity.

B. Codici R e Python

Vengono qui riportati i frammenti essenziali del codice utilizzato nell'analisi empirica dell'S&P 500, in particolare: (i) la procedura di costruzione del dataset mensile utilizzato da tutti i modelli; (ii) la definizione delle architetture neurali standard (MLP, LSTM, GRU, CNN-1D); (iii) il codice completo del modello N-BEATS, la cui costruzione richiede una specificazione non standard; (iv) la specifica e la procedura per le previsioni OOS dei modelli GARCH.

Preparazione dei dati

Listing B.1: Dai prezzi giornalieri alle serie mensili `ret_m` e `RV_m`

```python
import numpy as np
import pandas as pd

# Caricamento dei prezzi di chiusura giornalieri
px = pd.read_csv("data/SPX_px_last.csv", parse_dates=["Date"])
px = px.set_index("Date").sort_index()

# Rendimenti logaritmici giornalieri
px["ret_d"] = np.log(px["Close"]).diff()
px = px.dropna()

def realized_vol(ret_d: pd.Series) -> float:
    ret_d = ret_d.dropna()
    n = ret_d.size
    if n == 0:
        return np.nan
    return float(np.sqrt(252.0 * (ret_d**2).sum() / n))

# Serie mensili
RV_m  = px["ret_d"].resample("ME").apply(realized_vol)
ret_m = px["ret_d"].resample("ME").sum()

# Dataset finale utilizzato nei modelli econometrici e neurali
data_m = pd.concat([RV_m.rename("RV_m"), ret_m.rename("ret_m")], axis
    =1).dropna()

# Esportazione del dataset finale
data_m.to_csv("data/dataset_m.csv")
```

Modelli GARCH

Si riporta la procedura utilizzata per stimare modelli GARCH con la libreria `rugarch` e produrre previsioni one-step-ahead nel campione OOS (split 80/20).

Listing B.2: Specifica e rolling one-step-ahead OOS in `rugarch`

```r
library(xts)
library(rugarch)
library(dplyr)

# Load dataset mensile (costruito in Python)
data_m <- read.csv("data/dataset_m.csv", stringsAsFactors = FALSE)
data_m$Date <- as.Date(data_m$Date)
data_m <- data_m[order(data_m$Date), ]

n <- nrow(data_m)
train_end <- floor(0.80 * n)

train_data <- data_m[1:train_end, ]
test_data  <- data_m[(train_end + 1):n, ]

ret_train <- xts(train_data$ret_m, order.by = train_data$Date)
ret_test  <- xts(test_data$ret_m,  order.by = test_data$Date)
ret_full  <- rbind(ret_train, ret_test)

RV_test <- xts(test_data$RV_m, order.by = test_data$Date)

outN <- NROW(ret_test)
ANN_FACTOR <- sqrt(12)

rmse <- function(a, b) sqrt(mean((a - b)^2))
mae  <- function(a, b) mean(abs(a - b))

garch_models <- c("sGARCH", "eGARCH", "gjrGARCH", "apARCH")
distros      <- c("norm", "std", "ged", "sstd", "sged")

garch_results <- list()

for (m in garch_models) {
  for (distr in distros) {

    spec <- ugarchspec(
      variance.model = list(model = m, garchOrder = c(1, 1)),
      mean.model     = list(armaOrder = c(0, 0), include.mean = TRUE),
      distribution.model = distr
    )

    fit <- tryCatch(
      ugarchfit(spec = spec, data = ret_full, out.sample = outN, solver
          = "hybrid"),
      error = function(e) NULL
    )
    if (is.null(fit)) next

    fore <- tryCatch(
      ugarchforecast(fit, n.ahead = 1, n.roll = outN - 1),
      error = function(e) NULL
```

```r
    )
    if (is.null(fore)) next

    sigma_oos <- as.numeric(sigma(fore))
    sigma_oos <- xts(sigma_oos, order.by = index(ret_test)) * ANN_
        FACTOR

    merged_oos <- na.omit(merge(sigma_oos, RV_test))
    colnames(merged_oos) <- c("sigma_hat", "RV_true")

    rmse_out <- rmse(merged_oos$sigma_hat, merged_oos$RV_true)
    mae_out  <- mae( merged_oos$sigma_hat, merged_oos$RV_true)

    garch_results[[length(garch_results) + 1]] <- data.frame(
      Model = m, Dist = distr, RMSE_out = rmse_out, MAE_out = mae_out
    )
  }
}

garch_perf <- bind_rows(garch_results) %>% arrange(RMSE_out, MAE_out)
print(garch_perf)
```

Modelli neurali standard: definizione delle architetture

La fase di data preparation, scaling, split e training segue una procedura uniforme per tutti i modelli. In questa sezione si riportano esclusivamente le architetture adottate per i modelli neurali standard.

Listing B.3: MLP

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input

# Input: vettore 2D (samples, timesteps * features)
model = Sequential()
model.add(Input(shape=(train_X.shape[1],)))
model.add(Dense(neurons, activation="relu"))
model.add(Dense(1, activation="linear"))
model.compile(loss="mse", optimizer="adam")
```

Listing B.4: LSTM

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Input

# Input: tensore 3D (samples, timesteps, features)
model = Sequential()
model.add(Input(shape=(train_X.shape[1], train_X.shape[2])))
model.add(LSTM(neurons, activation="tanh"))
model.add(Dense(1, activation="linear"))
model.compile(loss="mse", optimizer="adam")
```

Listing B.5: GRU

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GRU, Dense, Input
```

```
3
4  # Input: tensore 3D (samples, timesteps, features)
5  model = Sequential()
6  model.add(Input(shape=(train_X.shape[1], train_X.shape[2])))
7  model.add(GRU(neurons, activation="tanh"))
8  model.add(Dense(1, activation="linear"))
9  model.compile(loss="mse", optimizer="adam")
```

Listing B.6: CNN-1D

```
1  from tensorflow.keras.models import Sequential
2  from tensorflow.keras.layers import Conv1D, Dense, Input,
       GlobalAveragePooling1D
3
4  # Input: tensore 3D (samples, timesteps, features)
5  model = Sequential()
6  model.add(Input(shape=(train_X.shape[1], train_X.shape[2])))
7  model.add(
8      Conv1D(
9          filters=filters,
10         kernel_size=kernel_size,
11         activation="relu",
12         padding="valid"
13     )
14 )
15 model.add(GlobalAveragePooling1D())
16 model.add(Dense(1, activation="linear"))
17 model.compile(loss="mse", optimizer="adam")
```

N-BEATS: implementazione completa

In quanto architettura non standard, si riporta di seguito il codice completo utilizzato per N-BEATS: costruzione del dataset supervisionato, definizione del blocco, costruzione del modello e procedura di training/diagnostica.

Listing B.7: Costruzione dataset supervisionato

```
1  import numpy as np
2
3  def make_nbeats_supervised(data, lags):
4      X, y = [], []
5      for t in range(lags, data.shape[0]):
6          window = data[t-lags:t, :]
7          X.append(window.flatten())
8          y.append(data[t, 0])
9      X = np.array(X)
10     y = np.array(y).reshape(-1, 1)
11     return X, y
```

Listing B.8: Definizione del blocco base

```
1  import tensorflow as tf
2  from tensorflow.keras.layers import Dense
3
4  class NBeatsBlock(tf.keras.layers.Layer):
```

```
    def __init__(self, input_size, theta_size, horizon, n_neurons,
        n_layers, **kwargs):
        super().__init__(**kwargs)
        self.input_size = int(input_size)
        self.theta_size = int(theta_size)
        self.horizon    = int(horizon)
        self.hidden = [Dense(int(n_neurons), activation="relu") for _
            in range(int(n_layers))]
        self.theta_layer = Dense(self.theta_size, activation="linear",
            name="theta")

    def call(self, inputs):
        x = inputs
        for layer in self.hidden:
            x = layer(x)
        theta = self.theta_layer(x)
        backcast = theta[:, :self.input_size]
        forecast = theta[:, -self.horizon:]
        return backcast, forecast
```

Listing B.9: Costruzione del modello

```
from tensorflow.keras import Model, Input
from tensorflow.keras.layers import Add, Subtract

def build_nbeats_model(input_size, horizon=1, n_stacks=2, n_layers=1,
    n_neurons=2, name="NBEATS"):
    input_size = int(input_size)
    horizon    = int(horizon)
    theta_size = input_size + horizon

    inp = Input(shape=(input_size,), name="input")

    backcast, forecast = NBeatsBlock(
        input_size=input_size,
        theta_size=theta_size,
        horizon=horizon,
        n_neurons=n_neurons,
        n_layers=n_layers,
        name="block_0"
    )(inp)

    residuals = Subtract(name="residuals_0")([inp, backcast])

    for i in range(1, int(n_stacks)):
        backcast_i, forecast_i = NBeatsBlock(
            input_size=input_size,
            theta_size=theta_size,
            horizon=horizon,
            n_neurons=n_neurons,
            n_layers=n_layers,
            name=f"block_{i}"
        )(residuals)

        residuals = Subtract(name=f"residuals_{i}")([residuals,
            backcast_i])
        forecast  = Add(name=f"forecast_add_{i}")([forecast, forecast_i
            ])
```

```
34
35      model = Model(inputs=inp, outputs=forecast, name=name)
36      return model
```

Listing B.10: Training, diagnostica e previsione OOS

```
1  from sklearn.preprocessing import MinMaxScaler
2  from sklearn.metrics import mean_squared_error as mse
3  from sklearn.metrics import mean_absolute_error as mae
4  import tensorflow as tf
5
6  def NBEATS_model(data_m, lags, batch_size, epochs=250,
7                   n_stacks=2, n_layers=1, n_neurons=2,
8                   validation=False, early_stop=False):
9
10     tf.keras.backend.clear_session()
11
12     # 1) Scaling su [RV_m, ret_m]
13     values = data_m[["RV_m", "ret_m"]].values
14     scaler = MinMaxScaler(feature_range=(-1, 1))
15     scaled = scaler.fit_transform(values)
16
17     # 2) Supervised (flattened)
18     X, y = make_nbeats_supervised(scaled, lags=int(lags))
19
20     # 3) Split (60/20/20 per NN; 80/20 per run finale)
21     n = X.shape[0]
22     train_row = int(round(0.60 * n))
23     valid_row = int(round(0.80 * n))
24
25     if validation:
26         X_train, y_train = X[:train_row], y[:train_row]
27         X_oos,   y_oos   = X[train_row:valid_row], y[train_row:
               valid_row]
28     else:
29         X_train, y_train = X[:valid_row], y[:valid_row]
30         X_oos,   y_oos   = X[valid_row:], y[valid_row:]
31
32     # 4) Build + compile
33     model = build_nbeats_model(
34         input_size=X.shape[1],
35         horizon=1,
36         n_stacks=n_stacks,
37         n_layers=n_layers,
38         n_neurons=n_neurons,
39         name="NBEATS"
40     )
41     model.compile(loss="mse", optimizer="adam")
42
43     # 5) Fit
44     callbacks = []
45     if early_stop and validation:
46         callbacks.append(tf.keras.callbacks.EarlyStopping(
47             monitor="val_loss", mode="min", patience=5, min_delta=1e-5,
48             restore_best_weights=True, verbose=0
49         ))
50
51     learning = model.fit(
```

```
        X_train, y_train,
        epochs=int(epochs),
        batch_size=int(batch_size),
        validation_data=(X_oos, y_oos) if validation else None,
        shuffle=False, verbose=0, callbacks=callbacks
    )

    # 6) Inverse scaling
    def inv_yhat(yhat_scaled, X_scaled):
        last_ret_scaled = X_scaled[:, -1].reshape(-1, 1)
        inv = np.concatenate([yhat_scaled, last_ret_scaled], axis=1)
        return scaler.inverse_transform(inv)[:, 0]

    def inv_ytrue(y_scaled, X_scaled):
        last_ret_scaled = X_scaled[:, -1].reshape(-1, 1)
        inv = np.concatenate([y_scaled, last_ret_scaled], axis=1)
        return scaler.inverse_transform(inv)[:, 0]

    yhat_train = model.predict(X_train, verbose=0)
    yhat_oos   = model.predict(X_oos,   verbose=0)

    y_train_inv    = inv_ytrue(y_train, X_train)
    yhat_train_inv = inv_yhat(yhat_train, X_train)

    y_oos_inv      = inv_ytrue(y_oos, X_oos)
    yhat_oos_inv   = inv_yhat(yhat_oos, X_oos)

    rmse_in  = np.sqrt(mse(y_train_inv, yhat_train_inv))
    mae_in   = mae(y_train_inv, yhat_train_inv)
    rmse_out = np.sqrt(mse(y_oos_inv, yhat_oos_inv))
    mae_out  = mae(y_oos_inv, yhat_oos_inv)

    # yhat full allineata all'indice di data_m
    yhat_full = np.concatenate((np.full((lags,), np.nan),
       yhat_train_inv, yhat_oos_inv), axis=0)

    return model, learning.history, (rmse_in, mae_in, rmse_out, mae_out
       ), yhat_full
```

Elenco delle figure

1.1 Rendimenti logaritmici mensili e volatilità realizzata mensile dell'S&P 500. . . 9
1.2 Andamento giornaliero dell'indice VIX nel periodo 1990–2025. 10
1.3 Rendimenti mensili dell'S&P 500 e indice VIX. 11
1.4 Distribuzione dei rendimenti logaritmici giornalieri. 12
1.5 Q–Q plot dei rendimenti logaritmici giornalieri. 12
1.6 Relazione tra RV_m e RV_{m+1}. 13
1.7 Evidenza del fenomeno di mean reversion nella RV_m. 14
1.8 Relazione tra rendimenti logaritmici giornalieri e variazione del VIX. 14
1.9 ACF della volatilità realizzata settimanale RV_w. 15
1.10 Volatility Risk Premium. 16

3.1 Relazione lineare tra volatilità futura stimata $\hat{\sigma}_t$ e volatilità passata σ_{t-1}. 22
3.2 Schema semplificato di neurone artificiale. 23
3.3 Struttura generale di un neurone artificiale con n input. 24
3.4 Confronto tra le principali funzioni di attivazione. 25
3.5 Funzioni di attivazione e relative derivate. 26
3.6 Rete neurale semplice con struttura $\mathbb{R}^2 \to \mathbb{R}^3 \to \mathbb{R}^1$. 26
3.7 Esempio di DNN con 4 strati nascosti. 28
3.8 Flusso di derivazione nella backpropagation. 30
3.9 Confronto tra SGD e GD. 32
3.10 Confronto tra algoritmi di ottimizzazione. 33
3.11 Underfitting, ideal fit e overfitting su un dataset fisso. 34
3.12 Dropout. 35
3.13 Early stopping. 36

4.1 Architettura DNN: previsione $\hat{y}_t$ a partire dai valori passati. 40
4.2 Architettura RNN per l'elaborazione ricorrente di una sequenza di input. 40
4.3 Rappresentazione *unrolled* di una RNN. 41
4.4 Sviluppo temporale di una rete neurale ricorrente. 42
4.5 Interno di una cella ricorrente. 43
4.6 Configurazioni input-output nelle RNN. 44
4.7 Backpropagation Through Time 45
4.8 Funzionamento cella LSTM. 47
4.9 Struttura interna di una cella LSTM. 48
4.10 Struttura interna di una cella GRU. 50
4.11 Architettura CNN-1D. 51
4.12 Architettura del modello N-BEATS. Fonte: [34]. 54

5.1 Suddivisione temporale del dataset per i modelli econometrici e neurali. 57

5.2 ACF e PACF della serie dei prezzi dell'S&P 500. 58
5.3 Rendimenti logaritmici mensili ret_m (training set). 59
5.4 ACF e PACF della serie ret_m dell'S&P 500. 60
5.5 ACF e PACF dei residui al quadrato del modello AR(1). 62
5.6 QQ–plot dei residui standardizzati del modello ARMA(0,0)–GARCH(1,1) con innovazioni Normali. 65
5.7 Confronto tra RV_m e previsioni del modello gjrGARCH(1,1)–Student-t. 69
5.8 Schema della finestra temporale utilizzata per costruire gli input della MLP. . . 71
5.9 Andamento di training loss e validation loss durante l'addestramento della MLP. 73
5.10 Architettura della MLP ottimale selezionata. 73
5.11 Andamento di training loss e validation loss durante l'addestramento della LSTM. 75
5.12 Architettura della LSTM ottimale selezionata. 75
5.13 Andamento di training loss e validation loss per la GRU selezionata. 77
5.14 Architettura della GRU ottimale selezionata. 77
5.15 Andamento di training loss e validation loss durante l'addestramento della CNN-1D. 79
5.16 Architettura della CNN-1D ottimale selezionata. 79
5.17 Andamento di training loss e validation loss durante l'addestramento del modello N-BEATS selezionato. 81
5.18 Architettura della N-BEATS ottimale selezionata. 81
5.19 Confronto tra RV_m e previsioni dei modelli GARCH ed LSTM (S&P 500). . . . 83
A.1 Confronto delle metriche OOS per le diverse serie e architetture considerate. . . 94
A.2 Risultati grafici per XAUUSD Curncy. 95
A.3 Risultati grafici per NKY Index. 96
A.4 Risultati grafici per GBPUSD Curncy. 97
A.5 Risultati grafici per USGG10YR. 98
A.6 Risultati grafici per KO US Equity. 99
A.7 Risultati grafici per XOM US Equity. 100
A.8 Risultati grafici per G IM Equity. 101
A.9 Risultati grafici per 7203 JP Equity. 102

Elenco delle tabelle

3.1 Tensori con relative dimensioni. 37

4.1 Esempio di costruzione del dataset supervisionato. 39
4.2 Confronto tra reti neurali feedforward e ricorrenti. 42

5.1 Statistiche descrittive dell'indice S&P 500 (SPX Index, *PX_LAST*) 58
5.2 Risultati del test ADF sui livelli. 59
5.3 Risultati dei test statistici si ret_m dell'S&P 500 (training set). 60
5.4 Risultati del BDS Test sui residui del modello per la media. 62
5.5 Risultati del TNN Test sui residui del modello per la media. 62
5.6 Risultati del test ARCH-LM sui residui del modello per la media. 63
5.7 Stima del modello AR(1)–GARCH(1,1) con innovazioni Normali. 63
5.8 Stima del modello ARMA(0,0)–GARCH(1,1) con innovazioni Normali 64
5.9 Test diagnostici sul modello ARMA(0,0)–GARCH(1,1). 64
5.10 Confronto tra modelli sGARCH con diverse distribuzioni delle innovazioni. . . 66
5.11 Confronto tra modelli GARCH asimmetrici in base all'RMSE_oos. 67
5.12 Stima dei parametri del modello gjrGARCH(1,1) con innovazioni Student-t. . . 68
5.13 Test diagnostici sui residui del modello gjrGARCH(1,1)–Student-t. 68
5.14 Metriche IS e OOS del gjrGARCH(1,1)–Student-t. 69
5.15 Struttura del dataset di partenza (S&P 500). 70
5.16 Dataset supervisionato 2D con $L = 2$. 70
5.17 Dataset supervisionato 3D con $L = 2$. 74
5.18 Confronto complessivo delle performance previsive IS e OOS (S&P 500). . . . 82
5.19 Serie finanziarie analizzate. 83
5.20 Prestazioni IS e OOS dei modelli per ciascuna serie. 84

Elenco dei simboli

Variabile	Significato
η	Learning rate dell'algoritmo di ottimizzazione.
$\hat{y}_t$	Output (previsione) della rete neurale al tempo t.
$\mathbb{E}[\cdot]$	Valore atteso di una variabile casuale.
$\mathscr{L}(\cdot)$	Funzione di perdita.
MAE	Mean Absolute Error.
MSE	Mean Squared Error.
RMSE	Root Mean Squared Error.
RV_m	Volatilità realizzata su base mensile.
$\mathrm{Var}[\cdot]$	Varianza di una variabile casuale.
$\phi(\cdot)$	Funzione di attivazione.
σ_t	Volatilità condizionata al tempo t.
σ_t^2	Varianza condizionata al tempo t.
FX	Foreign Exchange (mercato dei cambi valutari).
IS	In-sample (periodo di stima o training).
OOS	Out-of-sample (periodo di test).
$\tilde{c}_t$	Stato candidato della cella LSTM.
$\tilde{h}_t$	Stato candidato della GRU.
$\widehat{\mathrm{RV}}_m$	Previsione della volatilità realizzata mensile.
$b^{(i)}$	Componente di backcast del blocco i in N-BEATS.
c_t	Stato della cella LSTM al tempo t.
$f^{(i)}$	Componente di forecast del blocco i in N-BEATS.
f_t	Forget gate di una cella LSTM.
h_t	Stato nascosto al tempo t.

i_t	Input gate di una cella LSTM.
o_t	Output gate di una cella LSTM.
R_t, ret_t	Rendimento logaritmico al tempo t.
rg_t	Reset gate di una GRU.
ug_t	Update gate di una GRU.
W, U, b	Matrici dei pesi e termini di bias nelle reti neurali.
X_t	Vettore di input della rete neurale al tempo t.
y_t	Target di previsione al tempo t.